

tpageHy118



UT San Antonio™

ELEVATE Academy

**Evaluación Basada en Evidencia y Herramientas
de IA para la Educación**

UT San Antonio



ELEVATE ACADEMY

ELEVATE (*Evidence-Led Evaluation & AI Tools for Education*, «Evaluación Basada en Evidencia y Herramientas de IA para la Educación») cuenta con el apoyo de la National Science Foundation bajo la adjudicación 2518973, «*Supporting Student Learning and Success in Early Undergraduate Mathematics Courses via AI-Enhanced Education*» («Apoyo al Aprendizaje y al Éxito Estudiantil en Cursos de Matemáticas de Pregrado Inicial mediante Educación Mejorada con IA»). El programa NSF IUSE: EDU financia proyectos de investigación y desarrollo orientados a mejorar la eficacia de la educación STEM para todos los estudiantes.

Juan B. Gutiérrez, Ph.D. Catedrático de Matemáticas, juan.gutierrez3@utsa.edu

The University of Texas at San Antonio

22 de abril de 2026

Índice general

1. Introducción	3
Introducción	3
2. Configuración del Entorno	7
2.1. Obligatorio: Instalar Python	7
2.2. Opcional (pero <u>muy</u> recomendado): Instalar una distribución de L ^A T _E X	9
2.3. Obligatorio: Instalar Visual Studio Code (VSC)	10
2.4. Obligatorio: Extensiones de Visual Studio Code	10
2.5. Opcional: Windows Terminal	11
2.6. Obligatorio: Instalar Git	12
2.7. Obligatorio: Abrir Git Bash (o cualquier terminal)	12
2.8. Obligatorio: Abrir el proyecto ELEVATE Academy en Visual Studio Code	13
2.9. Obligatorio: Instalar bibliotecas de Python	13
2.10. Opcional: Jupyter Notebook	13
2.11. Opcional: Comandos de Python en la Terminal	14
2.12. Solución de problemas	15
2.13. Opcional: Compiladores de C/C++	15
2.14. Opcional: Instalación de PostgreSQL en Linux	15
2.14.1. Instalar las utilidades requeridas	16
2.14.2. Agregar el repositorio de PostgreSQL	16
2.14.3. Importar la clave de firma del repositorio	16
2.14.4. Instalar PostgreSQL	16
2.14.5. Verificar la instalación	16
2.14.6. Iniciar el servidor de base de datos	17
2.14.7. Acceder al intérprete de comandos de PostgreSQL	17
3. Interfaces de línea de comandos, entornos virtuales y scripting de intérprete	18
3.1. ¿Qué es un archivo?	18
3.1.1. Archivos y texto	18
3.1.2. El sistema de archivos: carpetas y rutas	18
3.1.3. El directorio de trabajo	19
3.2. Introducción: ¿qué es una interfaz de línea de comandos?	19
3.2.1. Una analogía concreta	19

3.2.2.	Por qué las personas que desarrollan y las que investigan usan la CLI	20
3.2.3.	Los dos mundos principales de la CLI	20
3.2.4.	Sus primeros comandos	20
3.3.	Dos mundos: Windows y Linux/Mac	20
3.3.1.	Rutas y separadores	21
3.3.2.	Scripts y ejecutabilidad	21
3.3.3.	Variables de entorno y PATH	21
3.4.	WSL: un entorno Linux dentro de Windows	22
3.4.1.	Qué le ofrece WSL	22
3.4.2.	Instalación de WSL	22
3.4.3.	La solución elegante: Visual Studio Code y la extensión WSL	22
4.	Introducción a los entornos de programación	24
4.1.	Antes del editor: cómo escribir su primer programa en un archivo de texto	24
4.2.	Introducción	26
4.2.1.	Lenguajes compilados	26
4.2.2.	Lenguajes de scripting	27
4.2.3.	Ejemplos comparativos	28
4.2.4.	¿Qué es Python?	29
4.2.5.	Instalación de Python	29
4.2.6.	Ejecución de comandos en Python	30
4.2.7.	El editor de Python de VSC: ejecución de un programa simple	31
4.2.8.	Ejecución de funciones	34
4.3.	Introducción a NumPy	35
4.3.1.	Operaciones matriciales	36
5.	Entornos virtuales de Python y automatización del flujo de trabajo	39
5.1.	Gestión tradicional de paquetes: pip	39
5.1.0.1.	Windows (CMD o PowerShell)	39
5.1.0.2.	Linux / WSL	40
5.1.1.	Activación de un entorno virtual	40
5.1.1.1.	Windows	40
5.1.1.2.	Linux / WSL	40
5.1.2.	Instalación de paquetes	41
5.1.3.	Desactivación	41
5.1.4.	Instalación de las bibliotecas requeridas por ELEVATE Academy	41
5.2.	Gestión moderna de paquetes: uv	42
5.2.1.	Por qué uv es relevante hoy	42
5.2.2.	Qué ofrece uv	42
5.2.3.	Instalación de uv	43
5.2.3.1.	Windows (PowerShell)	43
5.2.3.2.	Linux / WSL	43

5.2.4.	Creación y gestión de entornos con <code>uv</code>	44
5.2.4.1.	Creación de un entorno	44
5.2.4.2.	Activación de un entorno	44
5.2.4.3.	Instalación de paquetes	44
5.2.4.4.	Declaración de dependencias con <code>pyproject.toml</code>	44
5.2.5.	<code>requirements.txt</code> frente a <code>uv.lock</code>	46
5.3.	Nivel 1: archivos de script	46
5.3.1.	La solución de Windows: archivos por lotes	46
5.3.1.1.	<code>ActivateEnv.bat</code>	46
5.3.1.2.	<code>ELEVATE Academy.bat</code>	47
5.3.2.	La solución de Linux: scripts de intérprete de comandos	48
5.3.2.1.	Comprensión de la sintaxis de los scripts de intérprete de comandos	48
5.3.2.2.	<code>ActivateEnv.sh</code>	48
5.3.2.3.	<code>ELEVATE Academy.sh</code>	49
5.3.3.	Hacer ejecutable un script	50
5.3.4.	Añadir sus scripts al <code>PATH</code>	50
5.3.5.	El enfoque con <code>uv</code> : <code>ActivateUVEnv</code>	50
5.3.5.1.	<code>ActivateUVEnv.bat</code> (Windows)	51
5.3.5.2.	<code>ActivateUVEnv</code> : función de intérprete de comandos para Linux / WSL	52
5.4.	La diferencia crítica: ejecutar frente a cargar con <code>source</code>	53
5.4.1.	Intérpretes de comandos hijos	53
5.4.2.	Cargar con <code>source</code> : ejecución en el intérprete de comandos actual	53
5.4.3.	El paralelo en Windows	54
5.5.	Nivel 2: funciones de intérprete de comandos, el enfoque elegante de Linux	54
5.5.1.	¿Qué es una función de intérprete de comandos?	54
5.5.2.	Definición de la función <code>ActivateEnv</code>	55
5.5.3.	Definición de la función <code>ELEVATE Academy</code>	55
5.5.4.	Activación de las nuevas funciones	56
5.5.5.	Por qué Windows no tiene un equivalente limpio	56
5.6.	Juntando todas las piezas	56
5.6.1.	La experiencia completa en WSL	56
5.7.	Hoja de referencia rápida	57
5.8.	Resumen	57
6.	Operaciones Numéricas	61
6.1.	Bits, bytes y punto flotante	61
6.1.1.	El estándar IEEE para aritmética de punto flotante	62
6.1.2.	Un bestiario de anomalías de punto flotante	63
6.2.	Introducción a NumPy	66
6.2.1.	Operaciones matriciales	67
6.3.	Tareas específicas para este laboratorio	69

7. Introducción a L^AT_EX	71
7.1. Su primer documento L ^A T _E X	71
7.2. Un ejemplo más completo	72
7.3. Estructura del documento	76
7.4. Matemáticas en L ^A T _E X: una hoja de referencia	78
7.4.1. Notación matemática común	78
7.5. Figuras	81
7.6. Tablas	81
7.7. Bibliografía	81
7.8. LaTeX Workshop en Visual Studio Code	82
7.9. Resumen	82
8. Control de versiones con Git y GitHub	84
8.1. Configuración de la autenticación SSH para GitHub en Linux	84
8.1.1. Generar una clave SSH	84
8.1.2. Iniciar el agente SSH	84
8.1.3. Agregar la clave pública a GitHub	85
8.1.4. Verificar la conexión	85
8.1.5. Clonar un repositorio utilizando SSH	85
8.2. Configuración de la autenticación SSH para GitHub en Windows . .	85
8.2.1. Abrir Git Bash	85
8.2.2. Generación de una clave SSH	86
8.2.3. Iniciar el agente SSH	86
8.2.4. Agregar la clave pública a GitHub	87
8.2.5. Verificación de la conexión	87
8.2.6. Clonación de un repositorio utilizando SSH	88
8.2.7. Push a múltiples repositorios remotos	88
8.2.7.1. Conceptos	89
8.2.7.2. Listar los remotos registrados	89
8.2.7.3. Agregar un segundo remoto	90
8.2.7.4. Push a cada remoto	90
8.2.7.5. Comprobar si un remoto existe antes de hacer push	91
8.2.7.6. Gestión de remotos	91
8.2.7.7. Alternativa: un único remoto con múltiples URL de push	91
8.3. Resumen	92
9. Infraestructura local de LLM con Ollama	94
9.1. Introducción	94
9.2. Inventario de hardware	94
9.2.1. Dell Precision 7770 (host de aplicaciones)	94
9.2.2. NVIDIA Spark (servidor de LLM)	95
9.3. Topología de red	95
9.3.1. Conmutador frente a enrutador	96
9.3.2. Filtrado por dirección MAC	96
9.3.3. Identificación de interfaces de red	96

9.4.	Instalación de Ollama	97
9.4.1.	Linux (Spark)	97
9.4.2.	Windows (Dell)	97
9.4.3.	Entornos Conda	97
9.5.	Configuración de Ollama para acceso de red	98
9.6.	Gestión del presupuesto de memoria y de la ventana de contexto	98
9.6.1.	La fórmula rectora	98
9.6.2.	El patrón Modelfile para modelos grandes	99
9.6.2.1.	Linux (Spark y WSL)	100
9.6.2.2.	Windows (PowerShell)	100
9.6.3.	Capacidad de agentes del hardware	100
9.6.4.	Modelos con razonamiento y presupuestos de tokens	100
9.7.	Configuración de claves de API	101
9.8.	El marco FOO y la viabilidad de agentes	102
9.8.1.	Número mínimo de agentes	102
9.8.2.	Puntaje de viabilidad de agentes	103
9.8.3.	Configuraciones de referencia	103
9.9.	Configuración de máquina única (Dell)	104
9.9.1.	Variable de entorno	104
9.9.2.	Selección de modelos para el desarrollo de máquina única	105
9.10.	Configuración de dos máquinas	105
9.10.1.	Arquitectura	105
9.10.2.	Variables de entorno	105
9.10.3.	Verificación de conectividad	106
9.11.	Pruebas de capacidad	106
9.11.1.	Consideraciones de plataforma	107
9.11.2.	Referencia de la CLI de Ollama	107
9.12.	Paridad de despliegue	108
9.13.	Configuración de Ollama con una GUI en Ubuntu	108
9.13.1.	Paso 1: Instalar el backend de Ollama	108
9.13.2.	Paso 2: Desplegar Open WebUI mediante Docker	109
9.13.3.	Paso 3: Acceder a la interfaz	109
9.13.4.	Consideraciones técnicas	109
10.	Evaluación y Registro de Progreso	110
10.1.	Obligatorio: Incorporación al Repositorio	111
10.2.	Esencial: Configuración del Entorno y Cadena de Herramientas	111
10.3.	Deseable: Comprensión Técnica	112
10.4.	Opcional: Infraestructura Local de LLM (Ollama)	113

Capítulo 1

Introducción

Información

Este documento **no** está pensado para leerse de principio a fin. El profesorado y el personal que ya se sientan cómodos con L^AT_EX, Git y Python pueden pasar directamente a los capítulos de unidades del curso. Quienes sean nuevos en una o más de las herramientas de apoyo deberían consultar primero el capítulo de infraestructura pertinente. Utilice la tabla de rutas de las páginas siguientes para identificar qué capítulos corresponden a su perfil.

Convenciones utilizadas en este documento

Cuadros destacados. En el texto aparecen cuatro tipos de cuadros destacados:

- **Información** (borde verde azulado): material de referencia y contexto neutral.
- **Consejo** (borde azul): orientación práctica y atajos.
- **Advertencia** (borde magenta): precauciones críticas que, si se ignoran, provocarán errores o pérdida de datos.
- **Nota** (borde naranja): elementos que requieren atención pero no son críticos.

Listados de código. Todos los comandos que se pretende escribir en una terminal se muestran en bloques de código con tipografía monoespaciada. Los comandos de Windows usan un fondo oscuro; los comandos de Linux/macOS usan un fondo más claro. Los comandos que funcionan de forma idéntica en todas las plataformas usan el estilo de listado neutral.

Referencias cruzadas. El texto subrayado en azul es un hipervínculo clickeable. Las referencias a capítulos, secciones, figuras y tablas también son clickeables en el PDF.

Cómo está organizado este documento

El documento se divide en dos partes. La primera parte cubre la infraestructura técnica necesaria para trabajar con el repositorio del currículo: configurar el entorno de software, usar la interfaz de línea de comandos, escribir en $\text{\LaTeX}$, gestionar versiones con Git y GitHub, y trabajar con herramientas locales de modelos de lenguaje. Estos capítulos están escritos para profesorado y personal que quizás no tengan formación en desarrollo de software; los colaboradores experimentados pueden oírearlos o saltárselos por completo.

La segunda parte contiene los capítulos de unidades del curso. Cada unidad corresponde a un curso del catálogo del departamento y documenta el título del curso, los resultados de aprendizaje del estudiante (SLO), el esquema temático y la estrategia de evaluación. Las unidades están numeradas según su identificador de catálogo (por ejemplo, la unidad 1023 corresponde a MATH 1023). Los capítulos dentro de cada parte son independientes; un lector interesado únicamente en un curso específico no necesita leer ningún otro capítulo de unidad.

Dependencias entre capítulos

La Figura 1 muestra la estructura de dependencias de los capítulos de infraestructura. Una flecha continua indica una dependencia estricta: el capítulo de destino requiere que el de origen se haya completado primero. Una flecha discontinua indica únicamente un orden recomendado.

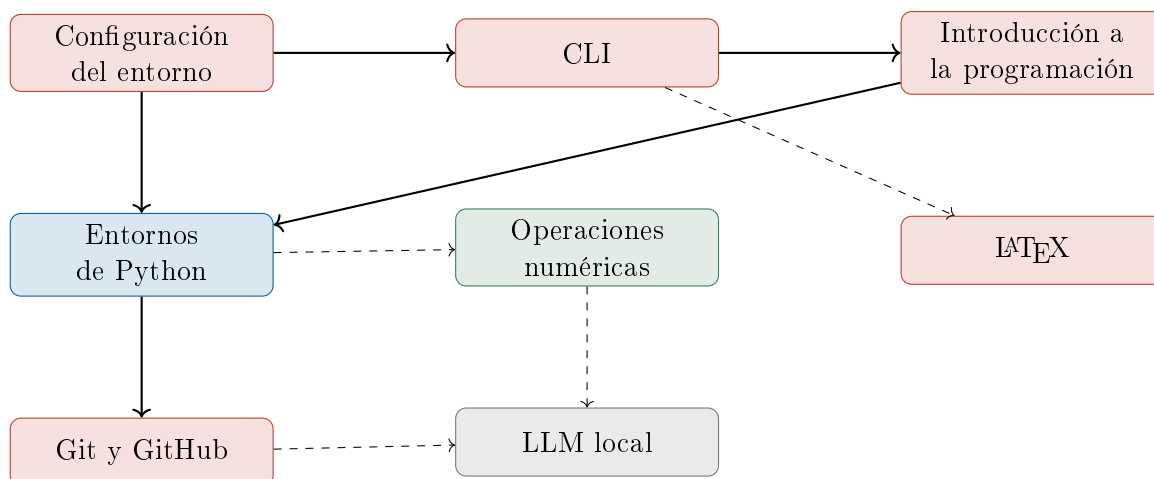


Figura 1: Grafo de dependencias para los capítulos de infraestructura. Las flechas continuas indican dependencias estrictas; las flechas discontinuas indican el orden recomendado. El color de los nodos codifica la categoría: **Obligatorio**, **Esencial**, **Deseable**, **Opcional**.

¿Qué capítulos necesita?

Utilice la Tabla 1.1 para identificar su ruta de lectura según su familiaridad con las herramientas de apoyo. Los capítulos marcados como **Obligatorio** deben completarse antes de trabajar con el repositorio. **Ojear** significa leer los títulos de las secciones y realizar los ejercicios, pero omitir la prosa explicativa. **Omitir** significa que el capítulo es opcional para su perfil.

Cuadro 1.1: Ruta de lectura recomendada según la experiencia previa con el conjunto de herramientas.

Perfil	Config. entorno	CLI	Intro. prog.	Entornos Python	Oper. numéricas	L ^A T _E X	Git	LLM
Nuevo en todas las herramientas	•	•	•	•	•	•	•	O
Familiarizado con L ^A T _E X, nuevo en Python y Git	•	•	•	•	O	–	•	O
Familiarizado con Python y Git, nuevo en L ^A T _E X	•	O	O	O	–	•	–	O
Experimentado con todas las herramientas	O	–	–	O	–	–	–	O

• = Obligatorio O = Ojear – = Omitir

Registro de progreso

El capítulo final, Capítulo 10 (Evaluación y registro de progreso), organiza los elementos clave de finalización de todos los capítulos de infraestructura en cuatro categorías: Obligatorio, Esencial, Deseable y Opcional. Abra el formulario PDF rellenable en Adobe Acrobat Reader para registrar y guardar sus respuestas. Utilícelo durante todo el proceso de incorporación como lista de verificación de preparación.

Tiempos estimados de finalización

La Tabla 1.2 muestra los tiempos estimados de finalización por capítulo de infraestructura. Los tiempos suponen que no se ha tenido exposición previa al tema; los colaboradores con formación en un área determinada la completarán más rápido. Los capítulos de unidades del curso se leen como material de referencia y no están incluidos en esta tabla; el tiempo de lectura de una sola unidad es aproximadamente de 20 a 30 minutos.

Cuadro 1.2: Tiempo estimado de finalización por capítulo de infraestructura.

Capítulo	Título	Tiempo estimado
1	Configuración del entorno	2 horas
2	Interfaces de línea de comandos y scripting de shell	2 horas
3	Introducción a los entornos de programación	2 horas
4	Entornos virtuales de Python y automatización de flujos de trabajo	1.5 horas
5	Operaciones numéricas con NumPy	2 horas
6	Introducción a L ^A T _E X	1.5 horas
7	Control de versiones con Git y GitHub	1.5 horas
8	Infraestructura local de LLM con Ollama	1 hora (colaboradores) / 3 horas (administradores)
9	Evaluación y registro de progreso	Continuo
Total (ruta completa)		≈ 13.5–15.5 horas

Capítulo 2

Configuración del Entorno

Este capítulo recorre la instalación de todo el software requerido para el proyecto ELEVATE Academy. Complete las secciones obligatorias en orden antes de la primera sesión. Las secciones opcionales pueden completarse en cualquier momento. Las instrucciones están escritas principalmente para Windows, pero incluyen pasos equivalentes para macOS y Linux a lo largo del texto.

La familiaridad con la línea de comandos y con las rutas de archivos es útil, pero no es requerida aquí. Estos conceptos se cubren completamente en el Capítulo 3, que puede leer después de completar este capítulo.

2.1. Obligatorio: Instalar Python

Instale cualquier versión reciente de Python desde <https://www.python.org/downloads/>. Vaya a sus descargas y haga doble clic en el archivo de instalación para iniciar la instalación. Por defecto, el instalador de Python para Windows coloca sus ejecutables en el directorio AppData del usuario, de modo que no requiere permisos administrativos. Esto funciona para la mayoría de los escenarios. Si usted es el único usuario en el sistema, podría preferir colocar Python en un directorio de nivel superior (por ejemplo, `C:\Python` o `/usr/local/bin`) para tener una ruta más corta a los binarios (a veces necesitará eso). Dependiendo de sus preferencias, seleccione «*Install Now*» o «*Customized installation*» (mi preferencia). Asegúrese de seleccionar «*Add python.exe to path*»; esto le ahorrará algunos dolores de cabeza más adelante.

En la ventana «Optional Features» seleccione todas las características.

Seleccione «*Install Python X.XX for all users*» si puede y lo desea. Esto requiere privilegios administrativos. Seleccione la carpeta de su elección para la ubicación de instalación.

Para PC: Si no agregó `python.exe` al PATH durante la instalación, espere hasta que la instalación se complete. Luego abra el Explorador de archivos y haga clic derecho sobre «Este equipo». Seleccione «Propiedades» del menú contextual. En la ventana del Sistema que se abre, haga clic en «Configuración avanzada del sistema» en la barra lateral izquierda. En la nueva ventana, asegúrese de que la

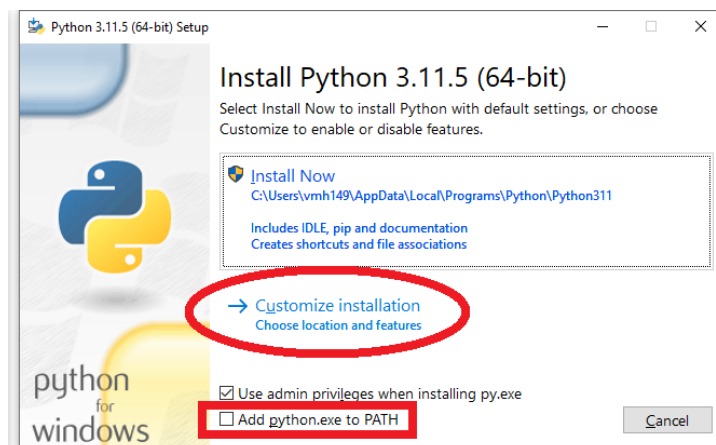


Figura 1: Instalador de Python mostrando la casilla «Add python.exe to PATH» en la parte inferior.

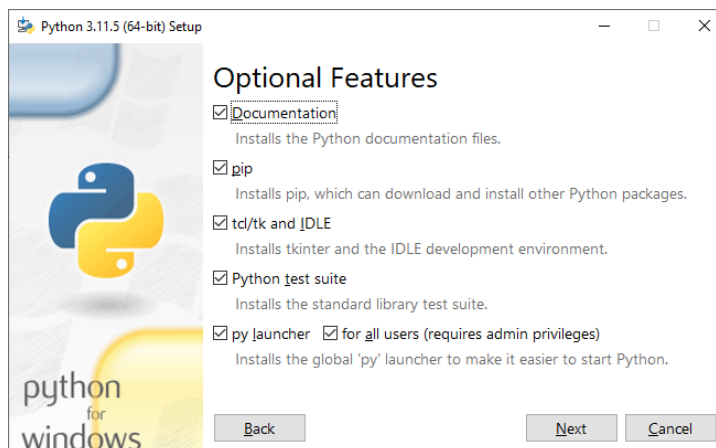


Figura 2: Pantalla de características opcionales de Python: seleccione todas las características.

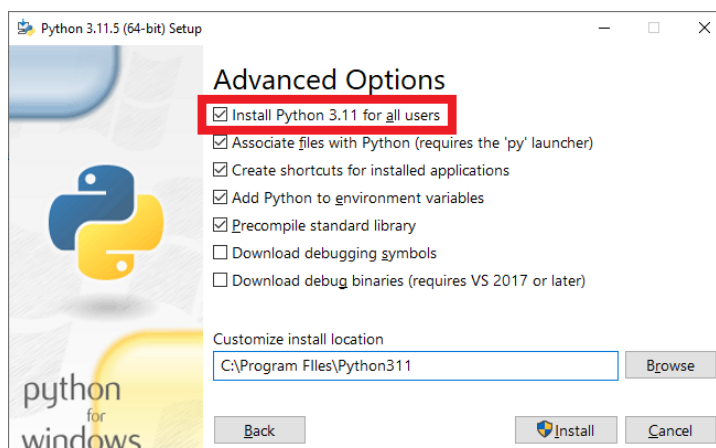


Figura 3: Pantalla de opciones avanzadas de Python mostrando «Install for all users» seleccionado.

pestaña «Opciones avanzadas» esté seleccionada y haga clic en el botón «Variables de entorno» cerca de la parte inferior. Bajo la sección «Variables del sistema», busque y haga doble clic en la variable llamada «Path». Esto abrirá una ventana donde podrá agregar la ruta al ejecutable de Python manualmente.

Para Mac: Si no se aseguró de que Python fuera agregado al PATH de su sistema durante la instalación, puede hacerlo manualmente usando los siguientes pasos en una Mac. Después de que la instalación esté completa, abra la aplicación Terminal. Ingrese el comando `which python3` o `which python` para encontrar la ruta al binario de Python instalado. Copie esa ruta. Luego abra su archivo de configuración del intérprete de comandos en un editor de texto. Este será típicamente `~/.zshrc` si está usando el shell Zsh (predeterminado en versiones más recientes de macOS) o `~/.bash_profile` si está usando Bash. Agregue la línea `export PATH="/path/to/python:$PATH"`, reemplazando `/path/to/python` con la ruta que copió. Guarde el archivo y cierre el editor. En la Terminal, ejecute `source ~/.zshrc` o `source ~/.bash_profile` dependiendo del archivo que editó, para que el nuevo PATH se cargue en su entorno.

2.2. Opcional (pero muy recomendado): Instalar una distribución de L^AT_EX

L^AT_EX es un sistema de composición tipográfica de alta calidad diseñado para la creación de documentos técnicos y científicos. En lugar de formatear el texto visualmente, los autores escriben archivos fuente en texto plano usando un lenguaje de marcado que describe la estructura y el significado del contenido (como secciones, ecuaciones, y referencias). Estos archivos fuente se compilan luego en documentos profesionalmente formateados (típicamente PDFs). L^AT_EX destaca en la composición de ecuaciones, la gestión de referencias cruzadas, y la producción de salida consistente y de calidad de publicación, lo que lo convierte en la herramienta estándar para muchos campos en ciencia, ingeniería y matemáticas.

Información

L^AT_EX es particularmente adecuado para la interacción con los Modelos Grandes de Lenguaje

L^AT_EX es particularmente adecuado para la interacción con los Modelos Grandes de Lenguaje (LLMs) porque es un lenguaje de marcado puramente textual y declarativo que se asemeja estrechamente al código fuente estructurado. A diferencia de los formatos de documento binarios (por ejemplo, Word o PDF), un documento de L^AT_EX expone su estructura lógica completa—secciones, ecuaciones, entornos y referencias—en texto plano. Esto lo hace directamente interpretable, editable y generable por los LLMs, que están fundamentalmente entrenados y optimizados para texto y código. Como resultado, los LLMs pueden asistir de manera fiable en tareas como la redacción de contenido matemático, la refactorización de estructura, la verificación de notación e incluso la depuración de problemas de

compilación. En este sentido, $\text{\LaTeX}$ sirve como una interfaz natural entre el razonamiento matemático humano y la generación asistida por máquina, permitiendo flujos de trabajo de documentos precisos, reproducibles y automatizables.

$\text{\LaTeX}$ es requerido si planea compilar documentos `.tex` localmente. Los documentos del programa se mantienen como archivos fuente $\text{\LaTeX}$ en el repositorio. Para la autoría y compilación de sus propios documentos, instale la distribución para su plataforma.

Cuadro 2.1: Distribuciones de $\text{\LaTeX}$ recomendadas por plataforma

Plataforma	Distribución	Descarga
Windows	MiKTeX	https://miktex.org
macOS	MacTeX	https://www.tug.org/mactex/
Linux (Ubuntu/Debian)	TeX Live	<code>sudo apt install texlive-latex-extra texlive-fonts-recommended latexmk</code>

Verifique la instalación en cualquier plataforma con:

```
pdflatex --version
```

El uso de $\text{\LaTeX}$ para la preparación de documentos, incluyendo una hoja de referencia de notación matemática, se cubre en el Capítulo 7.

2.3. Obligatorio: Instalar Visual Studio Code (VSC)

Si usted es nuevo en Python, instale Visual Studio Code (VSC): Hay muchas opciones para escribir código Python, y usted podría tener una favorita. Debo elegir una para impartir la instrucción, y la experiencia me ha mostrado que VSC ofrece la menor cantidad de fricción.

En VSC, hay dos opciones principales: «*User Installer*» y «*System Installer*». Use el User Installer si desea instalar solo para el usuario actual. Para todos los usuarios, use el System Installer; yo prefiero esta opción porque instala VSC en `C:\Program Files\Microsoft VS Code`.

Visual Studio Code está disponible para Windows, macOS y Linux; una alternativa idéntica de código abierto llamada VSCodium está disponible en <https://vscodium.com>.

2.4. Obligatorio: Extensiones de Visual Studio Code

Seleccione el ícono de Extensiones a la izquierda e instale las siguientes bibliotecas de Visual Studio Code.

1. **Obligatorio:** Extensión de Python para Visual Studio Code.
2. **Opcional:** Instale también Jupyter (esto también instala Pylance, Jupyter Keymap, Jupyter Notebook Renderers, Jupyter SlideShow). Preste atención al editor de la extensión; use las de Microsoft.

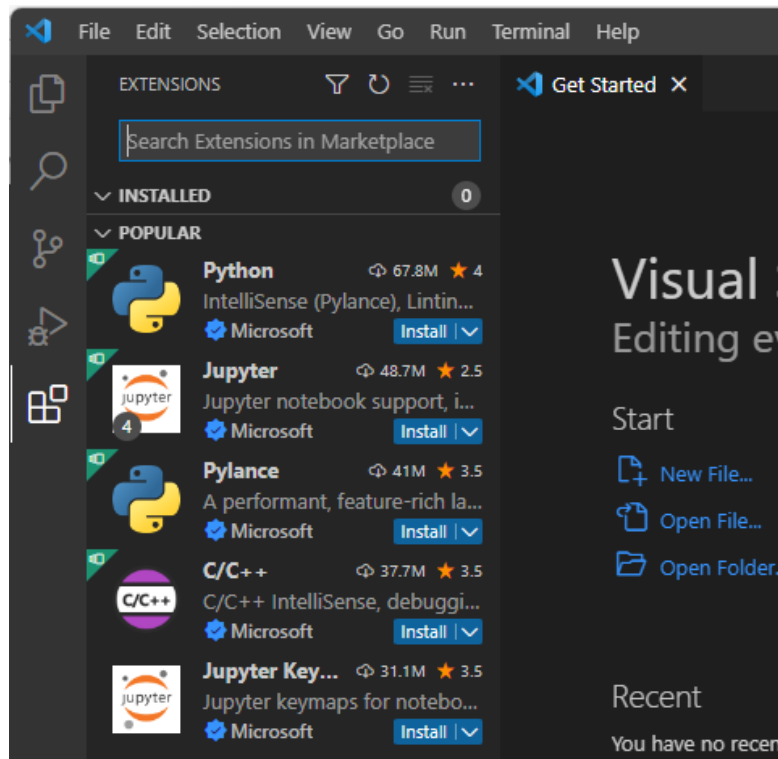


Figura 4: Mercado de Extensiones de VS Code mostrando las extensiones de Python y Jupyter de Microsoft.

3. **Opcional:** Instale C/C++ para Visual Studio Code, C/C++ Themes y C/C++ Extension Pack.
4. **Opcional:** Instale la extensión «**LaTeX Workshop**» de James Yu. Esta extensión proporciona resaltado de sintaxis, vista previa en vivo y compilación con un clic. Su uso se describe en la Sección 7.8.

2.5. Opcional: Windows Terminal

Para usuarios de Windows, instale Windows Terminal. Esta aplicación de Microsoft le permite utilizar una ventana de múltiples documentos con pestañas para diferentes consolas de comandos como PowerShell, Ubuntu, DOS, Git, etc. Necesitará esto si desea completar los pasos relacionados con Linux en la siguiente sección de este documento.

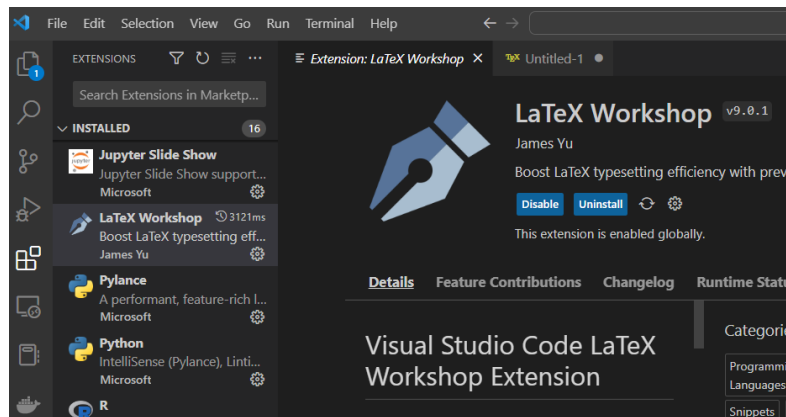


Figura 5: Página de la extensión LaTeX Workshop en VS Code.

2.6. Obligatorio: Instalar Git

Instale Git for Windows. Para Mac, instale Git for Mac. Linux también tiene una versión disponible. En la segunda pantalla, seleccione Visual Studio Code como su editor de Git. Use todas las demás opciones predeterminadas.

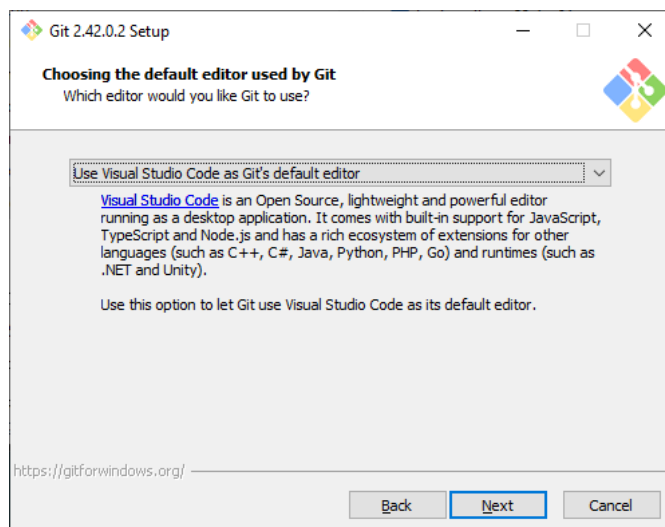


Figura 6: Pantalla de configuración de Git mostrando Visual Studio Code como el editor predeterminado.

2.7. Obligatorio: Abrir Git Bash (o cualquier terminal)

La línea de comandos le permite controlar su computadora escribiendo instrucciones, proporcionando acceso preciso a los archivos y características del sistema. Por ejemplo, al escribir `ls` en un sistema tipo Unix como Linux o macOS se lista los archivos y directorios en la ubicación actual, por ejemplo `ls /home/user/Documents` para mostrar el contenido de la carpeta Documents. En Windows, el comando `dir` cumple la misma función, de modo que `dir C:\Users\Alice\Desktop`

muestra lo que hay en el Escritorio. Para moverse entre directorios, se usa el comando `cd`; `cd Downloads` cambia el directorio de trabajo a Downloads, mientras que `cd ..` sube un nivel. Estos comandos permiten la navegación e inspección eficiente del sistema de archivos a través de la terminal.

Ahora que Git está instalado, clone el repositorio de ELEVATE Academy. Abra Git Bash (Windows) o una terminal (macOS/Linux), navegue a la carpeta donde desea almacenar el proyecto, y ejecute:

```
git clone https://github.com/biomathematicus/ELEVATE.git
```

Esto crea una carpeta llamada ELEVATE que contiene todos los materiales del taller. Solo necesita hacer esto una vez. Para actualizar su copia local más adelante, navegue dentro de la carpeta y ejecute `git pull`.

2.8. Obligatorio: Abrir el proyecto ELEVATE Academy en Visual Studio Code

Para abrir el proyecto ELEVATE Academy en VSC, abra una terminal, navegue dentro de la carpeta ELEVATE que clonó en la sección anterior, y ejecute:

```
cd ELEVATE
code .
```

El punto después de `code` instruye a VSC para usar la carpeta actual como la carpeta de trabajo. Todos los archivos del taller serán visibles en el explorador de archivos de VSC.

2.9. Obligatorio: Instalar bibliotecas de Python

Las bibliotecas de Python para el desafío de datos se instalan dentro de un entorno virtual. Complete el Capítulo 5 para configurar su entorno, luego instale las bibliotecas requeridas como se describe allí.

2.10. Opcional: Jupyter Notebook

Opcional: Ahora, cree un Jupyter Notebook.

1. Abundante información relevante sobre Jupyter en VSC está disponible en <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
2. Active el entorno que desea usar (véase el Capítulo 5).
3. Necesita adjuntar el kernel de su entorno virtual. Por ejemplo, si tengo un entorno virtual llamado `venv`:

```
python -m ipykernel install --user --name=venv
```

4. Abra VS Code en la carpeta de su proyecto (véase la Sección 2.8).
5. En VSC, ejecute el comando **Create:New Jupyter Notebook** desde la Paleta de Comandos (Ctrl+Shift+P) o creando un nuevo archivo .ipynb en su espacio de trabajo.
6. Podría recibir una alerta de seguridad respecto al firewall. Permita el acceso.

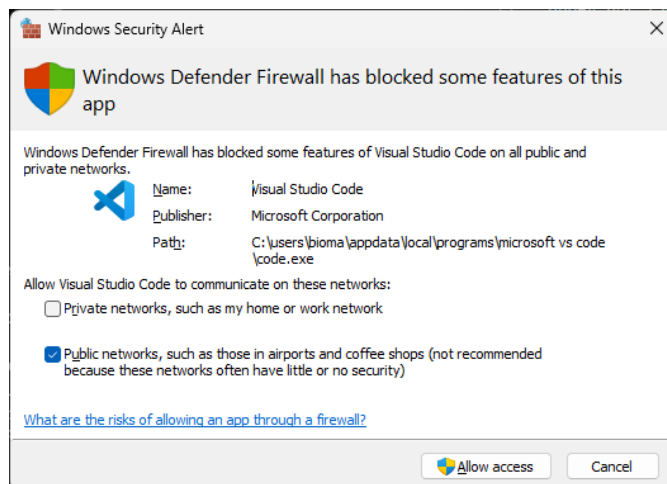


Figura 7: Alerta del Firewall de Windows Defender: permita el acceso para Visual Studio Code.

7. Después de ingresar un comando, podría ver el siguiente mensaje:

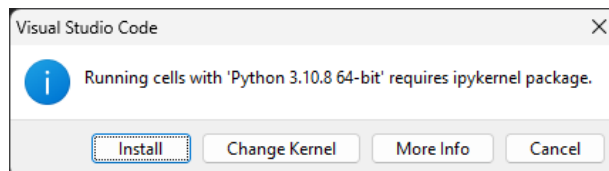


Figura 8: Indicación de VS Code para instalar el paquete ipykernel: haga clic en Instalar.

Instálelo.

2.11. Opcional: Comandos de Python en la Terminal

Finalmente, usted puede simplemente ingresar comandos de Python en la Terminal. Abra un símbolo del sistema y escriba el siguiente comando:

```
python
```

Esto le permitirá ingresar comandos de Python, por ejemplo 1+1

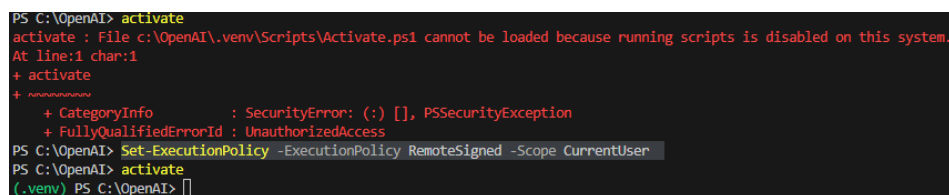
2.12. Solución de problemas

Los siguientes errores han sido reportados por participantes anteriores. Si encuentra un problema diferente, consulte el rastreador de problemas del repositorio de ELEVATE Academy en <https://github.com/biomathematicus/ELEVATE/issues>.

Según lo reportado por Lorena Roa de La Cruz, podría ver un error al intentar activar un entorno virtual. Esto se debe a la configuración de permisos. Si esto sucede, ejecute el comando:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Después de eso, ejecute el script `activate`.



```
PS C:\OpenAI> activate
activate : File c:\OpenAI\.venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on this system.
At line:1 char:1
+ activate
+ ~~~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\OpenAI> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\OpenAI> activate
(.venv) PS C:\OpenAI>
```

Figura 9: Error de política de ejecución de PowerShell y solución: ejecución de `Set-ExecutionPolicy` seguido de `activate`.

2.13. Opcional: Compiladores de C/C++

Si desea programar en C o C++, tiene varias opciones, incluyendo GCC, the GNU Compiler Collection, pero requiere cierto esfuerzo instalar los prerequisites. De Microsoft existe Visual Studio Community Edition (VSCE), que encapsula la complejidad de instalar múltiples compiladores (incluyendo el SDK del marco de trabajo .NET). Le recomiendo que instale VSCE con todas las «Workloads» en «Web & Cloud» y «Desktop & Mobile» (excepto Python).

2.14. Opcional: Instalación de PostgreSQL en Linux

Muchas distribuciones de Linux incluyen versiones antiguas de PostgreSQL en sus repositorios predeterminados. Para instalar la última versión estable, se recomienda utilizar el repositorio oficial de paquetes de PostgreSQL mantenido por el PostgreSQL Global Development Group.

El siguiente procedimiento instala la versión estable más reciente en sistemas Ubuntu o Debian.

2.14.1. Instalar las utilidades requeridas

Primero instale las utilidades requeridas para recuperar y verificar paquetes.

```
1 sudo apt update
2 sudo apt install curl ca-certificates
```

2.14.2. Agregar el repositorio de PostgreSQL

Agregue el repositorio oficial de PostgreSQL a las fuentes de paquetes del sistema.

```
1 sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt \
2 $(lsb_release -cs)-pgdg main" \
3 > /etc/apt/sources.list.d/pgdg.list'
```

2.14.3. Importar la clave de firma del repositorio

Importe la clave de firma del repositorio de PostgreSQL.

```
1 curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc \
2 | sudo gpg --dearmor -o /usr/share/keyrings/postgresql.gpg
```

Actualice la configuración del repositorio para referenciar la clave.

```
1 sudo nano /etc/apt/sources.list.d/pgdg.list
```

Modifique la línea para que se lea en una sola línea (el carácter \ significa retorno de línea; está aquí solo para indicar que todo es una sola línea; elimínelo y una las siguientes tres líneas de texto en una sola línea):

```
1 deb [signed-by=/usr/share/keyrings/postgresql.gpg] \
2 http://apt.postgresql.org/pub/repos/apt \
3 $(lsb_release -cs)-pgdg main
```

2.14.4. Instalar PostgreSQL

Actualice el índice de paquetes e instale la última versión de PostgreSQL.

```
1 sudo apt update
2 sudo apt install postgresql-16
```

2.14.5. Verificar la instalación

Confirme que PostgreSQL se instaló correctamente.

```
1 psql --version
```

La salida debería indicar la versión instalada de PostgreSQL.

2.14.6. Iniciar el servidor de base de datos

Si el servicio aún no está en ejecución, inicie el clúster de PostgreSQL.

```
1 sudo systemctl start postgresql
```

Verifique el estado del servidor:

```
1 sudo systemctl status postgresql
```

2.14.7. Acceder al intérprete de comandos de PostgreSQL

Para abrir la terminal interactiva de PostgreSQL:

```
1 sudo -u postgres psql
```

Salga del intérprete de comandos usando:

```
1 \q
```

Capítulo 3

Interfaces de línea de comandos, entornos virtuales y scripting de intérprete

3.1. ¿Qué es un archivo?

3.1.1. Archivos y texto

Un archivo es una secuencia nombrada de bytes almacenada en un dispositivo de almacenamiento. Desde la perspectiva de la computadora, todos los archivos son secuencias de bytes. Lo que distingue a un script de Python de una fotografía no es el mecanismo de almacenamiento sino la interpretación: un editor de texto interpreta los bytes como caracteres; un visor de imágenes los interpreta como valores de píxeles. Un archivo de Python es texto plano: cada carácter que usted escribe se almacena como un byte, y el intérprete de Python lee esos bytes como instrucciones. Por esta razón un documento de Word no puede servir como un programa de Python; Word almacena bytes de formato invisibles junto con el texto.

3.1.2. El sistema de archivos: carpetas y rutas

Cada archivo reside dentro de una carpeta (también llamada directorio). Las carpetas pueden contener otras carpetas, formando un árbol. La ubicación de cualquier archivo puede describirse mediante una *ruta*: una secuencia de nombres de carpetas separados por un delimitador, que termina con el nombre del archivo.

Cuadro 3.1: Sintaxis de rutas en cada sistema operativo.

Sistema operativo	Ruta de ejemplo
Windows	C:\Users\Alice\Documents\hello.py
macOS	/Users/alice/Documents/hello.py
Linux	/home/alice/Documents/hello.py

Windows utiliza barras invertidas y una letra de unidad; macOS y Linux utilizan barras diagonales y parten de una raíz /.

3.1.3. El directorio de trabajo

En cualquier momento, la línea de comandos está «dentro» de una carpeta en particular, llamada el *directorio de trabajo actual* (CWD, por sus siglas en inglés). Los comandos que hacen referencia a archivos sin una ruta completa se interpretan relativos a esa carpeta. Cuando usted escribe `python hello.py`, el intérprete busca `hello.py` en el CWD. Si el archivo se encuentra en otra ubicación, el comando falla. Comprender el CWD es la fuente más común de confusión para las personas que recién empiezan a usar la línea de comandos y la causa raíz de la mayoría de los errores de «archivo no encontrado».

La línea de comandos proporciona los comandos para navegar el sistema de archivos, inspeccionar su contenido y ejecutar archivos; estos se presentan en la siguiente sección.

«La línea de comandos es la interfaz más poderosa jamás inventada. También es la más intimidante. Este capítulo pretende cambiar eso.»

3.2. Introducción: ¿qué es una interfaz de línea de comandos?

Cuando usted utiliza una computadora, seguramente interactúa con ella a través de una **interfaz gráfica de usuario** (GUI, por sus siglas en inglés): ventanas, iconos, menús y botones sobre los que hace clic con el ratón. Las GUI son intuitivas y visualmente ricas, pero tienen una limitación fundamental: cada acción posible debe haber sido anticipada e implementada como un botón o elemento de menú por la persona que desarrolló el software. Si usted necesita hacer algo para lo que la GUI no fue diseñada, queda bloqueado.

Una **interfaz de línea de comandos** (CLI) resuelve esto al permitirle comunicarse con la computadora mediante *texto*. Usted escribe un comando; la computadora lo ejecuta y responde con texto. Esto no es un paso atrás; es un modo de interacción diferente y, en muchos contextos, más poderoso.

3.2.1. Una analogía concreta

Piense en la diferencia entre un restaurante con un menú fijo y un restaurante donde usted puede describir exactamente lo que desea al chef. Una GUI es el menú fijo: rápida, amigable y suficientemente buena para la mayoría de los clientes. Una CLI es hablar directamente con el chef: requiere más conocimiento de los ingredientes, pero los resultados pueden ser precisamente lo que usted necesita, y puede combinarlos de formas que el menú nunca anticipó.

3.2.2. Por qué las personas que desarrollan y las que investigan usan la CLI

- **Automatización.** Una secuencia de comandos puede guardarse en un archivo (llamado *script*) y reproducirse a la perfección cada vez.
- **Precisión.** Cada opción y parámetro es explícito y legible.
- **Acceso remoto.** Los servidores en la nube rara vez disponen de un escritorio gráfico; la CLI es a menudo la única vía de entrada.
- **Velocidad.** Cambiar el nombre de 10,000 archivos requiere un solo comando; hacerlo a mano en una GUI podría tomar horas.
- **Reproducibilidad.** Usted puede compartir un script con una persona colega y saber que obtendrá exactamente el mismo resultado.

3.2.3. Los dos mundos principales de la CLI

Este capítulo se centra en dos entornos de CLI:

Command Prompt / PowerShell de Windows Las herramientas CLI nativas en Microsoft Windows. Los archivos por lotes (*.bat*) y los scripts de PowerShell (*.ps1*) automatizan tareas aquí.

Bash (Linux / macOS) El intérprete de comandos dominante en sistemas Linux y macOS. Los scripts de intérprete (*.sh*) automatizan tareas aquí.

Estos dos mundos utilizan sintaxis diferentes, convenciones de ruta diferentes (barra invertida \ vs. barra diagonal /) y filosofías de scripting diferentes. Un objetivo clave de este capítulo es ayudarle a desplazarse con fluidez entre ellos.

3.2.4. Sus primeros comandos

La mejor manera de desmitificar la CLI es probarla. La Tabla 3.2 muestra comandos equivalentes en ambos entornos.

Consejo

En ambos entornos, presionar la tecla de **flecha arriba** recupera sus comandos anteriores, y presionar **Tab** autocompleta nombres de archivos y directorios. Solamente estos dos hábitos acelerarán notablemente su trabajo en la línea de comandos.

3.3. Dos mundos: Windows y Linux/Mac

Antes de escribir cualquier script, conviene comprender las diferencias filosóficas entre la forma en que Windows y Linux manejan la línea de comandos. Estas diferencias no son arbitrarias; reflejan décadas de historias de diseño divergentes.

Cuadro 3.2: Comandos CLI equivalentes: Windows vs. Linux/macOS

Tarea	Windows (CMD)	Linux / macOS (Bash)
Mostrar directorio actual	cd	pwd
Listar archivos	dir	ls -la
Cambiar de directorio	cd foldername	cd foldername
Subir un nivel	cd ..	cd ..
Crear una carpeta	mkdir myfolder	mkdir myfolder
Eliminar un archivo	del file.txt	rm file.txt
Copiar un archivo	copy a.txt b.txt	cp a.txt b.txt
Imprimir texto en pantalla	echo Hello	echo Hello
Limpiar la pantalla	cls	clear

3.3.1. Rutas y separadores

Windows utiliza *barras invertidas* y letras de unidad para describir ubicaciones de archivos:

```
1 C:\Users\DrG\Documents\ELEVATE Academy\main.py
```

Linux utiliza *barras diagonales* y un único árbol de archivos unificado que tiene como raíz /:

```
1 /home/drg/ELEVATE Academy/main.py
```

3.3.2. Scripts y ejecutabilidad

En Windows, la extensión del archivo determina si un archivo es ejecutable. Los archivos que terminan en `.bat`, `.cmd` o `.exe` se tratan como programas.

En Linux, la extensión es irrelevante. La ejecutabilidad es un *permiso* almacenado en el archivo mismo. Debe otorgarse explícitamente permiso de ejecución a un archivo antes de que el sistema lo ejecute. Esto se hace con el comando `chmod`, que cubrimos en la Sección 5.3.3.

3.3.3. Variables de entorno y PATH

Ambos sistemas utilizan una variable de entorno llamada `PATH` para encontrar programas. Cuando usted escribe un comando, el intérprete de comandos busca en cada directorio listado en `PATH` en orden y ejecuta la primera coincidencia que encuentra.

- **Windows:** `PATH` se administra a través de Propiedades del sistema → Variables de entorno, o mediante el comando `setx`.
- **Linux:** `PATH` se configura en archivos de configuración como `~/.bashrc` (para el intérprete Bash) o `~/.zshrc` (para Zsh), los cuales se leen cada vez que se inicia una nueva sesión de terminal.

3.4. WSL: un entorno Linux dentro de Windows

Para las personas que utilizan Windows y desean el poder del scripting de Linux sin abandonar su entorno familiar, Microsoft proporciona el **Subsistema de Windows para Linux** (WSL). WSL no es un emulador ni una máquina virtual en el sentido tradicional; es una capa de compatibilidad integrada en Windows que ejecuta un núcleo Linux real junto al núcleo de Windows.

3.4.1. Qué le ofrece WSL

- Un intérprete Bash completo (o Zsh, Fish, etc.)
- Un sistema de archivos completo de Ubuntu (u otra distribución)
- Acceso a miles de herramientas de línea de comandos de Linux a través de `apt`
- La capacidad de leer y escribir archivos de Windows desde Linux (su unidad `C:` de Windows aparece en `/mnt/c/`)
- Acceso a GPU para cargas de trabajo de aprendizaje automático

3.4.2. Instalación de WSL

Abra PowerShell o CMD *como Administrador* y ejecute:

```
1 wsl --install
```

Listing 3.1: Instalación de WSL desde PowerShell

Esto instala WSL 2 y la distribución Ubuntu de forma predeterminada. Después de reiniciar, se abrirá una terminal de Ubuntu y le pedirá que cree un nombre de usuario y una contraseña. Esa cuenta es independiente de su inicio de sesión de Windows.

Nota

A lo largo del resto de este capítulo, todas las instrucciones para Linux se aplican por igual a WSL en Windows y a una máquina Linux nativa. Trátelas como el mismo entorno.

3.4.3. La solución elegante: Visual Studio Code y la extensión WSL

Uno de los aspectos más poderosos de WSL es su integración con **Visual Studio Code** (VS Code), el editor de código gratuito de Microsoft. La instalación de la **extensión WSL** en VS Code crea un puente sin interrupciones entre su escritorio de Windows y su entorno de Linux:

- Usted abre VS Code desde la terminal de WSL escribiendo `code .`

- VS Code ejecuta su interfaz en Windows pero ejecuta todo el código, los comandos de terminal, las extensiones y los depuradores *dentro de Linux*
- El explorador de archivos muestra su sistema de archivos de Linux
- La terminal integrada es un intérprete Bash
- Los entornos de Python, los linters y los formateadores se resuelven a sus versiones de Linux

Esto significa que usted nunca tiene que elegir entre el ecosistema de scripting de Linux y la experiencia de escritorio de Windows: obtiene ambos simultáneamente. A partir de este punto, cuando digamos «abra VS Code», nos referiremos a iniciarlo desde la terminal de WSL con `code` .

Consejo

Para instalar la extensión WSL, abra VS Code, presione **Ctrl+Shift+X** para abrir el panel de Extensiones y busque «WSL» (editor: Microsoft). Un clic la instala.

Capítulo 4

Introducción a los entornos de programación

4.1. Antes del editor: cómo escribir su primer programa en un archivo de texto

Un programa de Python es texto plano almacenado en un archivo con la extensión `.py`. Cualquier herramienta que guarde texto plano (no texto con formato) puede crear uno. Antes de utilizar un editor sofisticado, usted creará y ejecutará un programa empleando el editor de texto más simple disponible en su sistema. Esto establece una idea fundamental: el editor es irrelevante; lo que importa es el archivo.

Cuadro 4.1: Cómo abrir un editor de texto plano en cada sistema operativo.

Sistema operativo	Cómo abrir el editor	Paso crítico antes de escribir
Windows	Presione <code>Win + R</code> , escriba <code>notepad</code> y presione <code>Enter</code>	En el diálogo Guardar como, establezca «Guardar como tipo» en Todos los archivos (*.*) antes de guardar; de lo contrario, el archivo se guardará como <code>hello.py.txt</code>
macOS	Abra Spotlight (Cmd + Space), escriba <code>TextEdit</code> y presione <code>Enter</code>	Vaya a Formato > Convertir en texto simple antes de escribir cualquier cosa; TextEdit utiliza texto enriquecido de manera predeterminada, lo cual corrompe el código
Linux	Presione <code>Ctrl + Alt + T</code> para abrir una terminal; luego escriba <code>gedit &</code> y presione <code>Enter</code>	No se requiere ningún paso adicional; <code>gedit</code> guarda texto plano de manera predeterminada

Ejercicio P-1: Su primer programa de Python

Escriba el programa que aparece abajo exactamente como se muestra en su editor de texto. Preste atención a la indentación: los espacios antes de `print(n)` y `if n == 5`: no son decorativos; son sintaxis de Python obligatoria y el programa fallará sin ellos. Guarde el archivo como `hello.py` en una carpeta que pueda encontrar (su Escritorio es una buena opción). Aplique el paso crítico de la Tabla 4.1 correspondiente a su sistema operativo antes de guardar.

Los procesadores de texto no son editores de texto

Microsoft Word, Google Docs y LibreOffice Writer guardan archivos con marcado de formato invisible. Un archivo de Python guardado con una de estas herramientas no podrá ejecutarse. Utilice únicamente los editores listados en la Tabla 4.1 o VSC.

```
1 # Los comentarios comienzan con el caracter #
2 # Los comentarios no forman parte del programa.
3 # Pueden usarse para introducir marcadores visuales para
  mayor claridad
4 # -----
5 # Este programa ejemplifica entrada-salida (I/O),
6 # bucles, condicionales y asignacion
7 print("This is programming!")
8 k = 10
9 for n in range(k):
10     print(n)
11     if n == 5:
12         print("here")
```

La línea `#` es un comentario; Python la ignora. Los comentarios existen para los humanos. `k = 10` es una asignación; crea una variable llamada `k` y almacena el valor 10. `for n in range(k):` es un bucle; todo lo indentado debajo se ejecuta 10 veces. `if n == 5:` es un condicional; el bloque indentado debajo solo se ejecuta cuando la condición es verdadera.

El programa existe como un archivo, pero no hace nada hasta que sea ejecutado por el intérprete de Python. Ahora que usted ha completado el Capítulo 3, dispone de todas las herramientas necesarias para ejecutarlo. Abra una terminal, navegue hasta la carpeta donde guardó `hello.py` y ejecute:

```
python hello.py
```

En macOS y Linux, podría ser necesario utilizar `python3` en lugar de `python`. La salida debería coincidir con lo que usted esperaría al leer el código.

4.2. Introducción

Existe una gran cantidad de lenguajes de programación. Para los fines de la computación científica, distinguiremos dos familias principales de lenguajes: lenguajes compilados frente a lenguajes de scripting. Representan dos paradigmas distintos en el diseño de lenguajes de programación, cada uno con sus propias ventajas y compensaciones. La diferencia clave radica en cuándo y cómo el código fuente que usted escribe se traduce a código de máquina que la computadora puede entender y ejecutar.

En el ámbito de la computación científica, la elección entre utilizar un lenguaje compilado como C++ y un lenguaje de scripting como Python a menudo depende de las particularidades del proyecto en cuestión, incluyendo su complejidad, la velocidad de ejecución requerida y la necesidad de acceso directo al hardware.

4.2.1. Lenguajes compilados

Los lenguajes compilados son aquellos en los que el código fuente se traduce íntegramente a código de máquina antes de la ejecución. Esta traducción la realiza un compilador, un programa que toma todo el código fuente como entrada y genera un archivo ejecutable. Ejemplos de lenguajes compilados incluyen C++, Java, Rust, Go y Swift.

Los lenguajes compilados ofrecen varios beneficios importantes, entre ellos:

- Rendimiento: dado que el código está precompilado a código de máquina, los programas compilados suelen ejecutarse más rápido y de manera más eficiente que los interpretados.
- Verificación de tipos: muchos lenguajes compilados poseen sistemas de tipos estrictos, los cuales detectan numerosos errores relacionados con tipos en tiempo de compilación, antes de que el código se ejecute.
- Seguridad: el código fuente no se distribuye con el software, lo que dificulta que terceros realicen ingeniería inversa del software.

Sin embargo, los lenguajes compilados también presentan algunas desventajas:

- Velocidad de desarrollo: el ciclo editar-compile-ejecutar puede consumir mucho tiempo, particularmente en proyectos de software de gran tamaño.
- Portabilidad: el ejecutable generado suele ser específico para un tipo de procesador y sistema operativo. La disponibilidad para diferentes sistemas operativos o procesadores podría requerir modificaciones específicas del código fuente y compilaciones independientes.

Los lenguajes compilados como C++ se traducen a código de máquina antes de la ejecución, lo que los hace muy rápidos porque son ejecutados directamente por el hardware de la computadora. Esto puede ser una ventaja significativa al

tratar con conjuntos de datos enormes o cuando la velocidad de cómputo es un factor crítico.

Sin embargo, la sintaxis de lenguajes como C++ es más compleja y puede requerir más líneas de código para lograr el mismo resultado que un lenguaje de scripting. Además, C++ carece de muchas de las funcionalidades incorporadas de análisis de datos que se encuentran en lenguajes como Python, MATLAB o R, por lo que es necesario utilizar bibliotecas externas como Eigen o Armadillo para operaciones matriciales y otros cálculos complejos.

4.2.2. Lenguajes de scripting

Los lenguajes de scripting, por otro lado, son típicamente interpretados de texto a instrucciones de máquina, línea por línea de código, justo a tiempo para la ejecución. Si bien esto puede hacer que los lenguajes de scripting sean más lentos que los lenguajes compilados, a menudo son más fáciles de escribir y leer debido a su sintaxis de alto nivel. Ejemplos de lenguajes de scripting incluyen Python, MATLAB, R, JavaScript, Ruby y PHP.

Los lenguajes de scripting ofrecen varias ventajas:

- **Facilidad de uso:** los lenguajes de scripting suelen ser más fáciles de aprender y usar. Tienen una sintaxis menos verbosa y más funcionalidad incorporada.
- **Flexibilidad:** como se interpretan línea por línea, es más fácil realizar cambios en su código y ver su efecto inmediatamente sin tener que recompilar.
- **Portabilidad:** el mismo script puede ejecutarse en cualquier sistema que tenga instalado el intérprete correcto, sin necesidad de cambiar el código.

Entre las desventajas de los lenguajes de scripting se encuentran cuestiones relacionadas con:

- **Rendimiento:** los lenguajes interpretados son generalmente más lentos que los lenguajes compilados debido a la sobrecarga de interpretar código línea por línea durante la ejecución.
- **Dependencia del intérprete:** el código solo puede ejecutarse en sistemas que tengan instalado el intérprete correcto.

Python es particularmente adecuado para la ciencia de datos biomédicos porque cuenta con numerosas bibliotecas poderosas para el análisis de datos (como Pandas, NumPy y SciPy) y la visualización (como Matplotlib y Seaborn). Además, la sintaxis simple y directa de Python lo convierte en una excelente opción para la creación rápida de prototipos y el análisis exploratorio de datos.

Cabe señalar que, incluso con todos los beneficios de Python, la sintaxis simplificada de MATLAB para operaciones matriciales facilita el desarrollo de algoritmos, ya que el código fuente se asemeja estrechamente a la notación matemática.

4.2.3. Ejemplos comparativos

Consideremos un ejemplo sencillo en el que deseamos multiplicar dos matrices A y B. Los ejemplos en cada lenguaje se listan abajo en orden alfabético por el nombre del lenguaje.

En C++:

```
1 #include <Eigen/Dense>
2 #include <iostream>
3
4 int main() {
5     Eigen::Matrix2f A;
6     A << 1, 2,
7         3, 4;
8     Eigen::Matrix2f B;
9     B << 5, 6,
10        7, 8;
11     std::cout << "The product AB is:\n" << A * B;
12 }
```

../source/IntroProgMatMul.c

En MATLAB:

```
1 A = [1, 2; 3, 4];
2 B = [5, 6; 7, 8];
3 disp('The product AB is:')
4 disp(A * B)
```

../source/IntroProgMatMul.m

En R:

```
1 A <- matrix(c(1, 3, 2, 4), nrow=2, ncol=2, byrow = TRUE)
2 B <- matrix(c(5, 7, 6, 8), nrow=2, ncol=2, byrow = TRUE)
3
4 print("The product AB is:")
5 print(A %*% B)
```

../source/IntroProgMatMul.r

En Python:

```
1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5 print("The product AB is:\n", np.matmul(A, B))
```

../source/IntroProgMatMul.py

Del ejemplo anterior, queda claro que el código en MATLAB es mucho más conciso y fácil de comprender que el de C++. Sin embargo, si estuviéramos tratando con matrices muy grandes o necesitáramos realizar esta operación muchas veces en un bucle, el código en C++ podría ejecutarse con mayor rapidez debido a ser un lenguaje compilado.

4.2.4. ¿Qué es Python?

Python es un lenguaje de programación *interpretado* y de *alto nivel* para computación de propósito general.

- *Interpretado* significa que un programa en Python debe ser decodificado línea por línea por otro programa llamado intérprete de Python y traducido a algo que una computadora pueda comprender.
- *Alto nivel* significa que existe una fuerte abstracción de los elementos del hardware de la computadora; por ejemplo, la memoria no necesita ser pre-asignada en Python para crear una matriz, mientras que el lenguaje C (un lenguaje de bajo nivel) sí lo requiere. Puede pensar en el nivel del lenguaje como cuán cercanas están las instrucciones al metal de la máquina.
- El proceso de interpretación del lenguaje se denomina *compilación*, es decir, la traducción de un lenguaje de computadora a otro lenguaje. Esto podría ocurrir múltiples veces hasta que las instrucciones se reduzcan a un lenguaje llamado *assembler* o *ensamblador*; este código coincide estrechamente con la arquitectura de la computadora que ejecuta el programa. Por ejemplo, los programas en Java se compilan a un lenguaje llamado Java bytecode, el cual la Máquina Virtual de Java traduce a ensamblador.

4.2.5. Instalación de Python

Remítase ahora al Capítulo 2 para ver las instrucciones completas de instalación de Python, Visual Studio Code y todas las herramientas requeridas.

Al ser un lenguaje interpretado, significa que Python requiere un *intérprete*. Esto es lo que usted instala cuando «*instala Python*». Un programa de Python es simplemente un archivo de texto que contiene instrucciones que el intérprete puede entender una línea (o bloque de líneas) a la vez. Puesto que Python es de código abierto, tiene muchos contribuidores; esto resulta en un ecosistema robusto, pero la desventaja es que existen numerosas bibliotecas de software que deben instalarse mientras se rastrea una compleja red de codependencias. Por ello, hay cierta dificultad en instalar un entorno de Python funcional para realizar análisis cuantitativo.

Una manera sencilla de usar el lenguaje Python es a través de un *Entorno de Desarrollo Integrado*. Existen muchos IDE que pueden ejecutar código de Python. Estar familiarizado con más de un entorno es importante, ya que cada uno tiene fortalezas y debilidades. Además, es común que los programas de Python parezcan fallar en un entorno pero funcionen correctamente en otro; esto, por supuesto, es una cuestión de configuración, pero puede ser fuente de frustración ante la falta de conciencia de las fuentes potenciales de conflictos en el software. Particularmente, usted desea saber cómo utilizar herramientas frecuentemente empleadas en el desarrollo de software y necesita exposición a las múltiples opciones que encontrará en la academia, el gobierno y la industria.

4.2.6. Ejecución de comandos en Python

Los comandos en Python distinguen entre mayúsculas y minúsculas, es decir, usted debe respetar el uso de mayúsculas en las instrucciones tal como aparecen en la documentación. El intérprete de Python ofrece múltiples mecanismos para interactuar con el entorno. El más comúnmente utilizado es la línea de comandos de Python. Puede iniciarla desde un intérprete de comandos. Simplemente escriba `python`. El indicador de la línea de comandos cambiará a `>>>`

Usted puede ingresar comandos de Python en el indicador de la línea de comandos. Presione *Enter* o *return* para ejecutar comandos. Como la mayoría de los demás lenguajes de programación, Python proporciona expresiones matemáticas. Los bloques de construcción de las expresiones son variables, números, operadores, scripts y funciones.

Las operaciones básicas se listan abajo.

Símbolo	Operación
+	Adición
-	Sustracción
*	Multiplicación
/	División
**	Potencia

Algunos comandos son como los que uno escribiría en una calculadora. Si usted desea hacer referencia al resultado del comando, necesita asignarle un nombre, o en otras palabras, asignar el resultado a una variable. El siguiente comando asigna el resultado a la variable `s`. (Observe que Python no guardó la respuesta verdadera $7/12$, sino una aproximación decimal.)

```
1 >>> s = 1/3 + 1/4
```

Si desea saber qué contiene la variable `s`, simplemente escriba su nombre en el indicador de comandos:

```
1 >>> s
2 0.5833333333333333
```

Si un comando no cabe en una sola línea, utilice una barra invertida (`\`) seguida de Return/Enter para indicar que la sentencia continúa en la línea siguiente. Por ejemplo,

```
1 >>> s = 1 - 1/2 + 1/3 - 1/4 + 1/5 - 1/6 + 1/7 \
2 ... :- 1/8 + 1/9 - 1/10 + 1/11 - 1/12
```

Escriba `s` y presione Enter para ver el nuevo valor de `s`.

```
1 >>> s
2 0.6532106782106782
```

Utilice el doble asterisco (`**`) para elevar algo a un exponente.

```
1 >>> 89**2
2 7921
```

Los números imaginarios pueden utilizarse con la constante j , que representa $\sqrt{-1}$ de forma predeterminada.

```
1 >>> (1j)**2
2 (-1+0j)
```

Por lo tanto, usted debe tener cuidado al utilizar la variable j cuando trabaje con números complejos. ¿Qué espera de la siguiente operación? ¿Puede explicar lo que observa? Quizás necesite completar el capítulo entero antes de poder responder esta pregunta. **Registre su explicación como respuesta n.º 1.**

```
1 >>> j = 10
2 >>> j = j*(-1)**(0.5)
3 >>> j
```

¿El resultado almacenado en la variable j coincide con su expectativa? Explique. **Registre su explicación como respuesta n.º 2.**

Python utiliza notación decimal convencional, con un punto decimal opcional y un signo más o menos inicial, para los números. La notación científica utiliza la letra e o E para especificar un factor de escala de potencia de diez. Los números imaginarios utilizan i como sufijo. (Este curso no utiliza números complejos, pero usted podría generar errores con alguno de ellos.)

Algunos ejemplos de números válidos son: 1, -1, 0.0001, 9.87654321, 1.2345e10, 1.2345*10**10, 1j, -2j, 3e5j.

Al asignar nombres a variables o expresiones, es una buena práctica hacer que los nombres sean útiles. Por ejemplo, en el segundo comando anterior, le dimos al resultado el nombre `rhoSq` para implicar «rho al cuadrado». Observe que **NO PUEDE** tener espacios en los nombres de las variables (es decir, sin espacios en los nombres del lado izquierdo del signo igual). Dar nombres útiles a sus variables o salidas le ayudará a llevar un registro de lo que está haciendo.

Usted puede utilizar las teclas de flecha arriba y abajo para recuperar fácilmente un comando y luego editarlo con las flechas izquierda y derecha. Pruébalo. Esta es la manera más rápida de repetir comandos de Python que pudo haber utilizado en el pasado.

4.2.7. El editor de Python de VSC: ejecución de un programa simple

Descargue el archivo `lab1SimpleInstruction.py`. Abra el archivo en VSC. Las siguientes instrucciones aparecerán en el editor:

```
1 k=0;
2 for i in range(1,11):
3     k=k+1/10;
4     print(k)
5 print(k==1)
```

Puede hacer clic en el botón *Run file* de la barra de herramientas (como se muestra en la Figura 1), o presionar F5. Llamamos a todas estas acciones *ejecutar*

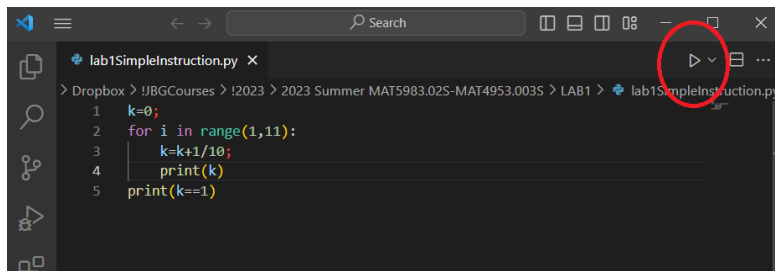


Figura 1: Para ejecutar un archivo de Python en VSC, simplemente haga clic en el botón Run, representado por un triángulo que apunta hacia la derecha. Se ha resaltado con un círculo.

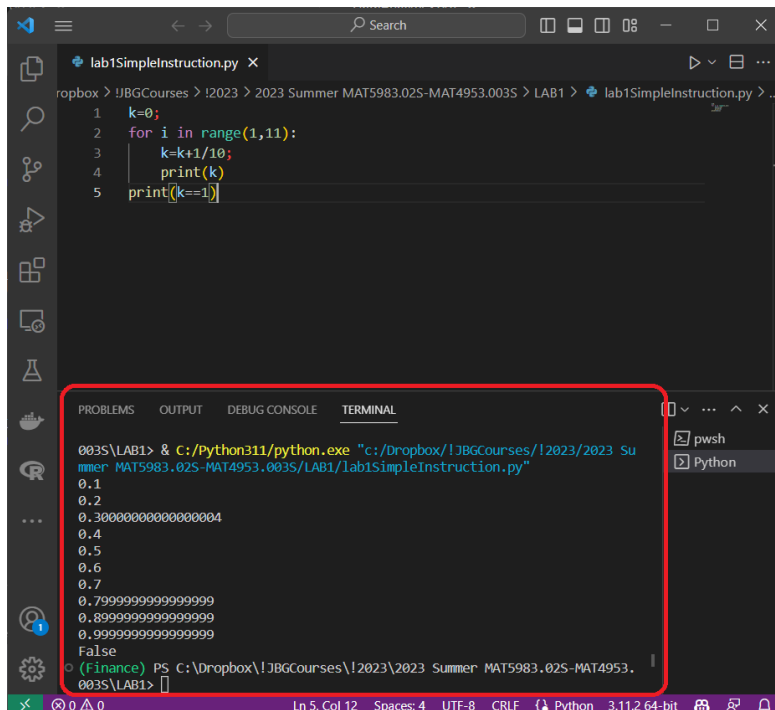


Figura 2: Ejecutar un programa de Python activa la línea de comandos. Los resultados se imprimen en ese panel. La línea de comandos se resalta con un recuadro redondeado.

un programa. El programa `lab1SimpleInstruction.py` suma diez veces el número 0.1. La comparación del número 1 contra la suma devuelve un valor de *false*, mientras que la aritmética elemental nos dice que sumar el número 0.1 diez veces debería resultar en 1, y el valor devuelto debería ser *true*. ¿Por qué este resultado es *false*? **Registre su explicación como respuesta n.º 3.** Una vez ejecutado el programa, la parte inferior de VSC muestra un panel etiquetado TERMINAL, así como otras pestañas etiquetadas PROBLEMS, OUTPUT y DEBUG CONSOLE, como se muestra en la Figura 2

Observe que usted puede ingresar comandos en la terminal. Por ejemplo, escriba `1+1` y presione Enter. Pruebe operaciones matemáticas como raíz cuadrada o funciones trigonométricas. ¿Qué retos encuentra? **Registre su explicación como respuesta n.º 4.**

En el programa `lab1SimpleInstruction.py` todas las líneas de código se ejecutaron secuencialmente, una a la vez. Esto es lo que llamamos un **archivo de**

script. Es decir, cuando usted ejecuta un archivo de script, espera que ocurra algo tangible, ya sea un mensaje en pantalla, un archivo alterado, una comunicación que tiene lugar, etc.

Existe una forma de utilizar archivos de código fuente para mantener código que se ejecutará más tarde, sin necesariamente ejecutar ninguna instrucción. En este punto, abra el archivo `lab1Function.py`. Verá las siguientes instrucciones:

```
1 def sum1(n):
2     # sum1(n) calcula la suma de 1 + 2 + ... + n
3     total = 0;
4     for i in range(1,n):
5         total = total + i
6     return total
```

Este archivo define una función llamada `sum1(n)`. El comando `sum(3)` devuelve 6 puesto que $1 + 2 + 3 = 6$. Es instructivo ver cómo este código realiza su trabajo.

- La instrucción `def` indica el comienzo de una *función*. Observe que todas las instrucciones que se ejecutan dentro de esa función tienen una alineación vertical desplazada hacia la derecha para indicar que todos estos comandos están subordinados a `def`. Este desplazamiento en la alineación vertical se denomina indentación. Es parte de la sintaxis del lenguaje de programación Python; por lo tanto, siempre debe tratar las indentaciones de manera estricta.
- La instrucción `# sum1(n)...` es un *comentario*. Las líneas de código que comienzan con el símbolo `#` no se ejecutan. El punto y coma al final es opcional.
- La instrucción `total = 0;` es una *asignación*. Esta línea de código crea la variable `total` y le asigna el valor cero.
- La instrucción `for i in range(1,n):` inicia un *bucle*. Las instrucciones subordinadas, identificadas por indentación, se repetirán n veces. Particularmente, la instrucción `range(1,n)` crea una secuencia de enteros que comienzan en 1 y terminan en n . ¿Cuál es el resultado de `range(10,13)`?
- La instrucción `total = total + i` es una asignación. Es muy importante enfatizar que esto no es una ecuación; si lo fuera, `total` se cancelaría e `i` sería cero. Lo que ocurre es que por cada repetición del bucle, la variable `total` se redefine para contener su valor anterior más el valor de `i`.
- La instrucción `return total` es la salida de la función. Es decir, si de alguna manera pudiéramos ejecutar esta función como `x = sum1(3)`, la variable `x` contendría el valor 6, puesto que `sum1(3) = 1 + 2 + 3 = 6`.

Ahora modificaremos este archivo. Agregue al final un programa llamado `sum2` en el cual sumemos enteros elevados al cuadrado. La siguiente pregunta debe ser: ¿cómo ejecutamos estas funciones?

El comando `python [filename]` puede utilizarse para ejecutar un programa de Python desde la terminal, o desde cualquier indicador de línea de comandos. Pero si ejecutamos lo siguiente en el indicador de comandos, no ocurre nada:

```
1 python lab1Function.py
```

¿Por qué? **Registre su explicación como respuesta n.º 5.**

Ahora, agregue la siguiente línea de código al final del archivo, **sin indentación**:

```
1 print(sum1(2))
```

Ejecute el programa. ¿Cuál fue el resultado de `sum1(2)`? ¿Es esto lo que esperaba? ¿Por qué? **Registre su explicación como respuesta n.º 6.** Calcule a mano cuál debería ser la salida de `sum1(3)`. Ahora cambie la última línea del programa para que lea `print(sum1(2))`

Lo que hicimos fue simplemente definir una función y luego utilizarla. Observe que si usted coloca la línea de código anterior al inicio del archivo, el intérprete de Python lanzaría una excepción porque la función `sum1` aún no es conocida por el intérprete... tenga presente que el intérprete de Python ejecuta una línea de código a la vez en orden secuencial (quizás con ramificaciones y repetición, como exploraremos más adelante). Por lo tanto, las funciones deben definirse antes de que puedan invocarse dentro de un archivo.

¿El programa produjo el resultado que usted esperaba? En caso de que no lo hiciera, ingrese la siguiente instrucción en la terminal: `man range`. Este comando muestra el manual (`man`) para la función `range`. El hecho de que `range` se comporte de ciertas maneras no lo obliga a usted a regirse por ese diseño en particular. De hecho, usted puede modificar `sum1` de modo que el resultado corresponda a lo que un usuario ingenuo esperaría. Para lograrlo, necesitará cambiar el límite de `range` dentro de la función. Implementelo y explique. **Registre su explicación como respuesta n.º 7.**

4.2.8. Ejecución de funciones

En la sección anterior, agregamos una sola línea de código al final del programa para calcular la función que programamos. Sin embargo, existe un caso de uso más interesante: crear una función que otros programas puedan reutilizar. Para utilizar una función existente desde un archivo, debemos importar esa función al entorno de Python. Ya hicimos algo así cuando importamos NumPy.

Ahora importaremos la función `sum1` desde el archivo `lab1Function.py`. Para ello, ejecute el siguiente comando:

```
1 import lab1function as LAB1
```

Esta instrucción carga en memoria un directorio de todas las funciones presentes en ese archivo. Ahora usted puede invocar las funciones que creó. Por ejemplo,

```
1 LAB1.sum2(3)
```

También es posible cargar en memoria una sola función desde un archivo. Podemos lograrlo ejecutando

```
1 from lab1function import sum1
```

En este caso, no hay alias asociado con la función. Para usarla, usted simplemente puede llamar la función con un parámetro, como en `sum2(3)`

Investigue un poco sobre cómo cargar múltiples funciones con una sola instrucción. Explique. [Registre su explicación como respuesta n.º 8.](#)

4.3. Introducción a NumPy

Las operaciones numéricas pueden programarse en Python. Una biblioteca llamada **NumPy** encapsula mucha funcionalidad necesaria y se ha convertido en el estándar *de facto*. El elemento de datos básico en **NumPy** es un arreglo multidimensional. Esto le permite resolver muchos problemas de computación técnica, especialmente aquellos con formulaciones matriciales y vectoriales, en una fracción del tiempo que tomaría escribir un programa en un lenguaje de bajo nivel como C.

Por ejemplo, en el indicador de Python escriba el siguiente comando y presione Enter.

```
1 import numpy as np
```

Este comando carga en memoria una referencia a la biblioteca NumPy. Ahora puede acceder a todas las funciones de esta biblioteca utilizando el prefijo `np`; este es un *alias*. Puede elegir cualquier alias que desee, pero es costumbre usar `np` para NumPy. Ahora ingrese el siguiente comando:

```
1 np.sqrt(9)
```

La función `sqrt` calcula la raíz cuadrada de un número real. He aquí un ejemplo y los valores resultantes.

```
1 >>> rho = (1+np.sqrt(5))/2
2 >>> rhoSq = rho**2
3 >>> a = rhoSq + rho
4 >>> a
5 4.23606797749979
```

Sin embargo, usted no puede utilizar la función `sqrt` con números negativos. ¿Qué tendría que hacer para permitir que Python calcule la raíz cuadrada de un número negativo y devuelva un número complejo? [Registre su explicación como respuesta n.º 9.](#)

La biblioteca NumPy ofrece acceso fácil a constantes de uso común, tales como:

Constante	Valor
<code>np.e</code>	Devuelve el número e
<code>np.pi</code>	Devuelve el número π
<code>np.inf</code>	Devuelve el número ∞
<code>np.nan</code>	NaN, no-es-un-número

Las expresiones utilizan los operadores aritméticos y reglas de precedencia que usted ya conoce a estas alturas. También existen funciones, como coseno o seno. La entrada a la función se rodea con paréntesis. Con NumPy, ahora podemos acceder a funciones matemáticas de uso común. Entre otras:

Función	Operación
<code>np.abs(x)</code>	Valor absoluto $ x $
<code>np.sqrt(x)</code>	Raíz cuadrada $\sqrt{x}$
<code>np.exp(x)</code>	Función exponencial e^x
<code>np.log(x)</code>	$\log(x)$ donde x es un número real positivo
<code>np.sin(x)</code>	$\sin(x)$ donde se asume que x está en radianes
<code>np.cos(x)</code>	$\cos(x)$ donde se asume que x está en radianes

La biblioteca NumPy tiene *muchas* funciones. Puede encontrarse una lista exhaustiva en <https://numpy.org/doc/stable/reference/>. Ahora ingrese el siguiente comando:

```
1 x = np.random.standard_normal(3)
```

No hay salida. ¿Por qué cree usted que este es el comportamiento? **Registre su explicación como respuesta n.º 10.** En el lado derecho de la ventana de Spyder hay una pestaña llamada *Variable Explorer*. Esta mostrará el valor de x . Alternativamente, puede ingresar

```
1 print(x)
```

De manera alternativa, puede imprimir el resultado directamente sin asignarlo a una variable. Pruebe

```
1 print(np.random.standard_normal(3))
```

¿Por qué los resultados de esta última operación son diferentes de los valores almacenados en la variable x ? **Registre su explicación como respuesta n.º 11.**

4.3.1. Operaciones matriciales

El elemento de datos básico en NumPy es un arreglo multidimensional. A veces se le atribuye un significado especial a las matrices de 1 por 1, que se denominan escalares, y a las matrices con una sola fila o columna, las cuales se denominan vectores. Python tiene otras formas de almacenar datos tanto numéricos como no numéricos, pero al principio, normalmente es mejor pensar en todo como una matriz.

Los vectores fila se delimitan en notación por corchetes. Una coma separa los números individuales en un vector fila. Múltiples vectores fila del mismo tamaño se separan por comas y se delimitan con corchetes para definir una matriz. Es práctica común utilizar una letra mayúscula al nombrar matrices con el fin de distinguirlas de un vector, escalar o valor de variable individual.

Ahora, crearemos un vector y una matriz para demostrar operaciones matriciales simples

Función	Operación
<code>x = np.array([1,2,3,4])</code>	Crea el vector fila $\mathbf{x} = [1, 2, 3, 4]$
<code>b = np.array([1,2,3])</code>	Crea el vector fila $\mathbf{b} = [1, 2, 3]$
<code>A = np.array([[1,2,3],[4,5,6],[7,8,9]])</code>	Crea la matriz $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
<code>B = np.ones([2,3])</code>	Crea la matriz $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
<code>C = np.zeros([2,3])</code>	Crea la matriz $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

Los siguientes comandos demuestran casos básicos de manipulación de vectores:

Operación	Resultado
<code>x[-1]</code>	Devuelve 4, el último elemento de x
<code>x[1]</code>	Devuelve 2, el segundo elemento de x
<code>x[-2]</code>	Devuelve 3, el penúltimo elemento de x

El operador dos puntos (`:`) es notable. En notación vectorial, indica un rango de filas o columnas. En Python, el primer elemento de un vector tiene índice 0; el segundo elemento tiene índice 1, y así sucesivamente. El operador dos puntos $a : b$ comienza en la fila o columna de índice a y cubre hasta un índice menor que b . Así, `A[0 : 2, 0 : 2]` devuelve una matriz compuesta por las filas 1 a 2 y las columnas 1 a 2 con respecto a la matriz A .

Los siguientes comandos demuestran casos básicos de manipulación de matrices:

Operación	Resultado
<code>A[1, 2]</code>	Valor de la 2 ^{da} fila y 3 ^{ra} columna de A
<code>A[0:2, 0:2]</code>	Devuelve $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
<code>A[1:3, 1:3]</code>	Devuelve $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

La matriz A tiene tres filas y tres columnas. Entonces, ¿por qué `A[1 : 100, 1 : 100]` no produce un error? **Registre su explicación como respuesta n.º 12.**

El tamaño o dimensión de una matriz se refiere al número de filas y número de columnas que tiene una matriz. El comando `shape` devuelve el número de filas y columnas de una matriz. Observe que el resultado es un vector: una matriz unidimensional con 2 valores, donde el primero es el número de filas y el segundo valor es el número de columnas. Al acceder a cada elemento de este vector, podemos acceder a cada dimensión de manera individual.

Utilizando la matriz A de nuestros ejemplos anteriores,

```

1 >>> y = A.shape()
2 >>> y
3 (3, 3)
4 >>> y[0]
5 3
6 >>> y[1]
7 3

```

El siguiente ejemplo ilustrará una sutileza en la copia de matrices:

```

1 >>> D = A
2
3 array([[1, 2, 3],
4        [4, 5, 6],
5        [7, 8, 9]])
6 >>> D[0,0] = 9
7 >>> A
8
9 array([[9, 2, 3],
10        [4, 5, 6],
11        [7, 8, 9]])

```

Observe que al actualizar un valor en D , el valor correspondiente en A cambió. ¿Por qué? ¿Qué necesitaría hacer para evitar que una actualización en D modifique A ? **Registre su explicación como respuesta n.º 13.**

Las operaciones matriciales básicas se listan abajo, usando las matrices de los ejemplos anteriores.

Operación	Resultado
$A.T$	Transpuesta de A .
$A.conj().T$	Transpuesta conjugada de A .
$A @ B.T$	Multiplicación de matrices $A \cdot B$.
$A * A$	Multiplicación elemento a elemento
A / A	División elemento a elemento
$A ** 2$	Exponenciación elemento a elemento

NumPy ofrece una manera sencilla de resolver sistemas de ecuaciones lineales. Dada la matriz A y el vector $\mathbf{b}$ de los ejemplos anteriores, el sistema lineal $A\mathbf{x} = \mathbf{b}$ puede resolverse invocando `np.linalg.solve` de la siguiente manera

```

1 >>> x = np.linalg.solve(A,b)
2 array([-0.23333333,  0.46666667,  0.1        ])

```

Es fácil utilizar la multiplicación de matrices $A\mathbf{x}$ para comprobar la respuesta

```

1 >>> A @ x
2 array([1., 2., 3.])

```

Capítulo 5

Entornos virtuales de Python y automatización del flujo de trabajo

Antes de escribir cualquier script, necesitamos comprender *qué* van a gestionar esos scripts: los entornos virtuales de Python.

El problema: conflictos entre paquetes. Imagine que tiene dos proyectos en su computadora. El proyecto A usa la versión 1.5 de una biblioteca llamada `numpy`; el proyecto B requiere la versión 2.0. Si instala ambas versiones de forma global, entrarán en conflicto. Gana la última que se haya instalado, y uno de sus proyectos podría dejar de funcionar.

La solución: entornos virtuales. Un **entorno virtual** (o *venv*) es una copia aislada de Python con su propio conjunto de paquetes instalados, completamente separada de cualquier otro entorno en su máquina. Piénselo como una burbuja autocontenida alrededor de su proyecto.

- Cada proyecto obtiene su propio entorno.
- Los paquetes instalados en un entorno no pueden interferir con otro.
- Puede recrear un entorno de manera exacta a partir de una lista de paquetes.
- Eliminar un entorno es seguro y limpio.

Cubriremos dos estrategias para gestionar entornos: el enfoque tradicional mediante `pip` y una tendencia emergente llamada `uv`.

5.1. Gestión tradicional de paquetes: `pip`

5.1.0.1. Windows (CMD o PowerShell)

```
1 :: Navegue a la carpeta de su proyecto
2 cd C:\Projects\ELEVATE Academy
3
4 :: Cree el entorno (Python debe estar instalado globalmente)
5 python -m venv C:\penv\ELEVATE Academy
```

Listing 5.1: Creación de un `venv` en Windows

5.1.0.2. Linux / WSL

```
1 # Navegue a la carpeta de su proyecto
2 cd ~/ELEVATE Academy
3
4 # Cree el entorno
5 python3 -m venv ~/penv/ELEVATE Academy
```

Listing 5.2: Creación de un venv en Linux/WSL

En ambos casos, `C:\penv\ELEVATE Academy` o `~/penv/ELEVATE Academy` es donde se almacenará el entorno. Mantener todos los entornos en una única carpeta (`penv` significa «Python environments», es decir, «entornos de Python») facilita encontrarlos y gestionarlos.

5.1.1. Activación de un entorno virtual

Activar un entorno modifica la sesión actual de su intérprete de comandos de modo que `python` y `pip` hagan referencia a las copias aisladas dentro del `venv`, y no a las globales.

5.1.1.1. Windows

```
1 C:\penv\ELEVATE Academy\Scripts\activate.bat
```

Listing 5.3: Activación de un venv en Windows

Tras la activación, su indicación cambiará para mostrar el nombre del entorno:

```
1 (ELEVATE Academy) C:\Projects\ELEVATE Academy>
```

5.1.1.2. Linux / WSL

```
1 source ~/penv/ELEVATE Academy/bin/activate
```

Listing 5.4: Activación de un venv en Linux/WSL

Tras la activación:

```
1 (ELEVATE Academy) drg@machine:~/ELEVATE Academy$
```

Advertencia

Observe la palabra clave `source` en el comando de Linux. Esto es crítico y se explica en detalle en la Sección 5.4.

5.1.2. Instalación de paquetes

Una vez que el entorno está activo, use `pip` para instalar paquetes. La sintaxis es idéntica en ambas plataformas:

```
1 pip install numpy pandas matplotlib openai langchain
```

Listing 5.5: Instalación de paquetes con `pip`

Para guardar una lista de todos los paquetes instalados (de modo que usted o un colega puedan recrear el entorno más adelante):

```
1 pip freeze > requirements.txt
```

Para recrear un entorno a partir de dicha lista:

```
1 pip install -r requirements.txt
```

5.1.3. Desactivación

Para salir del entorno y regresar al intérprete de comandos global (idéntico en ambas plataformas):

```
1 deactivate
```

5.1.4. Instalación de las bibliotecas requeridas por ELEVATE Academy

Con el entorno de ELEVATE Academy activo, instale las bibliotecas necesarias para el reto de datos:

```
pip install numpy pandas matplotlib scipy jupyter
```

Estas cinco bibliotecas son suficientes para todo el análisis del reto de datos. Las bibliotecas adicionales requeridas para el capítulo sobre modelos grandes de lenguaje (incluyendo `openai`, `anthropic` y `langchain`) se instalan por separado en el Capítulo 9.

Para verificar la instalación:

```
python -c "import numpy, pandas, matplotlib, scipy, jupyter; print('OK')"
```

Sabrás que su entorno principal está completamente funcional cuando:

- `python -version` devuelve un número de versión.
- `code .` abre Visual Studio Code desde una terminal.
- `git -version` devuelve un número de versión.

- Las cinco bibliotecas requeridas se importan sin errores (véase arriba).

Los programas agente de modelos grandes de lenguaje (`agentGroq.py`, `agentGPT.py`, `agentClaude.py`) se encuentran en `resources/unit_7/F00/` dentro del repositorio de ELEVATE Academy. Ejecutarlos requiere claves de API configuradas en el Capítulo 9.

5.2. Gestión moderna de paquetes: uv

5.2.1. Por qué uv es relevante hoy

Las herramientas presentadas en la sección anterior (`python -m venv` para la creación de entornos y `pip` para la instalación de paquetes) son la base establecida y estable de la gestión de paquetes de Python. Son mantenidas por la Python Packaging Authority (PyPA) bajo el amparo de la Python Software Foundation (PSF), una organización sin ánimo de lucro con una estructura de gobernanza de largo plazo y sin dependencia comercial. Cada instalación de Python incluye `pip` por defecto, y saber cómo usarlo sigue siendo una habilidad esencial independientemente de qué otras herramientas adopte.

Ha surgido una herramienta más reciente llamada `uv` que está ganando adopción rápidamente en entornos profesionales. Comprender ambas herramientas es importante precisamente porque el panorama está en transición.

Información

Una nota sobre la gobernanza. `pip` es mantenido por la Python Packaging Authority bajo la Python Software Foundation, una organización sin ánimo de lucro con un modelo de gobernanza comunitaria y sin dependencia comercial. `uv` es una herramienta de código abierto desarrollada y mantenida por Astral, una empresa comercial financiada con capital de riesgo. Astral ha sido un excelente administrador de la herramienta y ha contribuido de manera significativa al ecosistema más amplio de empaquetado de Python. No obstante, su trayectoria a largo plazo depende de los resultados del negocio de una manera en que los proyectos respaldados por fundaciones no dependen. Hasta que el ecosistema se estabilice en torno a un estándar comunitario claro, conocer tanto `pip` como `uv` es la postura profesional prudente.

5.2.2. Qué ofrece uv

`uv` es un reemplazo directo para `pip` y `python -m venv`, reescrito en Rust por motivos de rendimiento. Sus principales ventajas sobre la cadena de herramientas tradicional son:

- **Velocidad.** La resolución e instalación de paquetes son de 10 a 100 veces más rápidas que `pip` debido a una implementación en Rust y a una descarga paralela agresiva.

- **Una caché central.** Los paquetes descargados se almacenan una sola vez en una caché global y se reutilizan entre entornos. En el mismo sistema de archivos, uv usa enlaces físicos (hardlinks) de modo que múltiples entornos que comparten un paquete no consumen espacio de disco adicional más allá de la primera copia.
- **Archivos de bloqueo.** uv produce un archivo `uv.lock` que registra el grafo completo y resuelto de dependencias con hashes criptográficos. Esta es una mejora significativa respecto a `pip freeze > requirements.txt`, que solo registra nombres y versiones de los paquetes instalados sin distinguir dependencias directas de las transitivas, y sin verificación por hash.
- **Integración con `pyproject.toml`.** uv trabaja de forma nativa con `pyproject.toml`, el estándar moderno para declarar dependencias de un proyecto, reemplazando el flujo de trabajo más antiguo basado en `requirements.txt`.
- **Gestión de versiones de Python.** uv puede descargar y gestionar múltiples versiones de Python de forma independiente, eliminando la necesidad de herramientas separadas como `pyenv`.

5.2.3. Instalación de uv

5.2.3.1. Windows (PowerShell)

```
1 powershell -ExecutionPolicy ByPass -c "irm
  ↪ https://astral.sh/uv/install.ps1 | iex"
```

Listing 5.6: Instalación de uv en Windows

uv se instala en `%USERPROFILE%\local\bin` y se añade automáticamente al PATH de su usuario. Cierre y reabra su terminal, y luego verifique:

```
1 uv --version
```

5.2.3.2. Linux / WSL

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

Listing 5.7: Instalación de uv en Linux/WSL

uv se instala en `~/.local/bin`. Recargue la configuración de su intérprete de comandos y verifique:

```
1 source ~/.bashrc
2 uv --version
```


5.2.4. Creación y gestión de entornos con uv

La estructura de entorno producida por `uv venv` es idéntica a la de un entorno estándar creado con `python -m venv`. Se encuentran presentes los mismos scripts `Scripts\activate.bat` (Windows) y `bin/activate` (Linux), y la activación no cambia.

5.2.4.1. Creación de un entorno

```
1 uv venv C:\uvenv\ELEVATE Academy --python 3.12
```

Listing 5.8: Creación de un entorno gestionado por uv (Windows)

```
1 uv venv ~/uvenv/ELEVATE Academy --python 3.12
```

Listing 5.9: Creación de un entorno gestionado por uv (Linux/WSL)

La bandera `-python` especifica la versión de Python. Si esa versión no está ya instalada, `uv` la descargará e instalará automáticamente.

5.2.4.2. Activación de un entorno

La activación es idéntica a la de un `venv` estándar:

```
1 C:\uvenv\ELEVATE Academy\Scripts\activate.bat
```

```
1 source ~/uvenv/ELEVATE Academy/bin/activate
```

5.2.4.3. Instalación de paquetes

Dentro de un entorno activado, `uv pip install` reemplaza a `pip install`:

```
1 uv pip install numpy pandas matplotlib
```

Listing 5.10: Instalación de paquetes con uv

5.2.4.4. Declaración de dependencias con `pyproject.toml`

Para proyectos que se compartirán o reproducirán, `uv` funciona mejor con un archivo `pyproject.toml` en la raíz del proyecto que declare las dependencias de forma explícita:

```
1 [project]
2 name = "my-project"
3 version = "0.1.0"
4 requires-python = ">=3.12"
5 dependencies = [
6     "numpy>=1.26",
7     "pandas>=2.2",
8     "matplotlib>=3.9",
```

9]

Listing 5.11: pyproject.toml mínimo

Con este archivo presente, un único comando instala todas las dependencias y genera el archivo de bloqueo:

1 `uv sync`

`uv sync` lee `pyproject.toml`, resuelve el grafo completo de dependencias, instala todos los paquetes y escribe `uv.lock`. Ejecutar `uv sync` nuevamente en otra máquina, o después de un `git pull` que haya añadido nuevas dependencias, reproduce el entorno de forma exacta.

Información

Haga commit de `pyproject.toml` y `uv.lock` en su repositorio. No haga commit de la carpeta del entorno en sí. Añádala a `.gitignore`.

5.2.5. requirements.txt frente a uv.lock

Cuadro 5.1: Comparación entre requirements.txt y uv.lock

Propiedad	requirements.txt	uv.lock
Generado por	pip freeze	uv sync / uv lock
Registra	Solo paquetes instalados	Grafo completo de dependencias
Distingue dependencias directas y transitivas	No	Sí
Verificación por hash	No	Sí
Reproducibilidad entre plataformas	Parcial	Sí
Debe incluirse en git	Sí	Sí

5.3. Nivel 1: archivos de script

Ahora que comprendemos los entornos virtuales, podemos escribir scripts para activarlos automáticamente. El objetivo es crear dos scripts:

1. **ActivateEnv**: un script reutilizable que acepta dos parámetros (nombre del entorno y directorio del proyecto), activa el entorno y abre VS Code.
2. **ELEVATE Academy**: un script específico del proyecto, de una sola línea, que invoca a **ActivateEnv** con los parámetros correctos para el proyecto ELEVATE Academy.

5.3.1. La solución de Windows: archivos por lotes

5.3.1.1. ActivateEnv.bat

La versión de Windows, **ActivateEnv.bat**, se almacena en `C:\penv` y se coloca en el PATH del sistema. Acepta dos parámetros: `%1` (nombre del entorno) y `%2` (directorio del proyecto).

```

1 @echo off
2 setlocal
3
4 :: Lea los parámetros
5 set "ENV_NAME=%~1"
6 set "BASE_DIR=%~2"
7 echo Iniciando la activación del entorno '%ENV_NAME%'...
8
9 :: Valide la entrada
10 if "%ENV_NAME%"==" " (
11     echo [ERROR] No se especificó el nombre del entorno.

```

```
12     goto :eof
13 )
14 if "%BASE_DIR%"==" " (
15     echo [ERROR] No se especificó el directorio base.
16     goto :eof
17 )
18
19 :: Verifique si existe el directorio base
20 if not exist "%BASE_DIR%" (
21     echo [ERROR] Directorio no encontrado: %BASE_DIR%
22     goto :eof
23 )
24
25 :: Cambie al directorio de trabajo
26 cd /d "%BASE_DIR%"
27
28 :: Defina la ruta del script de activación
29 set "VENV_ROOT=C:\penv"
30 set "VENV_ACTIVATE=%VENV_ROOT%\%ENV_NAME%\Scripts\activate.bat"
31
32 :: Active el entorno
33 if exist "%VENV_ACTIVATE%" (
34     echo Activando...
35     call "%VENV_ACTIVATE%"
36     echo Activación completa.
37 ) else (
38     echo [ERROR] No encontrado: %VENV_ACTIVATE%
39     goto :eof
40 )
41
42 :: Abra VS Code en el directorio del proyecto
43 echo Iniciando VS Code...
44 code .
45
46 endlocal
```

Listing 5.12: ActivateEnv.bat: lanzador de entorno reutilizable (Windows)

Elementos clave específicos de Windows:

- `@echo off` suprime el eco de cada comando en la pantalla.
- `setlocal` asegura que los cambios de variables sean locales a este script.
- `%~1` elimina las comillas que rodean al primer argumento.
- `call` se usa para invocar otro archivo por lotes y devolver el control después.
- `goto :eof` salta al final del archivo (una salida limpia).

5.3.1.2. ELEVATE Academy.bat

Este archivo reside en el directorio del proyecto ELEVATE Academy. Simplemente invoca a `ActivateEnv.bat` con los argumentos correctos:

```
1 ActivateEnv ELEVATE Academy "C:\Projects\ELEVATE Academy"
```

Listing 5.13: ELEVATE Academy.bat: lanzador del proyecto (Windows)

Debido a que `C:\penv` está en el PATH del sistema, Windows encuentra `ActivateEnv.bat` automáticamente. Puede ejecutar `ELEVATE Academy.bat` haciendo doble clic en él o escribiendo `ELEVATE Academy` en cualquier ventana de CMD.

5.3.2. La solución de Linux: scripts de intérprete de comandos

5.3.2.1. Comprensión de la sintaxis de los scripts de intérprete de comandos

Un script de intérprete de comandos de Linux es un archivo de texto plano que contiene comandos de Bash. Cada script debe comenzar con una **línea shebang**, una primera línea especial que indica al sistema qué intérprete usar:

```
1 #!/bin/bash
```

Los parámetros pasados a un script se acceden como `$1`, `$2`, `$3`, etc. El nombre propio del script es `$0`.

5.3.2.2. ActivateEnv.sh

```
1 #!/bin/bash
2 # ActivateEnv.sh
3 # Uso: source ~/penv/ActivateEnv.sh <ENV_NAME> <PROJECT_DIR>
4 # Nota: Debe ser *cargado con source*, no ejecutado. Véase la
   ↪ explicación abajo.
5
6 ENV_NAME="$1"
7 BASE_DIR="$2"
8
9 echo "Iniciando la activación del entorno '${ENV_NAME}'..."
10
11 # Valide la entrada
12 if [ -z "$ENV_NAME" ]; then
13     echo "[ERROR] No se especificó el nombre del entorno."
14     return 1
15 fi
16
17 if [ -z "$BASE_DIR" ]; then
18     echo "[ERROR] No se especificó el directorio base."
19     return 1
20 fi
21
22 # Verifique si existe el directorio del proyecto
23 if [ ! -d "$BASE_DIR" ]; then
24     echo "[ERROR] Directorio no encontrado: ${BASE_DIR}"
25     return 1
26 fi
```

```
27
28 # Cambie al directorio del proyecto
29 cd "$BASE_DIR" || return 1
30
31 # Construya la ruta al script de activación
32 VENV_ROOT="$HOME/penv"
33 VENV_ACTIVATE="${VENV_ROOT}/${ENV_NAME}/bin/activate"
34
35 # Active el entorno
36 if [ -f "$VENV_ACTIVATE" ]; then
37     echo "Activando..."
38     # shellcheck source=/dev/null
39     source "$VENV_ACTIVATE"
40     echo "Activación completa."
41 else
42     echo "[ERROR] No encontrado: ${VENV_ACTIVATE}"
43     return 1
44 fi
45
46 # Abra VS Code en el directorio del proyecto (con soporte para
47   ↪ WSL)
48 echo "Iniciando VS Code..."
49 code .
```

Listing 5.14: ActivateEnv.sh: lanzador de entorno reutilizable (Linux/WSL)

Observe varias diferencias respecto de la versión de Windows:

- -z comprueba si una cadena está vacía (longitud cero).
- -d comprueba si una ruta es un directorio.
- -f comprueba si una ruta es un archivo regular.
- \$HOME es el equivalente en Linux de %USERPROFILE%: se expande a su directorio de inicio.
- return 1 sale del script con un código de error (usamos return en lugar de exit por una razón explicada en la siguiente sección).

5.3.2.3. ELEVATE Academy.sh

```
1 #!/bin/bash
2 # ELEVATE Academy.sh
3 # Uso: source ~/ELEVATE Academy/ELEVATE Academy.sh
4 # Inicia el entorno del proyecto ELEVATE Academy.
5
6 source ~/penv/ActivateEnv.sh ELEVATE Academy "$HOME/ELEVATE
  Academy"
```

Listing 5.15: ELEVATE Academy.sh: lanzador del proyecto (Linux/WSL)

5.3.3. Hacer ejecutable un script

En Linux, crear un archivo no lo convierte automáticamente en ejecutable. Debe conceder permiso de ejecución con el comando `chmod`:

```
1 chmod +x ~/penv/ActivateEnv.sh
2 chmod +x ~/ELEVATE Academy/ELEVATE Academy.sh
```

`chmod` significa *cambiar el modo* (change mode). La bandera `+x` añade el permiso de ejecución (`x`). Los permisos de Linux se organizan en torno a tres roles (propietario, grupo y otros), pero para scripts personales, `chmod +x` es casi siempre todo lo que necesita.

5.3.4. Añadir sus scripts al PATH

En Windows, añadió `C:\penv` al `PATH` del sistema para que `ActivateEnv.bat` pudiera encontrarse desde cualquier ubicación. En Linux, hace lo mismo editando el archivo de configuración de su intérprete de comandos.

```
1 # Abra su archivo bashrc en un editor de texto
2 nano ~/.bashrc
3
4 # Añada esta línea al final:
5 export PATH="$HOME/penv:$PATH"
6
7 # Guarde y recargue la configuración
8 source ~/.bashrc
```

Listing 5.16: Añadir `~/penv` al `PATH` en `~/.bashrc`

Después de esto, el intérprete de comandos puede encontrar `ActivateEnv.sh` desde cualquier directorio.

5.3.5. El enfoque con `uv`: `ActivateUVEnv`

Los siguientes scripts reflejan a `ActivateEnv`, pero están dirigidos a entornos gestionados por `uv` almacenados en `C:\uvenv` (Windows) o `~/uvenv` (Linux). Almacene todos los entornos de `uv` en `uvenv` en lugar de en `penv` para mantener las dos cadenas de herramientas visualmente distintas en el disco.

Dos comportamientos distinguen a `ActivateUVEnv` de su contraparte basada en `pip`:

1. **Creación automática.** Si el entorno con el nombre indicado no existe todavía, `uv venv` lo crea antes de la activación. No hay un paso de configuración separado para proyectos nuevos.
2. **Sincronización automática.** Si hay un `pyproject.toml` presente en el directorio del proyecto, `uv sync` se ejecuta después de la activación. Las dependencias se instalan por sí mismas después de cada `git pull` que añada nuevos requisitos, por lo que no se necesita ningún `pip install` manual.

5.3.5.1. ActivateUVEnv.bat (Windows)

Almacene este archivo en C:\uvenv y añada esa carpeta al PATH del sistema.

```
1 @echo off
2 setlocal
3
4 set "ENV_NAME=%~1"
5 set "BASE_DIR=%~2"
6 set "VENV_ROOT=C:\uvenv"
7 set "VENV_PATH=%VENV_ROOT%\%ENV_NAME%"
8 set "VENV_ACTIVATE=%VENV_PATH%\Scripts\activate.bat"
9
10 :: Valide las entradas
11 if "%ENV_NAME%"==" " (
12     echo [ERROR] Uso: ActivateUVEnv ^<env-name^> ^<project-dir^>
13     goto :eof
14 )
15 if "%BASE_DIR%"==" " (
16     echo [ERROR] Uso: ActivateUVEnv ^<env-name^> ^<project-dir^>
17     goto :eof
18 )
19 if not exist "%BASE_DIR%" (
20     echo [ERROR] Directorio del proyecto no encontrado: %BASE_DIR%
21     goto :eof
22 )
23
24 :: Cree el entorno si aún no existe
25 if not exist "%VENV_ACTIVATE%" (
26     echo [INFO] Entorno '%ENV_NAME%' no encontrado. Creándolo con
27     ↪ uv...
28     uv venv "%VENV_PATH%" --python 3.12
29     if errorlevel 1 (
30         echo [ERROR] No se pudo crear el entorno.
31         goto :eof
32     )
33 )
34
35 :: Active
36 cd /d "%BASE_DIR%"
37 call "%VENV_ACTIVATE%"
38
39 :: Sincronice las dependencias si está presente pyproject.toml
40 if exist "pyproject.toml" (
41     echo [INFO] pyproject.toml encontrado. Sincronizando
42     ↪ dependencias...
43     uv sync
44 )
45
46 echo.
47 echo [OK] Entorno '%ENV_NAME%' activo.
48 echo Iniciando VS Code...
49 code .
50
51 endlocal
```


Listing 5.17: ActivateUVEnv.bat: lanzador de entorno uv (Windows)

El uso es idéntico en forma al de ActivateEnv.bat:

```
1 ActivateUVEnv lunar "C:\Dropbox\Research\Lunar\python"
2 ActivateUVEnv forecasting "C:\Projects\forecasting"
```

5.3.5.2. ActivateUVEnv: función de intérprete de comandos para Linux / WSL

Añada la siguiente función a ~/.bashrc. Al igual que su contraparte basada en pip, se ejecuta en el intérprete de comandos actual, de modo que la activación y los cambios de directorio persisten en su sesión.

```
1 # -----
2 # ActivateUVEnv: Activa un venv gestionado por uv y abre VS Code
3 # Uso: ActivateUVEnv <ENV_NAME> <PROJECT_DIR>
4 # -----
5 ActivateUVEnv() {
6     local ENV_NAME="$1"
7     local BASE_DIR="$2"
8     local VENV_ROOT="$HOME/uvENV"
9     local VENV_PATH="${VENV_ROOT}/${ENV_NAME}"
10    local VENV_ACTIVATE="${VENV_PATH}/bin/activate"
11
12    if [ -z "$ENV_NAME" ]; then
13        echo "[ERROR] Uso: ActivateUVEnv <env-name> <project-dir>"
14        return 1
15    fi
16
17    if [ -z "$BASE_DIR" ]; then
18        echo "[ERROR] Uso: ActivateUVEnv <env-name> <project-dir>"
19        return 1
20    fi
21
22    if [ ! -d "$BASE_DIR" ]; then
23        echo "[ERROR] Directorio del proyecto no encontrado:
24        ↪ ${BASE_DIR}"
25        return 1
26    fi
27
28    # Cree el entorno si aún no existe
29    if [ ! -f "$VENV_ACTIVATE" ]; then
30        echo "[INFO] Entorno '${ENV_NAME}' no encontrado.
31        ↪ Creándolo con uv..."
32        uv venv "$VENV_PATH" --python 3.12 || return 1
33    fi
34
35    # Active
36    cd "$BASE_DIR" || return 1
37    # shellcheck source=/dev/null
38    source "$VENV_ACTIVATE"
```

```
37
38 # Sincronice las dependencias si está presente pyproject.toml
39 if [ -f "pyproject.toml" ]; then
40     echo "[INFO] pyproject.toml encontrado. Sincronizando
41         ↳ dependencias..."
42     uv sync
43 fi
44
45 echo "[OK] Entorno '${ENV_NAME}' activo."
46 echo "Iniciando VS Code..."
47 code .
}
```

Listing 5.18: Función de intérprete de comandos `ActivateUVEnv()` para `~/bashrc`

Recargue `.bashrc` después de guardar:

```
1 source ~/.bashrc
```

Luego invóquela desde cualquier directorio:

```
1 ActivateUVEnv lunar "$HOME/Research/Lunar/python"
```

5.4. La diferencia crítica: ejecutar frente a cargar con `source`

Esta sección explica uno de los conceptos más importantes, y a la vez más incomprensidos, de los scripts de intérprete de comandos de Linux, especialmente para desarrolladores que vienen de Windows.

5.4.1. Intérpretes de comandos hijos

Cada vez que usted ejecuta un script con `./script.sh` o simplemente con `script.sh`, Linux lanza un **intérprete de comandos hijo**: un proceso de intérprete de comandos nuevo y separado que hereda una copia de su entorno, ejecuta el script y después *termina*. Cualquier cambio que el script realice en el entorno (activar un venv, cambiar de directorio, establecer una variable) existe únicamente en ese intérprete hijo. Cuando el hijo termina, esos cambios desaparecen. Su terminal original no se ve afectado en absoluto.

Por eso ejecutar `./ActivateEnv.sh` parecería no hacer nada útil: el venv se activa dentro del hijo, luego el hijo termina, y su terminal sigue apuntando al Python global.

5.4.2. Cargar con `source`: ejecución en el intérprete de comandos actual

El comando `source` (o su abreviatura, un único punto `.`) ejecuta un script *en el intérprete de comandos actual*: no se crea ningún hijo. Cada comando se ejecuta como si lo hubiera escrito directamente en la indicación. Esto significa:

- `source /penv/ELEVATE Academy/bin/activate` activa el venv en *su* terminal.
- `cd` cambia *su* directorio de trabajo.
- Las variables establecidas por el script persisten en *su* sesión.

```
1 # Estos dos son equivalentes:  
2 source ActivateEnv.sh ELEVATE Academy ~/ELEVATE Academy  
3 . ActivateEnv.sh ELEVATE Academy ~/ELEVATE Academy  
4  
5 # Esto NO funciona para la activación del venv:  
6 ./ActivateEnv.sh ELEVATE Academy ~/ELEVATE  
   Academy # intérprete hijo; los cambios se pierden
```

5.4.3. El paralelo en Windows

Windows no tiene exactamente esta distinción de la misma manera. El comando `call` en un archivo por lotes ejecuta otro `.bat` en la misma sesión de CMD y regresa, razón por la cual la versión de Windows funciona como se espera. El comportamiento por defecto de Linux de generar un proceso hijo es en realidad el diseño *más seguro*, ya que evita que los scripts modifiquen accidentalmente su entorno, pero le exige optar explícitamente por el comportamiento de sesión compartida mediante `source`.

Nota

Esta es también la razón por la que `ActivateEnv.sh` usa `return` en lugar de `exit` para el manejo de errores. Cuando un script se carga con `source`, `exit` cerraría toda su terminal. `return` sale únicamente del script, dejando su sesión intacta.

5.5. Nivel 2: funciones de intérprete de comandos, el enfoque elegante de Linux

Cargar scripts con `source` es poderoso, pero requiere escribir `source` o `.` cada vez. También implica que su script lanzador debe residir en algún lugar del `PATH` y que el usuario debe recordar la invocación exacta. Linux ofrece una solución más elegante: las **funciones de intérprete de comandos**.

5.5.1. ¿Qué es una función de intérprete de comandos?

Una función de intérprete de comandos es como un script, pero se define directamente dentro de su archivo de configuración del intérprete de comandos (`~/bashrc`). Debido a que forma parte del propio intérprete (y no es un archivo separado), se ejecuta automáticamente en la sesión actual sin necesidad de `source`. Usted la invoca simplemente escribiendo su nombre.

5.5.2. Definición de la función ActivateEnv

Añada lo siguiente al final de ~/.bashrc:

```

1 # -----
2 # ActivateEnv: Activa un venv de Python y abre VS Code
3 # Uso: ActivateEnv <ENV_NAME> <PROJECT_DIR>
4 # -----
5 ActivateEnv() {
6     local ENV_NAME="$1"
7     local BASE_DIR="$2"
8
9     echo "Iniciando la activación del entorno '${ENV_NAME}'..."
10
11     # Valide la entrada
12     if [ -z "$ENV_NAME" ]; then
13         echo "[ERROR] No se especificó el nombre del entorno."
14         return 1
15     fi
16
17     if [ -z "$BASE_DIR" ]; then
18         echo "[ERROR] No se especificó el directorio base."
19         return 1
20     fi
21
22     if [ ! -d "$BASE_DIR" ]; then
23         echo "[ERROR] Directorio no encontrado: ${BASE_DIR}"
24         return 1
25     fi
26
27     local VENV_ACTIVATE="$HOME/penv/${ENV_NAME}/bin/activate"
28
29     if [ ! -f "$VENV_ACTIVATE" ]; then
30         echo "[ERROR] Entorno virtual no encontrado:
31             ↪ ${VENV_ACTIVATE}"
32         return 1
33     fi
34
35     # Cambie de directorio, active y abra VS Code
36     cd "$BASE_DIR" || return 1
37     source "$VENV_ACTIVATE"
38     echo "Entorno '${ENV_NAME}' activado."
39     code .
40 }
```

Listing 5.19: Versión como función de intérprete de comandos de ActivateEnv en ~/.bashrc

La palabra clave `local` restringe las variables al ámbito de la función, de modo que no contaminan la sesión del intérprete de comandos.

5.5.3. Definición de la función ELEVATE Academy

Ahora añade una función de una línea que invoque a `ActivateEnv` para el proyecto ELEVATE Academy:

```

1 # -----
```

```
2 # ELEVATE Academy: Inicia el entorno del proyecto ELEVATE Academy
3 # Uso: ELEVATE Academy
4 # -----
5 ELEVATE Academy() {
6     ActivateEnv "ELEVATE Academy" "$HOME/ELEVATE Academy"
7 }
```

Listing 5.20: Función específica del proyecto para ELEVATE Academy en ~/.bashrc

5.5.4. Activación de las nuevas funciones

Después de editar ~/.bashrc, recárguelo:

```
1 source ~/.bashrc
```

Ahora, desde *cualquier directorio*, simplemente escriba:

```
1 ELEVATE Academy
```

El intérprete de comandos cambiará al directorio de su proyecto ELEVATE Academy, activará el entorno virtual y abrirá VS Code, todo en una sola palabra y sin que se requiera el prefijo `source`.

5.5.5. Por qué Windows no tiene un equivalente limpio

Los archivos por lotes de Windows siempre se ejecutan como procesos hijos; no existe el comando `source` en CMD. PowerShell es lo más cercano: puede definir funciones en su perfil de PowerShell (\$PROFILE) y se comportan de manera similar a las funciones de Bash. Sin embargo, el perfil de PowerShell está deshabilitado por defecto por razones de seguridad y requiere cambiar la política de ejecución. Para los usuarios de Windows que desean esta experiencia de un solo comando de forma nativa, configurar los perfiles de PowerShell es el camino a seguir, pero implica más sobrecarga administrativa que editar ~/.bashrc. Esta es una de las razones prácticas por las que muchos desarrolladores en Windows adoptan WSL.

5.6. Juntando todas las piezas

La Figura 5.2 resume el flujo de trabajo completo en cada plataforma.

5.6.1. La experiencia completa en WSL

Cuando usted escribe `ELEVATE Academy` en su terminal de WSL:

1. Bash busca `ELEVATE Academy` en su lista de funciones conocidas.
2. `ELEVATE Academy()` invoca a `ActivateEnv "ELEVATE Academy" "$HOME/ELEVATE Academy"`.

Cuadro 5.2: Comparación completa del flujo de trabajo: Windows frente a Linux/WSL

Paso	Windows	Linux / WSL
Crear venv	<code>python -m venv C:\penv\ELEVATE Academy</code>	<code>python3 -m venv ~/penv/ELEVATE Academy</code>
Lanzador reutilizable	<code>ActivateEnv.bat</code> en <code>C:\penv</code>	Función <code>ActivateEnv()</code> en <code>~/.bashrc</code>
Lanzador de proyecto	<code>ELEVATE Academy.bat</code> en la carpeta del proyecto	Función <code>ELEVATE Academy()</code> en <code>~/.bashrc</code>
Hacerlo disponible globalmente	Añadir <code>C:\penv</code> al PATH del sistema	Las funciones en <code>.bashrc</code> siempre están disponibles
Iniciar el proyecto	Escribir <code>ELEVATE Academy</code> en cualquier ventana de CMD	Escribir <code>ELEVATE Academy</code> en cualquier terminal
Resultado	venv activo, VS Code abre en la carpeta del proyecto	venv activo, VS Code abre en la carpeta del proyecto (modo WSL)

3. `ActivateEnv()` cambia al directorio del proyecto.
4. El entorno virtual se activa en su intérprete de comandos actual.
5. `code .` inicia VS Code con la extensión de WSL, conectando su interfaz de VS Code en Windows con su backend de Linux.
6. Ahora está trabajando en un entorno de desarrollo completamente configurado con una sola palabra.

5.7. Hoja de referencia rápida

5.8. Resumen

Este capítulo cubrió las herramientas y técnicas para gestionar entornos de Python y automatizar la configuración de proyectos en Windows y Linux:

- **Entornos virtuales de Python:** copias aisladas de Python, una por proyecto, que previenen conflictos de dependencias entre proyectos. Cada entorno tiene su propio conjunto de paquetes instalados, y eliminar uno no tiene efecto sobre ningún otro.
- **La cadena estándar de herramientas pip y venv:** las herramientas respaldadas por la fundación e incluidas con cada instalación de Python para crear entornos, instalar paquetes y registrar dependencias en `requirements.txt`. Conocer esta cadena de herramientas sigue siendo una habilidad esencial independientemente de qué otras herramientas adopte.

- **uv**: una alternativa moderna y de alto rendimiento desarrollada por Astral que ofrece instalaciones drásticamente más rápidas, una caché global de paquetes, archivos de bloqueo reproducibles con hashes criptográficos y soporte nativo para `pyproject.toml`. Debido a que **uv** sincroniza automáticamente las dependencias desde un archivo de bloqueo mediante `uv sync`, la reproducción del entorno se vuelve un único comando en lugar de un proceso manual.
- **Scripting de Nivel 1**: escribir scripts lanzadores reutilizables (`ActivateEnv.bat` y `ActivateEnv.sh` para entornos gestionados por `pip`; `ActivateUVEnv.bat` y la función de intérprete de comandos `ActivateUVEnv()` para entornos gestionados por `uv`) y scripts de una línea específicos del proyecto (`ELEVATE Academy.bat`, `ELEVATE Academy.sh`) que activan el entorno correcto, cambian al directorio del proyecto e inician VS Code en una sola invocación.
- **Scripting de Nivel 2**: funciones de intérprete de comandos definidas en `~/ .bashrc` que proporcionan un comando de inicio de una sola palabra desde cualquier directorio, sin necesidad del prefijo `source`.
- **La distinción entre source y ejecutar**: en Linux, ejecutar un script normalmente genera un intérprete de comandos hijo cuyos cambios de entorno se desvanecen cuando termina. El comando `source` (o su abreviatura `.`) ejecuta el script en el intérprete de comandos actual, razón por la cual la activación del entorno virtual requiere cargarlo con `source`. Esta misma distinción explica por qué `ActivateEnv.sh` usa `return` en lugar de `exit` para el manejo de errores.

El comando `ELEVATE Academy` que construyó en este capítulo es un ejemplo pequeño pero significativo de un principio más amplio: la línea de comandos le permite codificar su flujo de trabajo en scripts repetibles, compartibles y mejorables. Cada paso tedioso de configuración que elimine hoy es carga cognitiva liberada para el trabajo que realmente importa.

Cuadro 5.3: Hoja de referencia de scripts de intérprete de comandos y entornos virtuales

Tarea	Windows (CMD)	Linux / WSL (Bash)
Crear entorno virtual	<code>python -m venv C:\penv\NAME</code>	<code>python3 -m venv ~/penv/NAME</code>
Activar entorno	<code>C:\penv\NAME\Scripts\activate</code>	<code>~/penv/NAME/bin/activate</code>
Desactivar entorno	<code>deactivate</code>	<code>deactivate</code>
Instalar un paquete	<code>pip install pkg</code>	<code>pip install pkg</code>
Guardar lista de paquetes	<code>pip freeze > req.txt</code>	<code>pip freeze > req.txt</code>
Restaurar desde la lista	<code>pip install -r req.txt</code>	<code>pip install -r req.txt</code>
Hacer ejecutable un script	(no es necesario; la extensión lo gestiona)	<code>chmod +x script.sh</code>
Ejecutar un script	<code>script.bat</code>	<code>./script.sh</code>
Cargar con source un script	<code>call script.bat</code>	<code>source script.sh</code>
Editar configuración del intérprete	(sin equivalente en CMD)	<code>nano ~/.bashrc</code>
Recargar configuración del intérprete	(reiniciar CMD)	<code>source ~/.bashrc</code>
Abrir VS Code (WSL)	<code>code .</code>	<code>code .</code>
Parámetro 1 del script	<code>%~1</code>	<code>\$1</code>
Comprobar si una variable está vacía	<code>if "%VAR%"==" "</code>	<code>if [-z "\$VAR"]</code>
Comprobar si existe un directorio	<code>if exist "DIR\"</code>	<code>if [-d "DIR"]</code>
Salir de un script o función	<code>goto :eof</code>	<code>return</code> (en función o script cargado con source)

Cuadro 5.4: Referencia rápida de uv: equivalencias con pip y venv

Tarea	pip / venv	uv
Instalar uv	(no aplicable)	Véase la Sección 5.2
Crear entorno	python -m venv PATH	uv venv PATH --python 3.12
Activar entorno	source PATH/bin/activate	source PATH/bin/activate
Instalar un paquete	pip install pkg	uv pip install pkg
Instalar desde pyproject.toml	pip install -e .	uv sync
Añadir una dependencia	Editar requirements.txt manualmente	uv add pkg
Eliminar una dependencia	Editar requirements.txt manualmente	uv remove pkg
Generar archivo de bloqueo	pip freeze > req.txt	uv lock (o implícito en uv sync)
Instalar desde archivo de bloqueo	pip install -r req.txt	uv sync
Script lanzador (Windows)	ActivateEnv.bat	ActivateUVEnv.bat
Función lanzadora (Linux)	ActivateEnv() en .bashrc	ActivateUVEnv() en .bashrc
Raíz del entorno	C:\penv / ~/penv	C:\uvenv / ~/uvenv

Capítulo 6

Operaciones Numéricas

6.1. Bits, bytes y punto flotante

Los computadores tienen un número finito de formas de representar números. Es decir, solo pueden aproximar un pequeño subconjunto de los números reales. Comprender esto es un componente esencial para realizar operaciones numéricas: usted debe entender cómo y por qué se acumulan los errores numéricos, y cuándo interfieren con un cálculo.

La unidad de representación computacional es un **bit** (unidad representada por b), es decir, un circuito que está abierto o cerrado. Ocho bits se agrupan en un **byte** (unidad representada por B), como se muestra en la Figura 1. Este número evolucionó a partir de las arquitecturas computacionales en los primeros días de la computación, y fue estandarizado por primera vez por la ISO/IEC 2382-1:1993. En un byte, las posiciones van de 0 a 7, usualmente representadas de derecha a izquierda. Un 1 en la primera posición representa 2^0 , en la segunda posición es 2^1 , y así sucesivamente.

Una manera ingenua de representar números con un computador es asignar N dígitos decimales para cada número. Por ejemplo:

$$012.345$$

Este ejemplo representa un sistema de punto fijo de 6 dígitos. Este sistema de punto fijo de 6 dígitos produce errores en algunos casos de suma:

$$999.999 + 999.999 = \text{DESBORDAMIENTO}$$

También se producen errores en la multiplicación y la división

$$013.000/003.000 = 004.333$$

En este caso, perdemos todos los dígitos después del tercer decimal.

Podemos permitir que el punto decimal «flote» para ganar exactitud con el mismo número de dígitos. Por ejemplo:

$$013.000/003.000 = 4.33333$$

Incluso podríamos representar números más grandes que los que 6 dígitos nos permitían antes:

$$999.999 \times 999.999 = 9.99998 \times 10^5$$

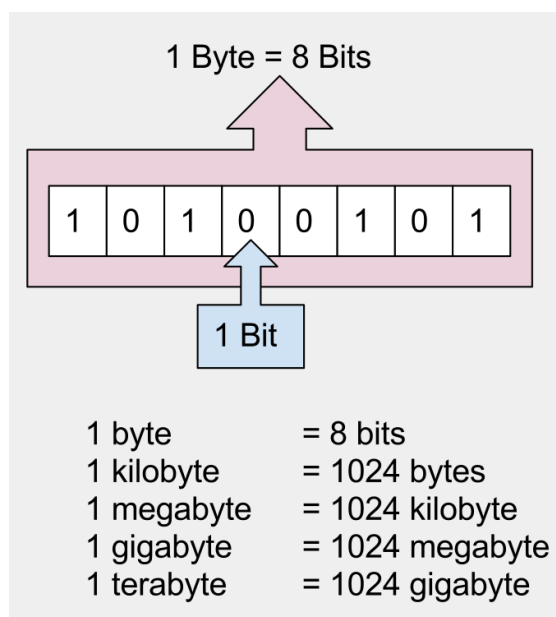


Figura 1: Bits y bytes [?]. Leyendo de izquierda a derecha, el ejemplo representa: $2^0 + 2^2 + 2^5 + 2^7 = 1 + 4 + 32 + 128 = 165$. Si usted lee de derecha a izquierda (esta es la forma estándar de representar bits), obtendría $2^0 + 2^2 + 2^5 + 2^7 = 165$. En este caso, de izquierda a derecha y de derecha a izquierda producen el mismo resultado. Pero en general no tienen que ser iguales. ¿Qué ocurre si usted reemplaza algún cero por un uno y calcula de nuevo en ambas direcciones? Así, la representación importa, y usted necesita conocer la convención.

6.1.1. El estándar IEEE para aritmética de punto flotante

El estándar IEEE para aritmética de punto flotante (IEEE 754) es un estándar técnico para el cálculo en punto flotante, establecido en 1985 por el Instituto de Ingenieros Eléctricos y Electrónicos (IEEE).

- Los números finitos pueden ser en base 2 (binario) o base 10 (decimal).
- Cada número finito se describe mediante tres enteros:
 1. s = un signo (cero o uno),
 2. c = un significando (o «coeficiente»),
 3. q = un exponente.
- El valor numérico de un número finito es $(-1)^s \times c \times bq$ donde b es la base (2 o 10), también llamada radix.

Los tres campos en un flotante IEEE-754. Los números de punto flotante binarios se almacenan en una forma de signo-magnitud donde el bit más significativo es el bit de *signo*, *exponente* es el exponente sesgado, y *fracción* es el significando menos el bit más significativo. La Figura 2 muestra un número de 32 bits, también conocido como número de punto flotante de precisión simple, mientras que la Figura 3 representa un número de 64 bits, también conocido como número de punto flotante de doble precisión.

Si queremos representar un número real con un número finito N de posiciones de memoria, podemos reservar una posición para el signo, $N - k - 1$ para la parte

Cap. X.

ALGEBRA.

41

variatur. Et quidem Augetur, si majore multiplicatore multiplicetur Numerator, quam Denominator: Minuitur, si contra.

Sive (quod eodem recidit)

Si Fractionis duo termini, (Numerator & Denominator,) ita (addendo) augentur ambo, ut augmenta sint ipsis terminis proportionalia; idem qui prius manet fractionis Valor: Augetur vero, si Numeratoris augmentum, ad augmentum Denominatoris, majorem rationem habeat, quam habet ipse Numerator ad Denominatorem: Minuitur, si minorem.

Quodque de Numeratore & Denominatore Fractionis dictum est, similiter intelligendum est de Rationis Antecedente & Consequente, & pariter in sequentibus.

$$\begin{array}{l} \text{Put, } \frac{1}{3} = \frac{1 \times 2}{3 \times 2} = \frac{2}{6} = \frac{3}{9} = \frac{4}{12} \text{ & c. } \quad \frac{n}{d} = \frac{2n}{2d} = \frac{3n}{3d} = \frac{an}{ad} \\ \text{Sen, } \frac{1}{3} = \frac{1+1}{3+3} = \frac{1+2}{3+6} = \frac{1+3}{3+9} \text{ & c. } \quad \frac{n}{d} = \frac{n+n}{d+d} = \frac{n+2n}{d+2d} = \frac{n+3n}{d+3d} \\ \text{Sed, } \frac{1}{3} < \frac{1 \times 3}{3 \times 2} = \frac{3}{6} = \frac{1+2}{3+3} \quad \frac{n}{d} < \frac{3n}{2d} = \frac{n+2n}{d+d} \\ \text{Et, } \frac{1}{3} > \frac{1 \times 2}{3 \times 3} = \frac{2}{9} = \frac{1+1}{3+6} \quad \frac{n}{d} > \frac{2n}{3d} = \frac{n+n}{d+2d} \end{array}$$

SOLUTIO.

His premis; si datae fractionis, ad minimos terminos (communi divisione per maximam communem mensuram) reducta, tum Numerator, tum Denominator, dato numero non sit major: fit quod imperatur. Quippe Fractio est in minimis terminis, & qualis requiritur.

Si Fractio, sic reducta, terminus vel alter vel uterque sit exposito numero major: quaerenda est fractio quae sit quam exposita vel proxime major, vel proxime minor, terminos habens exposito numero non majores; ea quae sequitur methodo.

Pro Proxime-Majori.

Pro invenienda proxime majori, sic procedo.

Denominatorem fractionis expositae (aut ejus ad quam reducitur, si praecesserit reductio,) per Numeratorem divido; ut habeam, qui Numeratori 1 respondeat, Denominatorem, in integris cum annexis partibus decimalibus, tam accuratum quam videbitur expedire.

Put, 2684769) 8376571 (3.12003416 +.

Nam, ut 2684769 ad 8376571, sic 1 ad 3.12003416 +.

Adeoq, pro exposita fractione $\frac{8376571}{2684769}$, hanc habeo ei (quam prope censetur necessarium) aequipollentem $\frac{312003416}{2684769}$ quae numeratorem habet 1. Quam Fractionem primam completam appello; eandemque, neglectis partibus decimalibus, appello fractionem primam truncatam. Ejusque partes decimales abscissas, 0.12003416, Appendicem voco, sive Mantissam fractionis primae, seu Mantissam primam.

Estque haec truncata Fractio $\frac{1}{3}$, tum justo major (propter denominatorem justo minorem, utpote partibus decimalibus mulctatum;) tum proxime-major, omnium habentium numeratorem 1, & denominatorem integrum, (nam $\frac{1}{3}$ est adhuc major, & $\frac{1}{4}$ justo minor, & de reliquis similiter:) Neque, ex justo majoribus, propius ad verum valorem accedere potest ullus, nisi aucto numeratore 1.

F

Loco

Figura 4: Página 41 de «A Treatise of Algebra», donde se introduce la palabra mantisa. J. Wallis. *Opera mathematica: De Algebra Tractatus*. Número v. 5 Anno 1685 Anglice editus; Nunc Auctus Latine. Theatrum Sheldonianum, 1693.

$1 - 1/3 - 2/3$ resulta algebraicamente en cero. Sin embargo,

$$1 - 1/3 - 2/3 = 1.1102e - 16,$$

$$1 - 2/3 - 1/3 = 5.5511e - 17.$$

Así, no solo la operación no es cero, sino que además el orden de los sumandos importa, es decir, la suma en punto flotante no es conmutativa. Por lo tanto, $1 - 1/3 - 2/3 \neq 1 - 2/3 - 1/3 \neq 0$

Debido a que los números de punto flotante no pueden representar todos los números reales, se producen errores de representación en muchos casos. Pero, ¿por qué la operación no es conmutativa? [Registre su explicación como respuesta n.º 14.](#)

La siguiente instrucción compara si $10^{30} + 1 - 10^{30}$ es igual al valor 1. Usando aritmética elemental, la respuesta debería ser un rotundo *sí*. En el lenguaje Python, el exponente se representa mediante un doble asterisco, como en Fortran, en lugar del símbolo de acento circunflejo (^) usado en la mayoría de las demás plataformas de cómputo. El doble signo igual prueba si los dos lados son iguales, y devuelve un valor booleano de *verdadero* o *falso*. Por lo tanto, la pregunta se traduce en la siguiente instrucción:

```
1 10**30 + 1 - 10**30 == 1
```

Esta instrucción en particular devuelve un valor booleano de *verdadero* en Python y *falso* en Matlab (la sintaxis correcta de Matlab es $10^30 + 1 - 10^30 == 1$). Sin embargo, la siguiente sentencia:

```
1 10**30 + 1. - 10**30 == 1
```

devuelve *falso* en Python. ¿Por qué? [Registre su explicación como respuesta n.º 15.](#)

Una operación como

```
1 10**20 + 1 - 10**20 == 1
```

devuelve resultados distintos en Matlab, C, Fortran (*falso*) frente a Python (*verdadero*). Podría ser tentador declarar a Python el ganador en el cómputo numérico, pero lo que realmente sucede es que Python no sigue estrictamente el estándar IEEE-754. Para conocer más sutilezas del punto flotante en Python, visite <https://docs.python.org/3/tutorial/floatpoint.html> En la mayoría de los casos, las desviaciones del estándar en el intérprete de Python son inofensivas y tienen un origen ideológico más que técnico. Sin embargo, es concebible que algoritmos extremos desarrollados bajo el estándar IEEE tendrían que ser adaptados cuidadosamente en Python. Existen alternativas, pero están fuera del alcance de este capítulo. Por ahora, basta decir que existen campos minados en el paisaje de Python.

Podemos explorar más discrepancias en valores extremos de punto flotante examinando en dos entornos numéricos los resultados de $10 \cdot 10^b + 1 - 10 \cdot 10^b$

para diferentes valores de b . Descargue e instale Python, Matlab, R, o compile un programa en C/C++ que ejecute las mismas operaciones. Compare lo que ocurre en estos entornos cuando elige $b \in \{10, 1e2, 1e10, 1e100, 1e1000\}$. Describa las discrepancias y analícelas. **Registre su explicación como respuesta n.º 16.**

6.2. Introducción a NumPy

Para ejecutar operaciones matemáticas más allá de las operaciones básicas con números reales, usted necesita importar una biblioteca numérica. Las operaciones matriciales se ejecutan mediante funciones en NumPy. Por lo tanto, el primer paso es cargar NumPy.

```
1 In : import numpy as np
```

La biblioteca NumPy ofrece acceso fácil a constantes de uso común, tales como:

Constante	Valor
np.e	Devuelve el número e
np.pi	Devuelve el número π
np.inf	Devuelve el número ∞
np.nan	NaN, no-es-un-número

Con NumPy, ahora podemos acceder a funciones matemáticas de uso común. Entre otras:

Función	Operación
np.abs(x)	Valor absoluto $ x $
np.sqrt(x)	Raíz cuadrada $\sqrt{x}$
np.exp(x)	Función exponencial e^x
np.log(x)	$\log(x)$ donde x es un número real positivo
np.sin(x)	$\sin(x)$ donde se asume que x está en radianes
np.cos(x)	$\cos(x)$ donde se asume que x está en radianes

Las expresiones usan los operadores aritméticos y las reglas de precedencia que usted ya conoce a estas alturas. También hay funciones, como coseno o seno. Usted rodea la entrada de la función con paréntesis. Por ejemplo, para calcular la raíz cuadrada de dos, se escribe

```
1 In : np.sqrt(2)
2 Out: 1.4142135623730951
```

Sin embargo, usted no puede usar la función `sqrt` con números negativos. ¿Qué tendría que hacer para permitir a Python calcular la raíz cuadrada de un número negativo y devolver un número complejo? **Registre su explicación como respuesta n.º 17.**

La biblioteca NumPy tiene *muchas* funciones. Se puede encontrar una lista exhaustiva en <https://numpy.org/doc/stable/reference/>. Usted observará que inmediatamente debajo del comando que ingresó, la indicación mostrará

`Out[n]: 3.0`, donde `n` representa el número de secuencia del comando (recuerde, el primer comando ejecutado tiene $n = 1$, el segundo $n = 2$, y así sucesivamente).

Ahora ingrese el siguiente comando:

```
1 x = np.random.standard_normal(3)
```

No hay salida. ¿Por qué piensa usted que este es el comportamiento? [Registre su explicación como respuesta n.º 18](#). En el lado derecho de la ventana de Spyder hay una pestaña llamada *Variable Explorer*. Mostrará el valor de x . Alternativamente, usted puede ingresar

```
1 print(x)
```

Alternativamente, usted puede imprimir el resultado directamente sin asignarlo a una variable. Pruebe

```
1 print(np.random.standard_normal(3))
```

¿Por qué los resultados de esta última operación son diferentes a los valores almacenados en la variable x ? [Registre su explicación como respuesta n.º 19](#).

6.2.1. Operaciones matriciales

El elemento de datos básico en NumPy es un arreglo multidimensional. A veces se atribuye un significado especial a las matrices de 1 por 1, que se denominan escalares, y a las matrices con una sola fila o columna, que se denominan vectores. Python tiene otras maneras de almacenar datos tanto numéricos como no numéricos, pero al principio es usualmente mejor pensar en todo como una matriz.

Los vectores fila se delimitan en la notación con corchetes. Una coma separa los números individuales en un vector fila. Múltiples vectores fila del mismo tamaño se separan por comas, y se delimitan con corchetes para definir una matriz. Es una práctica común usar una letra mayúscula al nombrar matrices para distinguirlas de un vector único, escalar o valor de variable.

Ahora, crearemos un vector y una matriz para demostrar operaciones matriciales simples

Función	Operación
$x = np.array([1, 2, 3, 4])$	Crea el vector fila $\mathbf{x} = [1, 2, 3, 4]$
$b = np.array([1, 2, 3])$	Crea el vector fila $\mathbf{b} = [1, 2, 3]$
$A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])$	Crea la matriz $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
$B = np.ones([2, 3])$	Crea la matriz $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
$C = np.zeros([2, 3])$	Crea la matriz $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

Los siguientes comandos demuestran casos básicos de manipulación de vectores:

Operación	Resultado
$x[-1]$	Devuelve 4, el último elemento de x
$x[1]$	Devuelve 2, el segundo elemento de x
$x[-2]$	Devuelve 3, el penúltimo elemento de x

El operador de dos puntos ($:$) es notable. En notación vectorial, indica un rango de filas o columnas. En Python, el primer elemento de un vector tiene índice 0; el segundo elemento tiene índice 1, y así sucesivamente. El operador de dos puntos $a : b$ comienza en la fila o columna de índice a y cubre hasta un índice menor que b . Así, $A[0 : 2, 0 : 2]$ devuelve una matriz compuesta por las filas 1 a 2 y las columnas 1 a 2 con respecto a la matriz A .

Los siguientes comandos demuestran casos básicos de manipulación de matrices:

Operación	Resultado
$A[1, 2]$	Valor de la 2 st fila y 3 rd columna de A
$A[0 : 2, 0 : 2]$	Devuelve $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
$A[1 : 3, 1 : 3]$	Devuelve $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

La matriz A tiene tres filas y tres columnas. Entonces, ¿por qué $A[1 : 100, 1 : 100]$ no produce un error? [Registre su explicación como respuesta n.º 20.](#)

El tamaño o dimensión de una matriz se refiere al número de filas y al número de columnas que tiene una matriz. El comando `shape` devuelve el número de filas y columnas de una matriz. Observe que el resultado es un vector: una matriz unidimensional con 2 valores donde el primero es el número de filas y el segundo valor es el número de columnas. Al acceder a cada elemento de este vector, podemos acceder a cada dimensión individualmente.

Usando la matriz A de nuestros ejemplos anteriores,

```

1 In : y = A.shape()
2 In : y
3 Out : (3,3)
4 In : y[0]
5 Out : 3
6 In : y[1]
7 Out : 3
```

El siguiente ejemplo ilustrará una sutileza al copiar matrices:

```

1 In : D = A
2 Out :
3 array([[1, 2, 3],
4        [4, 5, 6],
5        [7, 8, 9]])
6 In : D[0,0] = 9
7 In : A
8 Out :
9 array([[9, 2, 3],
10       [4, 5, 6],
11       [7, 8, 9]])
```

Observe que al actualizar un valor en D , el valor correspondiente en A cambió. ¿Por qué? ¿Qué necesitaría hacer para evitar que una actualización en D modifique a A ? **Registre su explicación como respuesta n.º 21.**

Las operaciones matriciales básicas se listan a continuación, usando las matrices de los ejemplos anteriores.

Operación	Resultado
$A.T$	Transpuesta de A .
$A.conj().T$	Transpuesta conjugada de A .
$A@B.T$	Multiplicación matricial $A \cdot B$.
$A \star A$	Multiplicación elemento a elemento
A/A	División elemento a elemento
$A \star \star 2$	Exponenciación elemento a elemento

NumPy ofrece una forma fácil de resolver sistemas de ecuaciones lineales. Dada la matriz A y el vector $\mathbf{b}$ de los ejemplos anteriores, el sistema lineal $A\mathbf{x} = \mathbf{b}$ se puede resolver invocando `np.linalg.solve` de la siguiente manera

```
1 In : x = np.linalg.solve(A,b)
2 Out: array([-0.23333333,  0.46666667,  0.1          ])
```

Es fácil usar la multiplicación matricial $A\mathbf{x}$ para verificar la respuesta

```
1 In : A@x
2 Out: array([1.,  2.,  3.] )
```

6.3. Tareas específicas para este laboratorio

El siguiente capítulo es una clase sobre el análisis de componentes principales (PCA, por sus siglas en inglés), una de las técnicas más básicas para el análisis de datos. Implemente un análisis de componentes principales de acuerdo con el siguiente procedimiento (use NumPy para realizar las operaciones matriciales):

1. Seleccione el peso al nacer junto con otras variables numéricas del conjunto de datos NCHS. Llamaremos a esto la *matriz de características*.
2. Asegúrese de que la media sea cero para cada variable. ¿Cómo logra esto? **Registre su explicación como respuesta n.º 22.** Llamaremos a esto la *matriz de características normalizada*.
3. Calcule el producto interno de la matriz de características normalizada. ¿Cuál es la dimensión de la matriz resultante? **Registre su explicación como respuesta n.º 23.**
4. Calcule los valores propios y los vectores propios para el producto interno de la matriz de características normalizada. Necesitará ordenar los valores propios por magnitud. ¿Cómo hace esto en Python? **Registre su explicación como respuesta n.º 24.**

5. Proyecte los datos de la matriz de características a lo largo de los vectores propios correspondientes a los dos valores propios más grandes. Grafique cada observación en este espacio proyectado según el cuartil del peso al nacer. Es posible que necesite elegir diferentes variables hasta que surja un patrón aproximado. Interprete la gráfica. **Registre su explicación como respuesta n.º 25.**

Observe que no se espera que usted simplemente llame a una biblioteca de PCA.

Capítulo 7

Introducción a L^AT_EX

L^AT_EX es un sistema de preparación de documentos en el que el autor escribe texto plano intercalado con comandos de marcado, y un compilador transforma ese código fuente en un PDF compuesto tipográficamente. Esto contrasta con procesadores de texto como Microsoft Word o Google Docs, donde el formato se aplica de manera interactiva en un editor de tipo «lo que ve es lo que obtiene» (WYSIWYG). La propiedad clave que convierte a L^AT_EX en el estándar en ciencia y matemáticas es su capacidad de representar notación matemática arbitrariamente compleja con precisión y consistencia. Cada documento producido en este programa es un documento L^AT_EX.

7.1. Su primer documento L^AT_EX

Un archivo fuente de L^AT_EX es un archivo de texto plano con la extensión `.tex`. Así como `hello.py` era un archivo de texto plano que Python interpretaba, `hello.tex` es un archivo de texto plano que el compilador de L^AT_EX procesa para producir un PDF. Se aplican las mismas reglas sobre editores de texto (véase la Sección 4.1): utilice Notepad, TextEdit en modo de texto plano o `gedit`; nunca un procesador de texto.

Información

Ejercicio L-1: Su primer documento L^AT_EX Abra un editor de texto plano (véase la Tabla 4.1), escriba exactamente lo siguiente y guarde el archivo como `hello.tex`:

```
1 \documentclass{article}
2 \begin{document}
3
4 Hello, world. This is my first \LaTeX{} document.
5
6 Here is some mathematics:  $e^{i\pi} + 1 = 0$ .
7
8 \end{document}
```

Cada línea del listado cumple un propósito específico:

- `\documentclass{article}` declara la clase de documento. Para la mayoría de los informes y artículos, `article` es la elección correcta.
- `\begin{document}` y `\end{document}` delimitan el cuerpo. Todo lo que esté fuera de este par constituye el *preámbulo* (se aborda en la Sección 7.3).
- El texto entre los delimitadores es el contenido visible.
- $e^{i\pi} + 1 = 0$ es el modo matemático en línea: el texto entre signos de dólar se compone como matemáticas.

Cuadro 7.1: Compilación de un documento $\text{\LaTeX}$ en cada plataforma

Plataforma	Comando
Windows (Símbolo del sistema o Git Bash)	<code>pdflatex hello.tex</code>
macOS (Terminal)	<code>pdflatex hello.tex</code>
Linux	<code>pdflatex hello.tex</code> o <code>latexmk -pdf hello.tex</code>

Tenga en cuenta que, con frecuencia, se requieren dos pasadas de compilación para documentos con referencias cruzadas o un índice general; `latexmk` se encarga de esto automáticamente.

La ejecución de `pdflatex hello.tex` produce varios archivos. El importante es `hello.pdf`, que puede abrirse en cualquier visor de PDF. Los archivos `.aux`, `.log` y `.out` son auxiliares; pueden eliminarse sin problema.

7.2. Un ejemplo más completo

El Ejercicio L-1 produjo el documento mínimo viable. Esta sección presenta un documento más realista que demuestra las características más necesarias en la escritura académica: una jerarquía estructurada de secciones, un índice general generado automáticamente, citas en el texto enlazadas a una bibliografía y la gama completa de modos de presentación matemática. Cada característica mostrada aquí se utiliza en los documentos producidos en este programa.

Información

Ejercicio L-2: Un documento estructurado con matemáticas y bibliografía
Cree dos archivos en la misma carpeta. El primero es el documento principal, `report.tex`. El segundo es la base de datos bibliográfica, `sample.bib`. Ambos archivos deben estar en el mismo directorio para que la bibliografía se resuelva.

Archivo 1: `report.tex`

```
1 \documentclass{article}
2
```

```
3 \usepackage{amsmath}    % entornos matematicos extendidos
4 \usepackage{booktabs}  % tablas profesionales
5 \usepackage{hyperref}  % enlaces y referencias cruzadas
   navegables
6
7 \title{A Short Report on Mathematical Notation}
8 \author{Your Name}
9 \date{\today}
10
11 \begin{document}
12
13 \maketitle
14 \tableofcontents
15 \newpage
16
17 \section{Introduction}
18
19 This report demonstrates document structure and
   mathematical
20 notation in \LaTeX{}. For background on literate
   programming,
21 see Knuth~\cite{knuth1984literate}. The \LaTeX{} system
   itself
22 is described in~\cite{lamport1994latex}.
23
24 \subsection{Motivation}
25
26 Scientific writing requires precise notation. \LaTeX{}
   provides
27 that precision through its mathematical typesetting
   engine.
28
29 \subsubsection{A Note on Style}
30
31 Use display equations for any expression that warrants
   visual
32 emphasis, and inline math for symbols embedded in prose.
33
34 \section{Mathematical Display Modes}
35
36 \subsection{Inline Math}
37
38 Inline math is enclosed in single dollar signs:  $f(x) =$ 
    $x^2 + 1$ .
39 It flows within the surrounding text without a line
   break.
40
```

```

41 \subsection{Display Math (unnumbered)}
42
43 The preferred form for unnumbered display equations is
44 the
45 \texttt{\textbackslash[...\textbackslash]} environment:
46 \[
47 \int_{-\infty}^{\infty} e^{-x^2} \, dx = \sqrt{\pi}
48 \]
49
50 The legacy \texttt{\$ \$... \$ \$} form produces the same
51 visual
52 result but is deprecated in \LaTeX{} and should be
53 avoided in
54 new documents.
55
56 \subsection{Numbered Equations}
57
58 The \texttt{equation} environment produces a numbered
59 equation
60 that can be cross-referenced:
61
62 \begin{equation}
63 \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i
64 \quad \label{eq:mean}
65 \end{equation}
66
67 Equation~\ref{eq:mean} defines the sample mean. An
68 approximation
69 for the standard deviation is given
70 in~\cite{gurland1971simple}.
71
72 \subsection{Multi-line Aligned Equations}
73
74 The \texttt{align} environment aligns multiple
75 equations at a
76 designated column, marked with \texttt{\&}:
77
78 \begin{align}
79 \quad \mu &= \mathbb{E}[X] & \label{eq:mu} \\
80 & \\
81 \quad \sigma^2 &= \mathbb{E}[(X-\mu)^2] & \label{eq:var}
82 \end{align}
83
84 Equations~\ref{eq:mu} and~\ref{eq:var} define the mean
85 and
86 variance of a random variable  $X$ .

```

```

79 \section{A Simple Table}
80
81
82 \begin{table}[htbp]
83 \centering
84 \caption{Comparison of equation display modes}
85 \label{tab:modes}
86 \begin{tabular}{lll}
87 \toprule
88 Mode & Syntax & Use case \\
89 \midrule
90 Inline &  $\text{\texttt{\$...\$}}$  &
91 Symbols in prose \\
92 Unnumbered &  $\text{\texttt{\backslash\textbackslash[...]}}$  &
93 & Emphasis, no reference needed \\
94 Numbered &  $\text{\texttt{equation}}$  &
95 Cross-referenceable \\
96 Aligned &  $\text{\texttt{align}}$  &
97 Multi-line derivations \\
98 \bottomrule
99 \end{tabular}
100 \end{table}
101
102 \bibliographystyle{plain}
103 \bibliography{sample}
104
105 \end{document}

```

Archivo 2: sample.bib

```

1 @article{knuth1984literate,
2   author   = {Knuth, Donald E.},
3   title    = {Literate Programming},
4   journal  = {The Computer Journal},
5   year     = {1984},
6   volume   = {27},
7   number   = {2},
8   pages    = {97--111}
9 }
10
11 @book{lamport1994latex,
12   author    = {Lamport, Leslie},
13   title     = {\LaTeX: A Document Preparation System},
14   edition   = {2nd},
15   publisher = {Addison-Wesley},
16   year      = {1994}
17 }

```



```

18
19 @article{gurland1971simple,
20   author = {Gurland, John and Tripathi, Ram C.},
21   title  = {A Simple Approximation for Unbiased
22             Estimation
23             of the Standard Deviation},
24   journal = {The American Statistician},
25   year    = {1971},
26   volume  = {25},
27   number  = {4},
28   pages   = {30--32}
29 }

```

Advertencia

La compilación con bibliografía requiere cuatro pasos. Una única pasada de `pdflatex` no es suficiente cuando el documento contiene comandos `\cite{}`. La secuencia completa es:

```

pdflatex report.tex
bibtex report
pdflatex report.tex
pdflatex report.tex

```

La primera pasada crea el archivo `.aux` que enumera todas las claves de cita. `bibtex` lee el archivo `.aux` y la base de datos `.bib` para producir un archivo `.bbl` que contiene la bibliografía formateada. La segunda y tercera pasadas de `pdflatex` incorporan la bibliografía y resuelven todas las referencias cruzadas. La ejecución de `latexmk -pdf report.tex` realiza todas las pasadas necesarias de manera automática.

7.3. Estructura del documento

Todo archivo fuente de $\text{\LaTeX}$ consta de dos partes: el *preámbulo* (todo lo que antecede a `\begin{document}`) y el *cuerpo* (todo lo que se encuentra entre `\begin{document}` y `\end{document}`). El preámbulo declara la clase de documento, carga paquetes con `\usepackage{...}` y establece los metadatos. El cuerpo contiene el contenido visible, organizado mediante comandos de seccionamiento.

```

1 \documentclass{article}
2
3 % --- Preambulo ---
4 \usepackage{amsmath}      % matematicas extendidas
5 \usepackage{graphicx}     % figuras
6 \usepackage{booktabs}     % tablas profesionales

```

Cuadro 7.2: Modos de presentación de ecuaciones en $\text{\LaTeX}$

Modo	Sintaxis	Estilo de salida	Notas
En línea	$\dots$	Dentro del texto, compacto	Se utiliza para símbolos y expresiones breves en la prosa
Presentación (heredado)	$\dots$	Centrado, tamaño completo	Obsoleto; evítese en documentos nuevos
Presentación (preferido)	$[\dots]$	Centrado, tamaño completo	Se utiliza para ecuaciones de presentación sin numerar
Numerado	$\begin{equation} \dots \end{equation}$	Centrado con número	Referenciable mediante $\backslash label/\backslash ref$
Alineado de varias líneas	$\begin{align} \dots \end{align}$	Varias líneas, alineadas en &	Cada línea puede llevar su propia $\backslash label$

```

7 \usepackage{hyperref} % enlaces navegables
8
9 \title{My First Report}
10 \author{Alice Smith}
11 \date{\today}
12
13 % --- Cuerpo ---
14 \begin{document}
15
16 \maketitle
17
18 \section{Introduction}
19 This is the introduction.
20
21 \subsection{Background}
22 Some background material.
23
24 \subsubsection{Details}
25 More specific details.
26
27 \section{Methods}
28 Description of methods.
29
30 \end{document}

```

Listing 7.1: Documento mínimo compilable con preámbulo y seccionamiento

El comando `\usepackage{...}` carga paquetes de extensión. Los paquetes más relevantes para este programa son `amsmath` (matemáticas extendidas), `graphicx` (figuras), `booktabs` (tablas) y `hyperref` (enlaces navegables).

7.4. Matemáticas en L^AT_EX: una hoja de referencia

L^AT_EX distingue dos modos matemáticos. El modo en línea ($\$. . \$$) compone matemáticas dentro de una línea de texto. El modo de presentación ($\backslash[. . \backslash]$ o el entorno `equation`) centra las matemáticas en su propia línea y, al utilizar `equation`, asigna un número de ecuación. Utilice el modo de presentación para cualquier expresión que merezca su propio énfasis visual, y el modo en línea para símbolos incrustados en la prosa.

7.4.1. Notación matemática común

Cuadro 7.3: Notación matemática común en L^AT_EX

Categoría	Código fuente L ^A T _E X	Renderizado	Notas
Modos en línea y de presentación			
Matemática en línea	$\$x + y = z\$$	$x + y = z$	Dentro de una oración
Matemática en presentación	$\backslash[x + y = z \backslash]$	(bloque centrado)	Sin número
Ecuación numerada	$\backslashbegin{equation}...\backslashend{equation}$	(centrado)	Referenciable
Aritmética básica			
Suma	$a + b$	$a + b$	
Resta	$a - b$	$a - b$	
Multiplicación (punto)	$a \backslashcdot b$	$a \cdot b$	Preferido en matemáticas
Multiplicación (cruz)	$a \backslashtimes b$	$a \times b$	Vectores, producto vectorial
División	$\backslashfrac{a}{b}$	$\frac{a}{b}$	Utilice siempre $\backslashfrac$ en presentación
Superíndice	$x^{\{2\}}$	x^2	Se requieren llaves para >1 carácter
Subíndice	$x_{\{i\}}$	x_i	Se requieren llaves para >1 carácter
Ambos	$x_{\{i\}}^{\{2\}}$	x_i^2	
Funciones comunes			
Raíz cuadrada	$\backslashsqrt{x}$	$\sqrt{x}$	
Raíz n -ésima	$\backslashsqrt[n]{x}$	$\sqrt[n]{x}$	
Exponencial	$e^{\{x\}}$ o $\backslashexp(x)$	e^x	$\backslashexp$ preferido en presentación
Logaritmo natural	$\backslashln x$	$\ln x$	

Continúa en la página siguiente

Categoría	Código fuente $\LaTeX$	Renderizado	Notas
Logaritmo en base b	<code>\log_{b} x</code>	$\log_b x$	
Seno	<code>\sin x</code>	$\sin x$	Utilice <code>\sin</code> , no <code>sin</code>
Coseno	<code>\cos x</code>	$\cos x$	
Valor absoluto	<code> x </code> o <code>\lvert x \rvert</code>	$ x $	<code>\lvert</code> se escala con el contenido
Parte entera inferior	<code>\lfloor x \rfloor</code>	$\lfloor x \rfloor$	
Parte entera superior	<code>\lceil x \rceil</code>	$\lceil x \rceil$	
Sumatoria e integración			
Sumatoria	<code>\sum_{i=1}^n x_i</code>	$\sum_{i=1}^n x_i$	Límites arriba/abajo en presentación
Productoria	<code>\prod_{i=1}^n x_i</code>	$\prod_{i=1}^n x_i$	
Integral	<code>\int_a^b f(x) \, dx</code>	$\int_a^b f(x) dx$	<code>\,</code> , añade un espacio fino antes de dx
Límite	<code>\lim_{x \rightarrow 0} f(x)</code>	$\lim_{x \rightarrow 0} f(x)$	
Letras griegas (comunes en bioestadística y aprendizaje automático)			
alfa	<code>\alpha</code>	α	Nivel de significancia, tasa de aprendizaje
beta	<code>\beta</code>	β	Coefficientes de regresión
gamma	<code>\gamma</code> , <code>\Gamma</code>	γ, Γ	Distribución gamma
delta	<code>\delta</code> , <code>\Delta</code>	δ, Δ	Cambio, delta de Dirac
épsilon	<code>\epsilon</code> , <code>\varepsilon</code>	ϵ, ε	Cantidad pequeña
theta	<code>\theta</code> , <code>\Theta</code>	θ, Θ	Parámetros, ángulo
lambda	<code>\lambda</code> , <code>\Lambda</code>	λ, Λ	Tasa de Poisson, valor propio
mu	<code>\mu</code>	μ	Media
nu	<code>\nu</code>	ν	Grados de libertad
pi	<code>\pi</code> , <code>\Pi</code>	π, Π	3,14..., productoria
rho	<code>\rho</code>	ρ	Correlación
sigma	<code>\sigma</code> , <code>\Sigma</code>	σ, Σ	Desviación estándar, sumatoria
tau	<code>\tau</code>	τ	Constante de tiempo

Continúa en la página siguiente

Categoría	Código fuente $\LaTeX$	Renderizado	Notas
phi	<code>\phi, \Phi</code>	ϕ, Φ	CDF normal, ángulo
chi	<code>\chi</code>	χ	Prueba de chi-cuadrado
omega	<code>\omega, \Omega</code>	ω, Ω	Frecuencia, espacio muestral
Relaciones y lógica			
Distinto de	<code>\neq</code>	$\neq$	
Menor/mayor o igual	<code>\leq, \geq</code>	$\leq, \geq$	
Aproximadamente	<code>\approx</code>	$\approx$	
Proporcional a	<code>\propto</code>	$\propto$	
Distribuido como	<code>\sim</code>	$\sim$	
Para todo	<code>\forall</code>	$\forall$	
Existe	<code>\exists</code>	$\exists$	
Implica	<code>\Rightarrow</code>	$\Rightarrow$	
Si y sólo si	<code>\Leftrightarrow</code>	$\Leftrightarrow$	
Pertenece a	<code>\in</code>	$\in$	
Subconjunto	<code>\subset, \subseteq</code>	$\subset, \subseteq$	
Unión, intersección	<code>\cup, \cap</code>	$\cup, \cap$	
Conjuntos numéricos			
Números reales	<code>\mathbb{R}</code>	$\mathbb{R}$	Requiere <code>amsfonts</code> o <code>amssymb</code>
Números naturales	<code>\mathbb{N}</code>	$\mathbb{N}$	
Enteros	<code>\mathbb{Z}</code>	$\mathbb{Z}$	
Racionales	<code>\mathbb{Q}</code>	$\mathbb{Q}$	
Complejos	<code>\mathbb{C}</code>	$\mathbb{C}$	
Vectores y matrices			
Vector en negrita	<code>\mathbf{x}</code>	$\mathbf{x}$	Preferido en estadística
Vector con flecha	<code>\vec{x}</code>	$\vec{x}$	Convención física
Transpuesta	<code>A^{\top}</code> o <code>A^T</code>	A^T	
Matriz 2×2	<code>\begin{bmatrix} a & b \\ c & d \end{bmatrix}</code>	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$	Requiere <code>amsmath</code>
Norma	<code>\ x\ </code>	$\ x\ $	
Producto interno	<code>\langle x, y \rangle</code>	$\langle x, y \rangle$	

7.5. Figuras

Las figuras se incluyen utilizando el entorno flotante `figure` y el comando `\includegraphics` del paquete `graphicx`. El compilador `pdflatex` acepta los formatos de imagen PDF, PNG y JPG.

```

1 \begin{figure}[htbp]
2   \centering
3   \includegraphics[width=0.7\textwidth]{images/my-plot.png}
4   \caption{Description of the figure.}
5   \label{fig:my-plot}
6 \end{figure}

```

Listing 7.2: Inclusión de una figura en $\text{\LaTeX}$

La opción `[htbp]` sugiere preferencias de ubicación a $\text{\LaTeX}$: aquí (*here*), arriba (*top*), abajo (*bottom*) o en una página dedicada. Los comandos `\caption` y `\label` proporcionan una leyenda numerada y un objetivo de referencia cruzada, respectivamente.

7.6. Tablas

Las tablas se construyen con el flotante `table` y el entorno `tabular`. El paquete `booktabs` proporciona líneas horizontales limpias. Por convención, las líneas verticales (`|`) se omiten en los documentos profesionales; las líneas horizontales de `booktabs` son suficientes.

```

1 \begin{table}[htbp]
2   \centering
3   \caption{Sample measurements}
4   \label{tab:sample}
5   \begin{tabular}{lrr}
6     \toprule
7     Subject & Height (cm) & Weight (kg) \\
8     \midrule
9     Alice   & 165           & 60          \\
10    Bob     & 178           & 75          \\
11    Carol  & 172           & 68          \\
12    \bottomrule
13  \end{tabular}
14 \end{table}

```

Listing 7.3: Una tabla mínima con `booktabs`

7.7. Bibliografía

$\text{\LaTeX}$ gestiona las bibliografías mediante un archivo externo (`.bib`) y un comando de estilo. El comando `\cite{key}` inserta una cita en cualquier punto del

texto; los comandos `\bibliography{filename}` y `\bibliographystyle{plain}` generan la lista de referencias.

Para una gestión bibliográfica más sofisticada, el paquete `biblatex` con el motor `biber` es preferido en los documentos modernos. El preámbulo compartido de este repositorio tiene el soporte bibliográfico comentado; los estudiantes que escriban informes independientes deben descomentarlo y configurarlo.

7.8. LaTeX Workshop en Visual Studio Code

La extensión LaTeX Workshop para VSC (mencionada en la Sección 2.4) proporciona un flujo de trabajo integrado completo: resaltado de sintaxis, vista previa de PDF en vivo, compilación con un solo clic y navegación de errores. Una vez instalada, al abrir cualquier archivo `.tex` se activa la extensión automáticamente. El atajo de compilación es `Ctrl+Alt+B` (Windows/Linux) o `Cmd+Option+B` (macOS), que ejecuta `pdflatex` y muestra el resultado en el visor de PDF integrado. Los estudiantes no necesitan utilizar la línea de comandos para la compilación después de configurar esto; el enfoque por línea de comandos de la Sección 7.1 se presenta primero para que se comprenda el proceso subyacente.

7.9. Resumen

Este capítulo introdujo $\text{\LaTeX}$, el sistema de preparación de documentos utilizado a lo largo de este programa y ampliamente adoptado como el estándar en ciencia y matemáticas:

- **Qué es $\text{\LaTeX}$.** Un lenguaje de marcado de texto plano compilado en PDF compuestos tipográficamente, en contraste con los procesadores de texto WYSIWYG.
- **El flujo de trabajo desde la fuente hasta el PDF.** Escribir un archivo `.tex` en un editor de texto, compilar con `pdflatex` o `latexmk` y visualizar el PDF resultante.
- **Estructura del documento.** El preámbulo (clase de documento, paquetes, metadatos) y el cuerpo (contenido organizado con `\section`, `\subsection` y `\subsubsection`).
- **Matemáticas en línea y en presentación.** Signos de dólar para matemáticas en línea y `\[...\]` o el entorno `equation` para matemáticas en modo presentación.
- **Modos de ecuación.** Existen cinco modos: en línea (`\[...\]`), presentación heredada (`$$...$$`, obsoleto), presentación sin numerar preferida (`\[...\]`), numerado (`equation`) y alineado de varias líneas (`align`). Utilice `\[...\]` para ecuaciones sin numerar y `equation` para ecuaciones de presentación referenciables.

- **La hoja de referencia matemática.** Una tabla de referencia exhaustiva (Tabla 7.3) que abarca aritmética, funciones, sumatoria, integración, letras griegas, relaciones, conjuntos numéricos y vectores/matrices.
- **Figuras y tablas.** Entornos flotantes con `\includegraphics`, `tabular` y `booktabs` para un formato profesional.
- **Flujo de trabajo bibliográfico.** Un archivo `.bib` almacena las entradas de referencia. El comando `\cite{key}` inserta una cita; `\bibliography{filename}` y `\bibliographystyle{plain}` generan la lista de referencias. Los documentos con citas requieren una secuencia de compilación de cuatro pasos: dos pasadas de `pdflatex` que encierran una pasada de `bibtex`, seguida de una `pdflatex` final. `latexmk` se encarga de esto automáticamente.
- **LaTeX Workshop.** La extensión de VSC que proporciona resaltado de sintaxis, vista previa en vivo y compilación con un solo clic, eliminando la necesidad de compilación por línea de comandos en el uso diario.

Capítulo 8

Control de versiones con Git y GitHub

8.1. Configuración de la autenticación SSH para GitHub en Linux

GitHub ya no admite la autenticación por contraseña para operaciones de Git sobre HTTPS. En su lugar, la autenticación debe realizarse utilizando un Token de Acceso Personal o una clave SSH. El método recomendado para entornos Linux es la autenticación SSH.

El siguiente procedimiento genera una clave SSH y configura Git para autenticarse con GitHub.

8.1.1. Generar una clave SSH

Cree una nueva clave SSH utilizando el algoritmo `ed25519`:

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

Cuando se le solicite la ubicación del archivo, presione **Enter** para aceptar la predeterminada:

```
1 ~/.ssh/id_ed25519
```

Opcionalmente, proporcione una frase de contraseña para mayor seguridad.

8.1.2. Iniciar el agente SSH

Inicie el agente SSH y agregue la clave recién creada:

```
1 eval "$(ssh-agent -s)"  
2 ssh-add ~/.ssh/id_ed25519
```

8.1.3. Agregar la clave pública a GitHub

Muestre la clave pública:

```
1 cat ~/.ssh/id_ed25519.pub
```

Copie toda la salida y agréguela a GitHub:

1. Navegue a GitHub → Settings → SSH and GPG keys
2. Haga clic en New SSH key
3. Pegue la clave copiada y guarde

8.1.4. Verificar la conexión

Pruebe la autenticación:

```
1 ssh -T git@github.com
```

Una configuración exitosa mostrará un mensaje similar a:

```
1 Hi <username>! You've successfully authenticated.
```

8.1.5. Clonar un repositorio utilizando SSH

Los repositorios deben clonarse utilizando la URL de SSH:

```
1 git clone git@github.com:USER/REPOSITORY.git
```

El uso de SSH evita las solicitudes de credenciales y permite operaciones de Git automatizadas y seguras.

8.2. Configuración de la autenticación SSH para GitHub en Windows

Git admite dos protocolos para comunicarse con repositorios remotos: HTTPS y SSH. Si bien HTTPS es más sencillo de configurar inicialmente, la autenticación SSH ofrece un flujo de trabajo más fluido para los colaboradores frecuentes porque elimina las solicitudes repetidas de credenciales. Esta sección recorre el proceso completo de configuración de la autenticación basada en SSH para GitHub en una máquina con Windows utilizando Git Bash.

8.2.1. Abrir Git Bash

Git para Windows se distribuye con su propio cliente OpenSSH incluido y un intérprete de comandos tipo Unix llamado Git Bash. Todos los comandos SSH en esta sección deben ejecutarse dentro de Git Bash, no en el Símbolo del sistema ni en PowerShell, ya que los ejecutables `ssh`, `ssh-keygen` y `ssh-agent` incluidos

se encuentran en el directorio de instalación de Git para Windows y no están automáticamente disponibles en otras terminales.

Para abrir Git Bash, haga clic con el botón derecho en el Escritorio o en cualquier carpeta del Explorador de archivos y seleccione **Git Bash Here** en el menú contextual. Alternativamente, busque **Git Bash** en el menú Inicio de Windows y ejecútelo desde allí.

8.2.2. Generación de una clave SSH

La autenticación SSH se basa en un par de claves pública/privada. La clave privada permanece en su máquina; la clave pública se carga en GitHub. Genere un nuevo par de claves utilizando el algoritmo **ed25519**, que es moderno, compacto y se recomienda por encima del antiguo RSA predeterminado:

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

Listing 8.1: Generar un par de claves SSH ed25519

Reemplace `your_email@example.com` con la dirección asociada a su cuenta de GitHub. Cuando se le solicite una ubicación de archivo, presione **Enter** para aceptar la ruta predeterminada:

```
1 C:\Users\YourName\.ssh\id_ed25519
```

Listing 8.2: Ubicación predeterminada de la clave en Windows

A continuación, se le pedirá que introduzca una frase de contraseña opcional. Una frase de contraseña agrega una segunda capa de protección en caso de que su archivo de clave privada quede expuesto alguna vez; sin embargo, debe introducirse cada vez que se utilice la clave (a menos que configure el agente SSH para almacenarla en caché). Presione **Enter** dos veces para omitir la frase de contraseña si prefiere no establecer una.

8.2.3. Iniciar el agente SSH

El agente SSH es un proceso en segundo plano que mantiene las claves privadas descifradas en memoria, de modo que usted no tenga que volver a introducir una frase de contraseña para cada operación de Git. En Linux y macOS el agente normalmente se inicia de forma automática por la sesión de escritorio, pero en Windows el **ssh-agent** incluido con Git Bash debe iniciarse manualmente en cada nueva sesión de terminal.

Inicie el agente y agregue su clave privada con los siguientes dos comandos:

```
1 eval "$(ssh-agent -s)"
2 ssh-add ~/.ssh/id_ed25519
```

Listing 8.3: Iniciar el agente SSH y agregar la clave privada

El primer comando lanza el agente y establece las variables de entorno necesarias en el intérprete de comandos actual. El segundo comando registra su clave privada en el agente que se está ejecutando. Si estableció una frase de contraseña durante la generación de la clave, se le pedirá que la introduzca ahora.

Consejo

Automatización del inicio del agente Los comandos anteriores deben repetirse cada vez que abra una nueva sesión de Git Bash. Para evitarlo, puede agregar las siguientes líneas a su archivo `~/.bashrc` para que el agente se inicie automáticamente:

```
1 eval "$(ssh-agent -s)" > /dev/null 2>&1
2 ssh-add ~/.ssh/id_ed25519 2>/dev/null
```

Abra o cree `~/.bashrc` en un editor de texto, pegue las dos líneas al final del archivo y guarde. Las redirecciones suprimen los mensajes de inicio para que no saturen su terminal. La próxima vez que abra Git Bash, el agente se iniciará y cargará su clave sin pasos manuales.

8.2.4. Agregar la clave pública a GitHub

GitHub necesita su clave *pública* para verificar su identidad. Muestre su contenido con:

```
1 cat ~/.ssh/id_ed25519.pub
```

Listing 8.4: Mostrar la clave pública

La salida será una sola línea larga que comienza con `ssh-ed25519`. Seleccione y copie la línea completa, incluyendo la dirección de correo electrónico al final.

A continuación, navegue a GitHub en su navegador y siga estos pasos:

1. Haga clic en la foto de su perfil en la esquina superior derecha y seleccione **Settings**.
2. En la barra lateral izquierda, haga clic en **SSH and GPG keys**.
3. Haga clic en **New SSH key**.
4. Asigne a la clave un título descriptivo (por ejemplo, *Windows laptop*).
5. Pegue la clave pública copiada en el campo **Key**.
6. Haga clic en **Add SSH key** y confirme con su contraseña de GitHub si se le solicita.

8.2.5. Verificación de la conexión

Antes de utilizar SSH para cualquier operación en un repositorio, confirme que GitHub puede autenticar su clave:

```
1 ssh -T git@github.com
```

Listing 8.5: Probar la conexión SSH a GitHub

En la primera conexión, SSH mostrará una huella digital y le preguntará si confía en el servidor. Escriba **yes** y presione **Enter**. Si la autenticación es exitosa, verá un mensaje similar a:

```
1 Hi username! You've successfully authenticated, but GitHub
  does
2 not provide shell access.
```

Listing 8.6: Respuesta esperada de GitHub

Si en cambio recibe un error **Permission denied**, verifique que el agente se está ejecutando (**ssh-add -l** debería listar su clave) y que la clave pública correcta se ha agregado a su cuenta de GitHub.

8.2.6. Clonación de un repositorio utilizando SSH

Una vez confirmada la autenticación, clone los repositorios utilizando la URL remota SSH:

```
1 git clone git@github.com:USER/REPOSITORY.git
```

Listing 8.7: Clonar un repositorio por SSH

Reemplace **USER** por el nombre de usuario de GitHub del propietario del repositorio y **REPOSITORY** por el nombre del repositorio.

Advertencia

Utilice la URL remota SSH, no HTTPS. La URL remota SSH comienza con **git@github.com:** y termina con **.git**. No es la misma que la URL HTTPS, que comienza con **https://github.com/**. Si usted clona o hace push utilizando la forma HTTPS después de configurar la autenticación SSH, Git seguirá solicitándole un nombre de usuario y una contraseña de GitHub (o token de acceso personal). Verifique siempre la URL remota con **git remote -v** y actualícela si es necesario:

```
1 git remote set-url origin
  ↪ git@github.com:USER/REPOSITORY.git
```

8.2.7. Push a múltiples repositorios remotos

Un único repositorio local puede mantener conexiones simultáneas con cualquier número de repositorios remotos. Esto es útil cuando un proyecto debe replicarse entre una cuenta de organización y una cuenta personal (por ejemplo, mantener los materiales del taller de ELEVATE Academy sincronizados entre la organización ELEVATE Academy y la cuenta personal de GitHub **YourVeryOwnGitHubAccount**).

Cuadro 8.1: Comparación de la configuración SSH: Linux/macOS vs. Windows (Git Bash).

Paso	Linux / macOS	Windows (Git Bash)
Abrir terminal	Abra la aplicación de terminal del sistema (Terminal, iTerm2, etc.)	Haga clic con el botón derecho en el Escritorio o en una carpeta y seleccione <i>Git Bash Here</i> , o busque Git Bash en el menú Inicio
Generar clave	<code>ssh-keygen -t ed25519 -C "email"</code> (almacenada en <code>~/.ssh/</code>)	Mismo comando en Git Bash; la ruta predeterminada es <code>C:\Users\Name\.ssh\</code>
Iniciar agente	El agente normalmente se inicia de forma automática por la sesión de escritorio; ejecute <code>eval "\$(ssh-agent -s)"</code> si es necesario	Debe iniciarse manualmente en cada sesión: <code>eval "\$(ssh-agent -s)"</code>
Agregar clave al agente	<code>ssh-add ~/.ssh/id_ed25519</code>	Mismo comando en Git Bash
Mostrar la clave pública	<code>cat ~/.ssh/id_ed25519.pub</code>	Mismo comando en Git Bash
Verificar conexión	<code>ssh -T git@github.com</code>	Mismo comando en Git Bash
Clonar con SSH	<code>git clone git@github.com:USER/REPO.git</code>	Mismo comando en Git Bash

8.2.7.1. Conceptos

Git identifica los repositorios remotos mediante *nombres* cortos. El nombre *origin* es el predeterminado convencional que se asigna cuando un repositorio se clona por primera vez. Se pueden registrar remotos adicionales bajo cualquier nombre. Cada entrada remota almacena una URL de fetch y una URL de push, y ambas pueden establecerse de forma independiente.

Información

Un nombre de remoto es puramente local: solo existe en el archivo `.git/config` de su máquina y nunca se envía a ningún servidor. Dos desarrolladores que clonen el mismo repositorio pueden dar al mismo remoto nombres totalmente distintos sin consecuencias.

8.2.7.2. Listar los remotos registrados

Antes de agregar un remoto, confirme qué está ya registrado:

```
1 git remote -v
```

Listing 8.8: Listar todos los remotos y sus URL

Cada remoto aparece dos veces, una para fetch y otra para push. Un repositorio recién clonado normalmente muestra solo `origin`:

```
1 origin https://github.com/biomathematicus/ELEVATE.git (fetch)
2 origin https://github.com/biomathematicus/ELEVATE.git (push)
```

8.2.7.3. Agregar un segundo remoto

Registre un remoto adicional con `git remote add`:

```
1 git remote add YourVeryOwnGitHubAccount \
2 https://github.com/biomathematicus/ELEVATE.git
```

Listing 8.9: Registrar un remoto espejo personal (Linux / WSL)

```
1 git remote add YourVeryOwnGitHubAccount '
2 https://github.com/biomathematicus/ELEVATE.git
```

Listing 8.10: Registrar un remoto espejo personal (PowerShell)

Verifique el resultado:

```
1 git remote -v
```

La salida ahora debería mostrar cuatro líneas:

```
1 YourVeryOwnGitHubAccount
  https://github.com/YourVeryOwnGitHubAccount/ELEVATE
  Academy.git (fetch)
2 YourVeryOwnGitHubAccount
  https://github.com/YourVeryOwnGitHubAccount/ELEVATE
  Academy.git (push)
3 origin https://github.com/biomathematicus/ELEVATE.git
  (fetch)
4 origin https://github.com/biomathematicus/ELEVATE.git
  (push)
```

8.2.7.4. Push a cada remoto

Con dos remotos registrados, cada push debe emitirse explícitamente a cada destino. Git no hace push a múltiples remotos de forma automática a menos que se configure un destino de push:

```
1 git push origin HEAD
2 git push YourVeryOwnGitHubAccount HEAD
```

Listing 8.11: Push explícito a ambos remotos

`HEAD` se resuelve a la rama actual, lo que constituye la forma más segura de usar en scripts porque no depende de conocer el nombre de la rama por adelantado.

8.2.7.5. Comprobar si un remoto existe antes de hacer push

En scripts automatizados es buena práctica verificar que un remoto esté registrado antes de intentar hacer push, de modo que un remoto faltante produzca un diagnóstico claro en lugar de un error fatal.

```

1 if git remote get-url YourVeryOwnGitHubAccount > /dev/null
   ↪ 2>&1; then
2     git push YourVeryOwnGitHubAccount HEAD
3 else
4     echo "Remote 'YourVeryOwnGitHubAccount' not configured."
5     echo "Register it with:"
6     echo "    git remote add YourVeryOwnGitHubAccount \
7 https://github.com/YourVeryOwnGitHubAccount/ELEVATE Academy.git"
8 fi

```

Listing 8.12: Proteger un push con una verificación de existencia del remoto (bash)

```

1 git remote get-url YourVeryOwnGitHubAccount >nul 2>&1
2 if errorlevel 1 (
3     echo Remote 'YourVeryOwnGitHubAccount' not configured.
4     echo Register it with:
5     echo    git remote add YourVeryOwnGitHubAccount ^
6         https://github.com/YourVeryOwnGitHubAccount/ELEVATE
           Academy.git
7 ) else (
8     git push YourVeryOwnGitHubAccount HEAD
9 )

```

Listing 8.13: Proteger un push con una verificación de existencia del remoto (PowerShell / batch)

8.2.7.6. Gestión de remotos

La Tabla 8.2 resume el conjunto completo de comandos de gestión de remotos.

8.2.7.7. Alternativa: un único remoto con múltiples URL de push

Git también admite configurar un único nombre de remoto para hacer push a dos URL de forma simultánea, de modo que un único `git push origin HEAD` alcance ambos destinos. Esto es conveniente pero reduce la visibilidad; una falla en la segunda URL puede pasar fácilmente desapercibida en la salida.

```

1 # La primera URL de push ya esta establecida desde el clon.
2 # Agregue el repositorio personal como destino adicional de
   ↪ push:
3 git remote set-url --add --push origin \
4     https://github.com/biomathematicus/ELEVATE.git
5 git remote set-url --add --push origin \
6     https://github.com/YourVeryOwnGitHubAccount/ELEVATE
       Academy.git

```

Listing 8.14: Agregar una segunda URL de push al remoto origin

Cuadro 8.2: Comandos de gestión de remotos de Git

Comando	Efecto
<code>git remote -v</code>	Lista todos los remotos con sus URL de fetch y push.
<code>git remote add <n> <url></code>	Registra un nuevo remoto bajo el nombre indicado.
<code>git remote remove <n></code>	Elimina una entrada remota de la configuración local.
<code>git remote rename <old> <new></code>	Renombra un remoto sin cambiar su URL.
<code>git remote set-url <n> <url></code>	Actualiza la URL de un remoto existente.
<code>git remote get-url <n></code>	Imprime la URL de un remoto con nombre; termina con error si el remoto no existe (útil en scripts).
<code>git remote show <n></code>	Muestra información detallada sobre un remoto, incluidas las ramas rastreadas y la configuración de push.

Advertencia

Al utilizar `set-url -add -push`, tenga en cuenta que la *primera* llamada reemplaza la URL de push predeterminada en lugar de añadirla. Ambas URL deben especificarse explícitamente como se muestra arriba. Verifique el resultado con `git remote -v` antes de confiar en esta configuración en scripts automatizados.

Para el proyecto ALICE, se prefiere el enfoque explícito de dos remotos (llamadas a `git push` separadas por remoto) porque `gh.bat` verifica `errorlevel` después de cada push y puede reportar fallas de forma independiente. Una URL de push combinada expondría únicamente la segunda falla.

8.3. Resumen

Este capítulo presentó el control de versiones con Git y GitHub, centrándose en los pasos prácticos necesarios para autenticar, clonar y sincronizar repositorios.

- **Autenticación basada en claves SSH.** GitHub ya no admite la autenticación por contraseña para operaciones de Git. Este capítulo cubrió el procedimiento completo de configuración de SSH tanto en Linux como en Windows (utilizando Git Bash), incluida la generación de claves con el algoritmo `ed25519`, la adición de la clave pública a GitHub y la verificación de la conexión.
- **Clonación de repositorios mediante SSH.** Una vez configurada la autenticación, los repositorios se clonan utilizando la forma de URL `git@github.com:`, que evita las solicitudes de credenciales en cada push y pull.

- **Múltiples repositorios remotos.** Un único repositorio local puede hacer push a más de un remoto. Este capítulo demostró cómo registrar un segundo remoto y hacer push a ambos, manteniendo sincronizadas las copias de la organización y la personal.
- **Preparación, commit y push automatizados con gh.bat.** El script `gh.bat` automatiza el ciclo de add-commit-push, incluidos el sellado de versión y el push a ambos remotos, reduciendo los pasos manuales repetitivos a un único comando.

Capítulo 9

Infraestructura local de LLM con Ollama

9.1. Introducción

Para los estudiantes: este capítulo tiene dos partes que le son relevantes. La Sección 9.7 explica cómo configurar las claves de API necesarias para ejecutar los programas de agente en `resources/unit_7/F00/`. La referencia de la CLI de Ollama en la Sección 9.4 cubre los comandos que usará durante los ejercicios de la unidad de LLM. No necesita leer el resto de este capítulo para completar el desafío de datos.

Para instructores y personal del programa: este capítulo documenta la configuración completa de la infraestructura de servicio de LLM de dos máquinas utilizada por el programa ELEVATE Academy, cubriendo el host de aplicaciones Dell Precision 7770 y el servidor de inferencia NVIDIA Spark. Incluye la instalación de Ollama, la configuración del servicio `systemd`, el control de acceso a la red, la gestión del presupuesto de memoria, los modos de despliegue de una y dos máquinas, las pruebas de conectividad y la planificación de capacidad.

9.2. Inventario de hardware

Las dos máquinas que participan en esta configuración tienen arquitecturas de memoria fundamentalmente distintas, lo cual determina la selección de modelos y la capacidad de agentes.

9.2.1. Dell Precision 7770 (host de aplicaciones)

- **RAM de CPU:** 128 GB (112 GB disponibles después de la sobrecarga del sistema operativo)
- **GPU:** NVIDIA RTX A5500 Laptop GPU, 16 GB de VRAM (discreta)

- **Arquitectura:** x86_64
- **Sistema operativo:** Windows 11 con WSL2 Ubuntu 24.04
- **Rol:** Ejecuta una aplicación FastAPI, PostgreSQL 16 y la capa de orquestación de agentes. Aloja una instancia local de Ollama para el modo de desarrollo de máquina única.

En el Dell, la VRAM de la RTX A5500 (16 GB) está *separada* de la RAM del sistema. Ollama carga los pesos del modelo primero en la VRAM y sólo desborda a la RAM del sistema cuando el modelo excede la capacidad de la VRAM. La inferencia se degrada significativamente una vez que comienza el desbordamiento: los tokens por segundo pueden caer en un orden de magnitud en las porciones desbordadas del modelo.

9.2.2. NVIDIA Spark (servidor de LLM)

- **Memoria unificada:** 128 GB físicos; aproximadamente 120 GB visibles para Ollama (110 GB disponibles de forma conservadora)
- **GPU:** NVIDIA GB10 SoC
- **Arquitectura:** ARM64 (AArch64)
- **Sistema operativo:** Ubuntu 24.04 (nativo, sin virtualización)
- **Rol:** Servidor dedicado de inferencia de LLM. La CPU y la GPU comparten el mismo grupo de memoria, por lo que los 110 GB completos están disponibles simultáneamente para los pesos del modelo y la caché KV.

Advertencia

El GB10 utiliza una arquitectura de *memoria unificada*. A diferencia de una GPU discreta, no existe un presupuesto de VRAM separado que rastrear. El grupo de 110 GB cubre los pesos, la caché KV y la sobrecarga del sistema operativo en conjunto. Esto hace que el Spark sea idóneo para servir múltiples modelos grandes simultáneamente (siempre que las ventanas de contexto estén acotadas), pero también significa que PagedAttention de vLLM, que está optimizado para GPU discretas con alta VRAM, proporciona menos beneficios aquí que el manejo nativo de memoria unificada de Ollama.

9.3. Topología de red

Ambas máquinas residen en la subred `utsarr.net` de UTSA. Al momento de la configuración tenían las siguientes direcciones:

Máquina	Interfaz	Dirección IP	Subred
Spark	enP7s7	10.2.14.24	/23
Dell	Ethernet 8	10.2.14.143	/23
Spark	MAC de enP7s7	4c:bb:47:2c:74:eb	
Dell	MAC de Ethernet 8	ac:91:a1:7d:06:ff	

9.3.1. Conmutador frente a enrutador

Un conmutador no administrado de 5 puertos Netgear ProSAFE GS105 conecta las dos máquinas. Un *conmutador no administrado* opera en la Capa 2 del modelo OSI: reenvía tramas Ethernet entre puertos basándose en direcciones MAC, pero no tiene concepto de subredes IP, enrutamiento o restricción de tráfico. Un *enrutador* opera en la Capa 3 y es necesario para separar subredes o aplicar políticas de tráfico a nivel de IP.

Debido a que ambas máquinas están en la misma subred /23 administrada por la red universitaria, las entradas ARP no son visibles entre los hosts; el tráfico cruza al menos un salto de enrutador, y ARP no atraviesa enrutadores. Este es el comportamiento esperado en una red enrutada y no debe interpretarse como un error de conectividad.

9.3.2. Filtrado por dirección MAC

Las reglas de firewall basadas en MAC (mediante `iptables`) sólo tienen sentido en el mismo segmento de Capa 2. Pueden ser suplantadas trivialmente y deben tratarse como una capa informativa más que como un límite de seguridad. Las reglas de firewall IP (`ufw`) son el control primario apropiado para restringir el acceso a Ollama a direcciones de cliente conocidas.

```

1 # En el Spark: permitir sólo al Dell, denegar todas las
   ↪ demás fuentes
2 sudo ufw allow from 10.2.14.143 to any port 11434
3 sudo ufw deny 11434

```

Listing 9.1: Restringir Ollama solamente a la IP del Dell

9.3.3. Identificación de interfaces de red

Use los siguientes comandos para enumerar las interfaces y confirmar las direcciones antes de configurar las reglas de firewall.

```

1 ip a
2 # Busque interfaces con estado UP y una dirección inet
   ↪ válida.
3 # La línea link/ether de cada interfaz muestra la dirección
   ↪ MAC.

```

Listing 9.2: Linux: listar interfaces y direcciones

```
1 ipconfig /all
2 # Busque adaptadores Ethernet activos con una IPv4 Address.
3 # El campo Physical Address muestra la MAC (formato separado
   ↪ por guiones).
```

Listing 9.3: Windows: listar interfaces y direcciones

Información

Formatos de dirección MAC. Las herramientas de Linux (`ip a`, `arp`) reportan las MAC en formato de minúsculas con dos puntos: `ac:91:a1:7d:06:ff`. El `ipconfig /all` de Windows las reporta en formato de mayúsculas con guiones: `AC-91-A1-7D-06-FF`. Ambos representan la misma dirección. Los archivos de configuración y los scripts deben normalizarse a un único formato antes de la comparación.

9.4. Instalación de Ollama

9.4.1. Linux (Spark)

El script de instalación oficial detecta la arquitectura automáticamente y selecciona el binario apropiado (ARM64 en este caso).

```
1 curl -fsSL https://ollama.com/install.sh | sh
```

Listing 9.4: Instalar Ollama en Ubuntu

El instalador crea el usuario y el grupo `ollama`, agrega al usuario actual al grupo `ollama`, instala el binario en `/usr/local/bin/ollama` y registra un servicio `systemd`. Confirme la instalación:

```
1 ollama --version
2 systemctl status ollama
```

9.4.2. Windows (Dell)

Descargue el instalador de Windows desde <https://ollama.com> y ejecútelo. Ollama se registra en la bandeja del sistema y se inicia automáticamente. La API escucha en `127.0.0.1:11434` de forma predeterminada.

9.4.3. Entornos Conda

El Spark viene con Miniconda, que activa un entorno (`base`) de forma predeterminada. Ollama no requiere Python y debe instalarse a nivel de sistema, no dentro de un entorno Conda. Desactive la activación automática de Conda para evitar interferencias con las herramientas del sistema:

```
1 conda config --set auto_activate_base false
```

Para todo el trabajo de Python en esta pila, use entornos `venv` simples, que se traducen directamente a rutas `ExecStart` de `systemd` y mantienen la paridad de producción sin la sobrecarga de dependencias de Conda.

9.5. Configuración de Ollama para acceso de red

De forma predeterminada, Ollama se enlaza únicamente a `localhost`. Para la configuración de dos máquinas, el Spark debe exponer su API a la red. Configure esto mediante una sobrescritura de `systemd` para que el ajuste sobreviva a los reinicios del servicio y del sistema.

```
1 sudo systemctl edit ollama
```

Listing 9.5: Crear una sobrescritura de `systemd` en el Spark

Esto abre `/etc/systemd/system/ollama.service.d/override.conf`. Agregue lo siguiente:

```
1 [Service]
2 Environment="OLLAMA_HOST=0.0.0.0:11434"
3 Environment="OLLAMA_MAX_LOADED_MODELS=4"
4 Environment="OLLAMA_NUM_PARALLEL=4"
5 Environment="OLLAMA_KEEP_ALIVE=24h"
6 Environment="OLLAMA_NUM_CTX=8192"
```

Listing 9.6: Sobrescritura de `systemd` para Ollama en el Spark

Aplique los cambios:

```
1 sudo systemctl daemon-reload
2 sudo systemctl restart ollama
3 systemctl status ollama
```

Las variables de entorno se explican en la Tabla 9.1.

9.6. Gestión del presupuesto de memoria y de la ventana de contexto

9.6.1. La fórmula rectora

La memoria total consumida por agente tiene dos componentes independientes:

$$M_{\text{agent}} = M_{\text{weights}} + M_{\text{KV}} \quad (9.1)$$

donde la huella de los pesos bajo cuantización Q4 se aproxima por:

$$M_{\text{weights}} \approx 0.5 \text{ GB} \times B_{\text{params}} \quad (9.2)$$

Cuadro 9.1: Variables de entorno clave de Ollama

Variable	Efecto
OLLAMA_HOST	Dirección y puerto de enlace. Use 0.0.0.0:11434 para aceptar conexiones desde cualquier interfaz de red.
OLLAMA_MAX_LOADED_MODELS	Número máximo de modelos residentes en memoria simultáneamente. Evita el thrashing cuando MAX despacha llamadas de agente concurrentes.
OLLAMA_NUM_PARALLEL	Número máximo de solicitudes de inferencia concurrentes por modelo.
OLLAMA_KEEP_ALIVE	Duración durante la cual un modelo permanece cargado después de su última solicitud. El valor predeterminado es 5 minutos; 24h mantiene los modelos calientes a lo largo de una jornada laboral.
OLLAMA_NUM_CTX	Ventana de contexto predeterminada en tokens. Crítico en el Spark: sin este límite, Ollama usa por defecto 262,144 tokens, lo que asigna una caché KV lo suficientemente grande como para exceder la memoria física en la mayoría de los modelos. Establezca en 8192 como valor predeterminado seguro.

y la huella de la caché KV es:

$$M_{KV} \approx \frac{n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times C \times 2 \times 2}{10^9} \text{ GB} \quad (9.3)$$

donde C es la longitud del contexto en tokens y el factor 2 aparece dos veces: una por el par clave/valor y otra por el tipo de dato de 2 bytes (BF16).

Advertencia

La ventana de contexto domina la memoria en modelos grandes.

Un modelo llama3.3:70b requiere aproximadamente 40 GB para los pesos en Q4. Con el contexto predeterminado de Ollama de 262,144 tokens, la caché KV agrega otros 134 GB, llevando la asignación total a 174 GB, lo que excede los 120 GB del Spark y hace que la carga falle. Con un contexto de 8,192 tokens, la caché KV se reduce a 2.5 GB, dando un total de 42.5 GB que encaja cómodamente.

9.6.2. El patrón Modelfile para modelos grandes

Ollama no admite sobrescrituras de contexto en tiempo de ejecución mediante la bandera `-num-ctx`. El mecanismo correcto es un *Modelfile*, que crea un derivado nombrado del modelo base con el parámetro incorporado.

9.6.2.1. Linux (Spark y WSL)

```

1 cat > /tmp/Modelfile.llama70b << 'EOF'
2 FROM llama3.3:70b
3 PARAMETER num_ctx 8192
4 EOF
5 ollama create llama3.3:70b-ctx8k -f /tmp/Modelfile.llama70b
6 ollama run llama3.3:70b-ctx8k "Reply with one word: ready"

```

Listing 9.7: Crear una variante de modelo con contexto acotado en Linux

9.6.2.2. Windows (PowerShell)

PowerShell usa una sintaxis de here-string para literales multilínea. El cierre "@ debe aparecer en el *inicio absoluto de una línea* sin espacios ni tabuladores al comienzo; este es un requisito estricto del analizador de PowerShell.

```

1 @"
2 FROM llama3.3:70b
3 PARAMETER num_ctx 8192
4 "@ | Out-File -FilePath "$env:TEMP\Modelfile.llama70b"
   ↳ -Encoding utf8
5 ollama create llama3.3:70b-ctx8k -f
   ↳ "$env:TEMP\Modelfile.llama70b"

```

Listing 9.8: Crear una variante de modelo con contexto acotado en Windows

Aplique el mismo patrón a cada modelo por encima de los 20 B parámetros. La convención de nombres <model>-ctx8k hace explícita la restricción operativa en el sitio de la llamada.

9.6.3. Capacidad de agentes del hardware

Dada la fórmula del presupuesto de memoria, el número máximo de agentes que pueden ejecutarse simultáneamente en una máquina es:

$$N_{hw} = \left\lfloor \frac{M_{available}}{M_{agent}} \right\rfloor \quad (9.4)$$

La Tabla 9.2 presenta el inventario de modelos confirmado con el estado medido en cada plataforma y la huella de memoria aproximada con un contexto de 8,192 tokens.

9.6.4. Modelos con razonamiento y presupuestos de tokens

Varios modelos del inventario implementan razonamiento de cadena de pensamiento internamente, emitiendo un *bloque de pensamiento* antes de su respuesta final. Estos incluyen qwen3 (todos los tamaños), deepseek-r1 y nemotron-3-super. Al llamar a estos modelos a través del endpoint compatible con OpenAI con un presupuesto pequeño de max_tokens, el presupuesto puede agotarse dentro del

Cuadro 9.2: Inventario de modelos confirmado y huella de memoria con contexto de 8K

Modelo	Nombre en Ollama	Parám. (B)	M_w (GB)	M_{KV} (GB)	M_{total} (GB)	Estado
nemotron-3-super:120b	...-ctx8k	120 (MoE)	65	4.0	69.0	✓ ambas
llama3.3:70b	...-ctx8k	70	40	2.5	42.5	✓ ambas
qwen3:32b	...-ctx8k	32	18	1.3	19.3	✓ ambas
mistral-small:22b	mistral-small:22b	22	13	1.0	14.0	✓ ambas
phi4:14b	phi4:14b	14	8	0.8	8.8	✓ ambas
qwen3:14b	qwen3:14b	14	8	0.8	8.8	✓ ambas
mistral-nemo:12b	mistral-nemo:12b	12	7	0.7	7.7	✓ ambas
mistral:7b	mistral:7b	7	4	0.5	4.5	✓ ambas
deepseek-r1:8b	deepseek-r1:8b	8	5	0.5	5.5	✓ ambas
gemma3:4b	gemma3:4b	4	2.5	0.3	2.8	✓ ambas
phi4-mini:3.8b	phi4-mini:3.8b	3.8	2.5	0.3	2.8	✓ ambas
qwen3:4b	qwen3:4b	4	2.5	0.3	2.8	✓ Dell; † Spark
gemma3:1b	gemma3:1b	1	0.8	0.1	0.9	✓ ambas
deepseek-v3.1:671b	—	671 (MoE)	404	8.0	412	× demasiado grande

† Falla ARM64 en Spark (error de Ollama); funciona en Dell x86_64.

bloque de pensamiento, produciendo ya sea un error HTTP 500 o una respuesta vacía.

La mitigación correcta *no* es suprimir el pensamiento mediante la indicación de sistema `/no_think`, lo cual puede dejar al modelo sin nada que decir fuera del bloque. En su lugar, establezca `max_tokens` de forma generosa (500 o más) y elimine las etiquetas de pensamiento de la respuesta:

```

1 # Estilo DeepSeek-R1:
2 <think> ... </think>
3
4 # Estilo nativo de Qwen3 / Ollama:
5 Thinking...
6 ... reasoning text ...
7 ...done thinking.
```

Listing 9.9: Patrones de etiquetas de pensamiento a eliminar de la salida del modelo

9.7. Configuración de claves de API

Recibirá una clave de API para OpenAI y Anthropic durante nuestra primera sesión. Puede obtener su propia API de forma gratuita en Groq. Necesita crear tres variables de clave de API siguiendo el procedimiento descrito a continuación: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GROQ_API_KEY`. Pruebe los programas ubicados en `resources/unit_7/F00/` del repositorio ELEVATE Academy: `agentGroq.py`, `agentGPT.py` y `agentClaude.py`.

En Windows, ejecute los siguientes tres comandos, uno a la vez, para cada clave de API. El ejemplo para `OPENAI_API_KEY` es el siguiente:

```
setx OPENAI_API_KEY "[sk-... API key]"
```

```
Exit
echo %OPENAI_API_KEY%
```

En el código anterior, reemplace `[sk-... API key]` con su clave de API real. El primer comando establece la variable de entorno de forma permanente para su cuenta de usuario. El segundo comando cierra el Símbolo del sistema para asegurar que la nueva variable de entorno quede registrada. El tercer comando, ejecutado en una nueva ventana del Símbolo del sistema, imprime el valor almacenado para verificar que se haya configurado correctamente.

En Mac/Linux ejecute los siguientes tres comandos, uno a la vez, para cada clave de API. El ejemplo para `OPENAI_API_KEY` es el siguiente:

```
echo "export OPENAI_API_KEY='[sk-... API key]'" >> ~/.zshrc
source ~/.zshrc
echo $OPENAI_API_KEY
```

En el código anterior, reemplace `'[sk-... API key]'` con su clave de API real.

Según lo reportado por Bryan Fowler, si experimenta errores al intentar agregar sus variables de entorno en Mac/Linux, lo más probable es que el problema sea la propiedad del archivo `~/.zshrc`. En este caso, ejecute

```
sudo chown $(whoami) ~/.zshrc
```

E intente crear las variables de entorno nuevamente.

Según lo reportado por Drew Stephen con respecto a Mac, *«aparentemente no funciona en el shell c predeterminado, pero cambiar a zsh permite ejecutarlo»*. Es posible que deba cerrar y volver a abrir la ventana de la terminal desde donde comenzó para que los cambios surtan efecto.

9.8. El marco FOO y la viabilidad de agentes

9.8.1. Número mínimo de agentes

El Sistema Multi-Agente de Consenso MAX implementa el algoritmo FOO para el consenso tolerante a fallas bizantinas. Dado que cualquier agente individual tiene una probabilidad de invalidación p , el número mínimo de agentes requeridos para alcanzar una confianza de consenso C es:

$$N_{\text{foo}}(p, C) = \left\lceil \frac{\log(1 - C)}{\log(p)} \right\rceil \quad (9.5)$$

Valores seleccionados se presentan en la Tabla 9.3.

Cuadro 9.3: Número mínimo de agentes N_{foo} según la tasa de invalidación y el objetivo de confianza

Tasa de invalidación por agente p	Confianza C	N_{foo}
0.30	0.95	4
0.30	0.99	6
0.20	0.95	3
0.20	0.99	5
0.10	0.99	3
0.05	0.99	2

9.8.2. Puntaje de viabilidad de agentes

El *Puntaje de Viabilidad de Agentes* (AFS, por sus siglas en inglés) acopla el presupuesto de hardware con la condición de validez de FOO:

$$\text{AFS} = \frac{N_{\text{hw}}}{N_{\text{foo}}} \quad (9.6)$$

Una configuración es *viable* si y sólo si $\text{AFS} \geq 1.0$. Cuando $\text{AFS} < 1.0$, se debe aplicar una o más de las siguientes mitigaciones: reducir la ventana de contexto (aumenta N_{hw}), usar un modelo más pequeño (aumenta N_{hw}), o relajar el objetivo de confianza C (reduce N_{foo}).

9.8.3. Configuraciones de referencia

Configuración VRAM-óptima en el Dell (rápida, sin desbordamiento a RAM). Cinco agentes que caben enteramente dentro de 16 GB de VRAM, dando $\text{AFS} = 8.00$ con $p = 0.20$, $C = 0.99$:

Modelo	M_{agent} (GB)	Ubicación
gemma3:1b	0.9	VRAM
phi4-mini:3.8b	2.8	VRAM
gemma3:4b	2.8	VRAM
qwen3:4b	2.8	VRAM
mistral:7b	4.5	VRAM
Total	13.8	(de 16 GB)

Configuración de capacidad completa del Spark. Configuración mixta de niveles que usa 110 GB de memoria unificada:

Rol	Modelo	M_{agent} (GB)	AFS
Orquestador	qwen3:32b-ctx8k	19.3	1.67
Analista primario	llama3.3:70b-ctx8k	42.5	0.67
Crítico A	mistral-small:22b	14.0	2.33
Crítico B	phi4:14b	8.8	4.00
Sintetizador	mistral-nemo:12b	7.7	4.67
Total		92.3	0.67

El AFS mínimo de 0.67 (sólo por llama3.3:70b en el grupo) indica que una ronda completa de cinco agentes no puede ejecutarse simultáneamente en el Spark con esta selección de modelos. La solución operativa es ejecutar los modelos T1 secuencialmente dentro de una ronda y almacenar en caché sus salidas, o reemplazar llama3.3:70b-ctx8k por qwen3:32b-ctx8k para liberar memoria para un quinto agente simultáneo.

9.9. Configuración de máquina única (Dell)

En el modo de máquina única, el Dell ejecuta tanto la pila de aplicaciones ALICE como la capa de inferencia de Ollama. Esta configuración es apropiada para el desarrollo de indicaciones y las pruebas de agentes a nivel de unidad, donde el aislamiento de red es deseable.

9.9.1. Variable de entorno

```

1 LLM_PROVIDER=ollama
2 LLM_BASE_URL=http://127.0.0.1:11434/v1
3 LLM_MODEL=mistral:7b

```

Listing 9.10: .env para el modo de máquina única

Desde WSL, la instancia de Ollama de Windows es accesible a través de la puerta de enlace predeterminada de WSL en lugar de localhost:

```

1 ip route | grep default | awk '{print $3}'
2 # Resultado típico: 172.24.0.1 (varía según la versión y
   ↪ configuración de WSL)

```

Listing 9.11: Encontrar la IP de la puerta de enlace de Windows desde WSL

```

1 LLM_PROVIDER=ollama
2 LLM_BASE_URL=http://172.24.0.1:11434/v1
3 LLM_MODEL=mistral:7b

```

Listing 9.12: .env para WSL accediendo a Ollama de Windows

9.9.2. Selección de modelos para el desarrollo de máquina única

Los 16 GB de VRAM del Dell restringen la capacidad simultánea de modelos. Para el trabajo de desarrollo enfocado al marco FOO, la siguiente asignación por niveles equilibra la capacidad con la residencia en VRAM:

- **Agente primario:** `deepseek-r1:8b` (5.5 GB), modelo de razonamiento con modo de pensamiento; buen sustituto para agentes de consenso más grandes.
- **Crítico rápido:** `mistral-nemo:12b` (7.7 GB), con capacidad de uso de herramientas, rápido, entrenado por NVIDIA.
- **Agente utilitario:** `gemma3:1b` (0.9 GB), respuestas en menos de un segundo para tareas de salida estructurada.

VRAM total: 14.1 GB de 16 GB. Esto deja 1.9 GB de margen para el servidor gráfico y los procesos en segundo plano.

9.10. Configuración de dos máquinas

En la configuración de dos máquinas, el Dell aloja la pila de aplicaciones y el Spark atiende toda la inferencia de LLM. El grupo de memoria unificada de 110 GB del Spark permite que múltiples modelos grandes permanezcan residentes simultáneamente, lo cual es el prerequisite de hardware para ejecutar una ronda completa de consenso FOO sin desalojo secuencial.

9.10.1. Arquitectura

+-----+ HTTP/REST +-----+		
Dell 7770	<-----	NVIDIA Spark
	port 11434	
Alice FastAPI		Ollama (port 11434)
PostgreSQL 16		systemd: ollama.service
Agents		OLLAMA_HOST=0.0.0.0
Python venv: ALICE		OLLAMA_NUM_CTX=8192
.env:		Models resident:
LLM_BASE_URL=		qwen3:32b-ctx8k
http://10.2.14.24:		llama3.3:70b-ctx8k
11434/v1		mistral-nemo:12b
+-----+		+-----+

9.10.2. Variables de entorno

```
1 # Modo de dos máquinas: el Spark atiende toda la inferencia
2 LLM_PROVIDER=ollama
3 LLM_BASE_URL=http://10.2.14.24:11434/v1
4 LLM_API_KEY=not-required
5
6 # Asignación de modelo por agente (política de enrutamiento
  ↪ de Max)
7 AGENT_MODEL_ORCHESTRATOR=qwen3:32b-ctx8k
8 AGENT_MODEL_ANALYST=llama3.3:70b-ctx8k
9 AGENT_MODEL_CRITIC=mistral-nemo:12b
10 AGENT_MODEL_FAST=mistral:7b
```

Listing 9.13: .env para el modo de dos máquinas (host de aplicaciones Dell)

El protocolo `ProviderAdapter` en MAX hace que cambiar entre los modos de máquina única y de dos máquinas sea una modificación de una sola línea en `LLM_BASE_URL`. No se requieren cambios en el código de la aplicación.

9.10.3. Verificación de conectividad

Un script de prueba de conectividad (`connectivity_test.py`) valida la ruta de extremo a extremo desde el host de aplicaciones hasta el servidor Ollama. Prueba cuatro condiciones en secuencia, deteniéndose en la primera falla:

1. **Negociación TCP:** confirma que el host es accesible y que el puerto está abierto.
2. **Verificación de MAC por ARP:** sólo informativa; confirma si las máquinas están en el mismo segmento de Capa 2. Una falla aquí es esperada en una red universitaria enrutada y no indica un problema.
3. **Lista de modelos:** confirma que Ollama está atendiendo y devuelve la lista de modelos instalados.
4. **Inferencia:** envía una solicitud mínima de completación de chat y valida la respuesta.

Los perfiles de máquina, los puertos y los parámetros de prueba se almacenan en `connectivity_config.json`, que está excluido del control de versiones (`.gitignore`) porque contiene direcciones IP y MAC específicas del entorno. Un `connectivity_config.json.template` con valores de marcador de posición se incluye junto con el script.

9.11. Pruebas de capacidad

Un script de prueba de capacidad (`capacity_test.py`) lee el registro de modelos de

`connectivity_config.json`, calcula los valores teóricos de N_{hw} y AFS para cada modelo, y valida empíricamente cada modelo instalado ejecutando una única llamada de inferencia.

9.11.1. Consideraciones de plataforma

- **ARM64 (Spark):** `qwen3:4b` falla con un error de `exit status 2` en el binario ARM64 de Ollama. Este es un error de Ollama que afecta a este modelo y cuantización específicos en ARM64. El mismo modelo funciona correctamente en el Dell x86_64. El script detecta la arquitectura en tiempo de ejecución y omite las fallas específicas de ARM64 de forma apropiada.
- **Variantes de Modelfile:** Las variantes con contexto acotado (por ejemplo, `llama3.3:70b-ctx8k`) deben crearse por separado en cada máquina. Un modelo creado mediante Modelfile en el Spark no está disponible automáticamente en el Dell. El script detecta los casos de *base instalada pero variante ausente* y emite los comandos de shell exactos para crear la variante, usando sintaxis de PowerShell en Windows y sintaxis de heredoc de bash en Linux.
- **VRAM frente a RAM:** En el Dell, el script distingue los modelos que caben enteramente en VRAM (inferencia rápida) de aquellos que se desbordan a la RAM del sistema (significativamente más lenta). Se producen dos recomendaciones de configuración: una configuración VRAM-óptima (la más rápida, que puede no alcanzar N_{foo}) y una configuración extendida a RAM (alcanza N_{foo} si la RAM lo permite, pero con una advertencia de rendimiento para los modelos desbordados).

9.11.2. Referencia de la CLI de Ollama

```
1 # Listar modelos instalados
2 ollama list
3
4 # Mostrar modelos actualmente cargados y uso de memoria
5 ollama ps
6
7 # Descargar un modelo desde el registro
8 ollama pull qwen3:32b
9
10 # Sesión de chat interactiva
11 ollama run mistral:7b
12
13 # Indicación de una sola ejecución (sin sesión interactiva)
14 ollama run mistral:7b "Explain the rhizome model in one
    ↪ paragraph"
15
16 # Eliminar un modelo
```



```
17 ollama rm mistral:7b
18
19 # Probar directamente la API compatible con OpenAI
20 curl http://localhost:11434/v1/models
21 curl http://localhost:11434/v1/chat/completions \
22     -H "Content-Type: application/json" \
23     -d '{"model":"mistral:7b",
24         "messages":[{"role":"user","content":"Hello"}]}'
```

Listing 9.14: Comandos esenciales de la CLI de Ollama

9.12. Paridad de despliegue

Un principio central de esta infraestructura es que el entorno de desarrollo refleja la producción sin contenedorización. Ambas máquinas ejecutan Ubuntu nativo (o WSL2 para el host de Windows), Ollama es administrado por `systemd`, y la configuración se inyecta a través de archivos de entorno en lugar de cambios en el código.

El modelo de configuración de dos niveles es:

- **Desarrollo:** archivo `.env` en la raíz del proyecto, cargado por `python-dotenv` con `override=False`.
- **Producción:** directiva `EnvironmentFile=` de `systemd` que apunta a un archivo fuera del repositorio. No se requieren cambios en el código al momento del despliegue.

Cambiar del modo de máquina única al de dos máquinas, o del Ollama local del Dell al Spark, sólo requiere actualizar `LLM_BASE_URL` en el archivo de entorno. El protocolo `ProviderAdapter` enruta de forma transparente las solicitudes al endpoint que esté configurado.

9.13. Configuración de Ollama con una GUI en Ubuntu

Para hacer la transición desde una interfaz de línea de comandos a una experiencia gráfica, siga estos pasos para instalar el backend de Ollama y la interfaz Open WebUI.

9.13.1. Paso 1: Instalar el backend de Ollama

Primero, instale el servicio central de Ollama usando el script de instalación oficial:

```
curl -fsSL https://ollama.com | sh
```

9.13.2. Paso 2: Desplegar Open WebUI mediante Docker

Open WebUI es la interfaz estilo ChatGPT más popular para Ollama. Ejecute el siguiente comando de Docker para descargar la imagen e iniciar el contenedor:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

9.13.3. Paso 3: Acceder a la interfaz

Una vez que el contenedor esté en ejecución, abra su navegador web preferido y navegue a:

`http://localhost:3000`

Se le pedirá crear una cuenta de administrador (almacenada localmente) para comenzar a chatear con sus modelos.

9.13.4. Consideraciones técnicas

- **Soporte de GPU:** Para usuarios de NVIDIA, asegúrese de que `nvidia-container-toolkit` esté instalado para habilitar la aceleración por hardware dentro de Docker.
- **Acceso externo:** Para acceder a esta GUI desde otra máquina, configure la variable de entorno `OLLAMA_HOST=0.0.0.0` en la configuración de su servicio `systemd`.

Capítulo 10

Evaluación y Registro de Progreso

Este capítulo es un formulario PDF rellenable dirigido al profesorado y al personal que se incorpora como colaborador del repositorio de currículo del Departamento de Matemáticas. Cada sección corresponde a una categoría de preparación: Obligatorio, Esencial, Deseable y Opcional. Las casillas de verificación registran la finalización binaria de los pasos de verificación; los campos de texto registran respuestas escritas breves que demuestran comprensión conceptual. Utilice este capítulo como una lista de verificación activa durante su incorporación.

Guardar sus respuestas requiere Adobe Acrobat Reader (gratuito para Windows, macOS y Linux; disponible en <https://get.adobe.com/reader/>). Otros visores se comportan de la siguiente manera:

- **Vista Previa de macOS:** muestra los campos del formulario y permite escribir en ellos, pero *no guarda los datos ingresados*. Si cierra y vuelve a abrir el archivo en Vista Previa, sus respuestas se perderán.
- **Google Chrome (visor de PDF integrado):** muestra y acepta entradas en los campos del formulario, pero *no guarda los datos*. Imprimir a PDF desde Chrome tampoco preservará el contenido de los campos.
- **La mayoría de los visores de PDF de Linux** (Evince, Okular, Zathura): *ignoran por completo los campos del formulario*. Los campos no aparecerán interactivos.
- **Adobe Acrobat Reader:** guarda, imprime y exporta completamente los datos del formulario. Use **Archivo > Guardar** (no Guardar como) después de completar cada sección.

Cómo interpretar las cuatro categorías:

Obligatorio Tareas básicas del repositorio. Estas deben completarse antes de que un colaborador pueda realizar cualquier cambio en el documento de currículo.

Esencial Configuración del entorno local: L^AT_EX, Python, Git y Visual Studio Code. Estos deben funcionar correctamente antes de comenzar cualquier trabajo de autoría o de programación.

Deseable Comprensión conceptual de las herramientas introducidas en los capítulos de infraestructura. Completar estos elementos demuestra estar preparado para trabajar de manera independiente en el repositorio.

Opcional Infraestructura local de modelos grandes de lenguaje (Ollama). No es obligatorio para la autoría del currículo; es relevante para los colaboradores que deseen usar modelos de lenguaje locales en su flujo de trabajo.

10.1. Obligatorio: Incorporación al Repositorio

Los siguientes elementos verifican que usted puede recuperar, compilar y contribuir al repositorio de currículo. Todos los elementos deben completarse antes de editar cualquier archivo fuente.

R1. El `git clone` del repositorio se completa sin errores y la copia de trabajo local contiene la carpeta `curriculum/`. (Véase el Capítulo 8, Sección 8.2.7.)

R2. Ejecutar `pdflatex MathCurriculum.tex` desde dentro de la carpeta `curriculum/` produce `MathCurriculum.pdf` sin errores fatales. (Véase el Capítulo 7, Sección 7.1.)

R3. Ejecutar `gh` (o `gh.bat` en Windows) con un mensaje de confirmación prepara, confirma y envía exitosamente un cambio al repositorio, y `VERSION.txt` se actualiza con el nuevo hash. (Véase el Capítulo 8.)

R4. El hash de confirmación en `VERSION.txt` coincide con el valor devuelto por `git rev-parse -short HEAD` en su repositorio local. (Véase el Capítulo 8.)

R5. Describa la estructura de dos partes del documento de currículo. ¿Qué está contenido en los capítulos de infraestructura y qué está contenido en los capítulos de unidades del curso? ¿Cómo se identifican los capítulos de unidades en el árbol de código fuente? (Véase la Introducción.)

10.2. Esencial: Configuración del Entorno y Cadena de Herramientas

Marque cada casilla cuando el paso de verificación correspondiente tenga éxito. Todos los elementos de esta sección deben estar marcados antes de comenzar cualquier trabajo de autoría o de programación en el repositorio.

E1. `pdflatex -version` devuelve un número de versión en la terminal. (Véase el Capítulo 2.)

E2. `python -version` devuelve un número de versión en la terminal. (Véase el Capítulo 2, Sección 2.1.)

E3. `git -version` devuelve un número de versión. (Véase el Capítulo 2, Sección 2.6.)

E4. `code .` abre Visual Studio Code desde una ventana de terminal. (Véase el Capítulo 2, Sección 2.8.)

E5. El entorno virtual de Python se activa sin errores, y los paquetes listados en `pyproject.toml` (o `requirements.txt`) se importan correctamente en ese entorno. (Véase el Capítulo 5, Sección 5.1.4.)

E6. Ejecutar `latexmk -pdf MathCurriculum.tex` produce un PDF correctamente paginado con una tabla de contenido y todas las referencias cruzadas resueltas. (Véase el Capítulo 7.)

10.3. Deseable: Comprensión Técnica

Estos elementos demuestran una comprensión funcional de las herramientas introducidas en los capítulos de infraestructura. Las respuestas escritas no necesitan ser extensas; dos o tres oraciones cada una son suficientes.

D1. ¿Cuál es el directorio de trabajo actual cuando usted abre por primera vez una terminal? Escriba el comando que utilizaría para navegar hasta la carpeta `curriculum/` dentro del repositorio clonado. (Véase el Capítulo 3, Sección 3.1.)

D2. ¿Cuál es la diferencia entre un entorno virtual de Python y la instalación global de Python? Proporcione un escenario en el contexto de este repositorio donde necesitaría un entorno virtual separado. (Véase el Capítulo 5.)

D3. ¿Por qué `git push` requiere un `git commit` previo? ¿Cuál es la diferencia conceptual entre las dos operaciones, y por qué el script `gh.bat` realiza ambas en secuencia? (Véase el Capítulo 8.)

D4. En $\text{\LaTeX}$, ¿cuál es la diferencia práctica entre $\text{\[...\]}$ y $\text{\$...\$}$? ¿Qué forma debería usarse en los nuevos archivos fuente del currículo y por qué? (Véase el Capítulo 7, Sección 7.4.)

D5. ¿Qué hace el script `regen.py` cuando se ejecuta para una unidad de curso determinada? Describa la entrada que lee, la salida que produce y el papel del diccionario `ASSESSMENTS` en el control de la salida. (Véase la carpeta `Script/` del repositorio y la documentación asociada.)

D6. Explique el esquema de etiquetas utilizado para las referencias cruzadas en los archivos fuente del currículo. ¿Cuál es el formato de una etiqueta de sección, cómo se deriva el mnemónico de cuatro caracteres y cómo se calcula el sufijo hash? (Véase la documentación de la canalización del currículo.)

10.4. Opcional: Infraestructura Local de LLM (Ollama)

Estos elementos se aplican solo a los colaboradores que instalan y configuran la infraestructura local del modelo grande de lenguaje Ollama. No son obligatorios para la autoría del currículo ni para el mantenimiento del repositorio.

O1. Ollama está instalado y `ollama list` devuelve al menos un modelo en la terminal. (Véase el Capítulo 9, Sección 9.4.)

O2. `connectivity_test.py` aprueba todas las verificaciones (protocolo de enlace TCP, lista de modelos, inferencia). (Véase el Capítulo 9.)

O3. Explique el equilibrio entre el tamaño de la ventana de contexto y el consumo de memoria en Ollama. ¿Por qué `OLLAMA_NUM_CTX=8192` es un valor predeterminado seguro para modelos grandes en hardware con VRAM limitada? (Véase el Capítulo 9, Sección 9.6.)